

目录

快速上手	4
一、好搭 AI 派介绍	4
二、离线编程软件	4
三、下载方式	6
四、程序运行	9
五、程序保存	13
基础操作	15
一、界面布局	15
二、文件目录	16
三、 充电	17
四、 添加扩展	17
外设接口使用	18
一、GPIO	18
二、 ADC	22
三、 PWM	23
一、传感器	25
二、执行器	27
三、显示器	30
音频处理	33
一、录音器的使用	33
二、播放器的使用	36
语音 AI	39
一、 在线语音合成	39
二、 在线语音识别	41
三、文本翻译	42
四、大语言模型	43
AI 视觉算法	49
一、 摄像头与算法基础	49
二、 AprilTag 检测	52

三、色块检测	55
四、颜色识别	56
五、人脸识别	60
六、物体识别	63
七、二维码检测	68
八、车牌识别	69
九、人流计数	72
十、黑线检测	73
十一、图像分类	75
十二、目标检测	77
十三、姿态检测	78
十四、文字识别	79
物联网	83
MQTT	83
OpenCV	90
一、图像处理	90
二、视频处理	95
Pygame 交互界面	98
一、窗口	98
二、表面	99
三、事件	101
四、字体	103
五、图片	105
六、绘制	106
七、声音	107
八、定时器	108
九、音乐	109
智能博物 2025	113
一、比赛规则	113

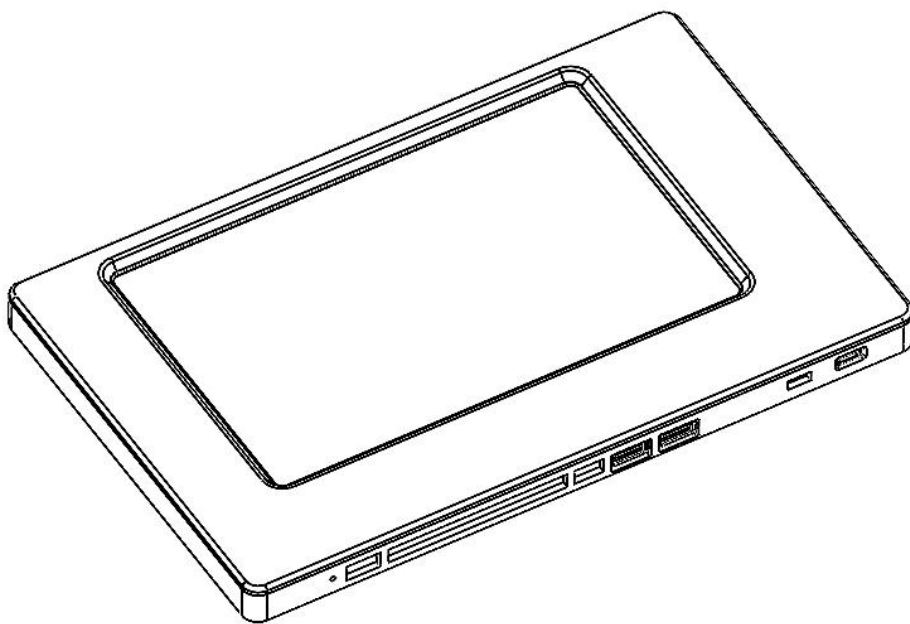
二、 解决方案.....	122
常见问题解决方法.....	133

快速上手

一、好搭 AI 派介绍

好搭 AI 派是一款人工智能应用开发平板，8 核处理器，6tops 高算力 NPU，不仅在多媒体应用、图像处理、人工智能运算中表现优异，同时兼容各类外接传感器、执行器，采用大小核异构架构实现高性能 AI 任务处理和外设控制。

该主控拥有强大的人工智能功能，包括语音识别、语音合成、文本翻译、自然语言处理、数十种视觉算法、内置 AI 训练模型等，支持图形化编程和 Python 编程，适用于各类人工智能教学应用。



二、离线编程软件

2.1 下载与安装

好搭 Block 离线软件的下载地址为：

http://www.haohaodada.com/new/art_show.php?id=220



一、好搭Block 2025.0.7【win7及以上系统】下载渠道选择

[渠道一：官网下载](#) [渠道二：语雀平台下载（不限速、推荐）](#)

二、好搭Block【国产系统】下载

[好搭Block国产系统下载](#)

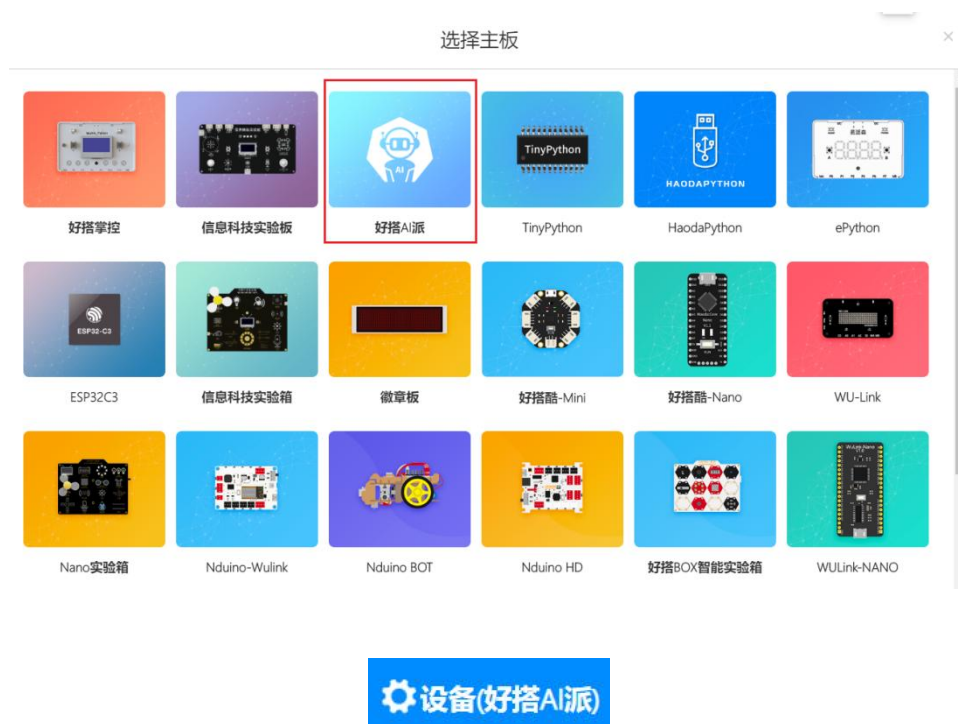
好搭Block介绍视频。好搭Block无缝对接好搭在线平台，支持好搭账号管理，支持好搭酷、徽章板、WU-Link、WULink-Nova、Nova HD、Nova Bot、Arduino-UNO、HaodaBox硬件离线环境下编程，并可以查看案例、上传作品，轻松保存程序。适用win7以上32位、64位操作系统。具体使用说明请点击 [好搭Block使用说明](#)。

根据电脑系统下载完最新版本软件后，根据提示逐步进行安装即可。注意需要将提示的驱动全部进行安装。



2.2 选择主板

安装完成后，打开软件，页面会自动弹出主控板选择对话框，点击选择**好搭AI派**。如果没有弹出，也可以点击右上角的设备按钮进行主控板的切换。



2.3 编程界面



菜单栏：包含了软件使用过程中的一些基本功能，例如程序保存、截图、连接、运行等各类功能。学习使用菜单栏，会使我们在编写程序的过程中如虎添翼。

指令区：包含了各种分类的指令，可以直接拖至图形化编程区使用。

图形化编程区：按照特定逻辑组合指令，编写出各种功能的程序。

字符代码区：在编写图形化编程的同时，代码区会出现对应的 python 代码。

范例代码：有各类基础程序，可以直接打开使用。

右上角更多：包含编程手册、驱动安装等功能

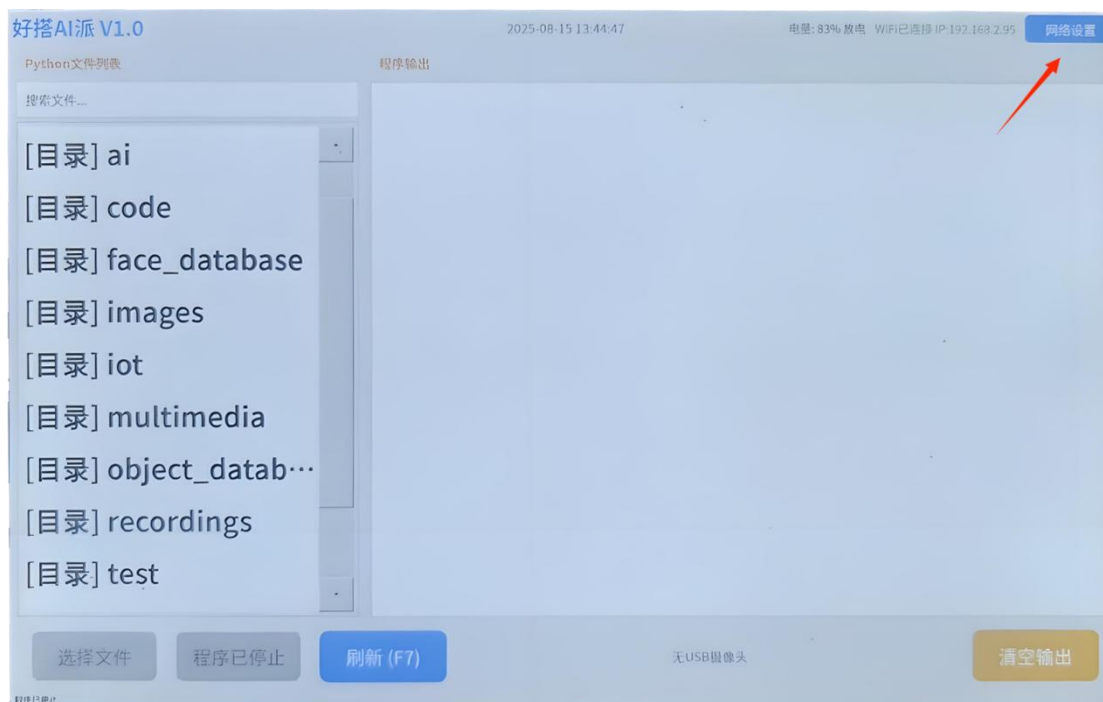
三、下载方式

好搭 AI 派一般推荐使用 IP 网络连接方式进行程序下载，也可使用 USB 方式进行连接并下载程序。

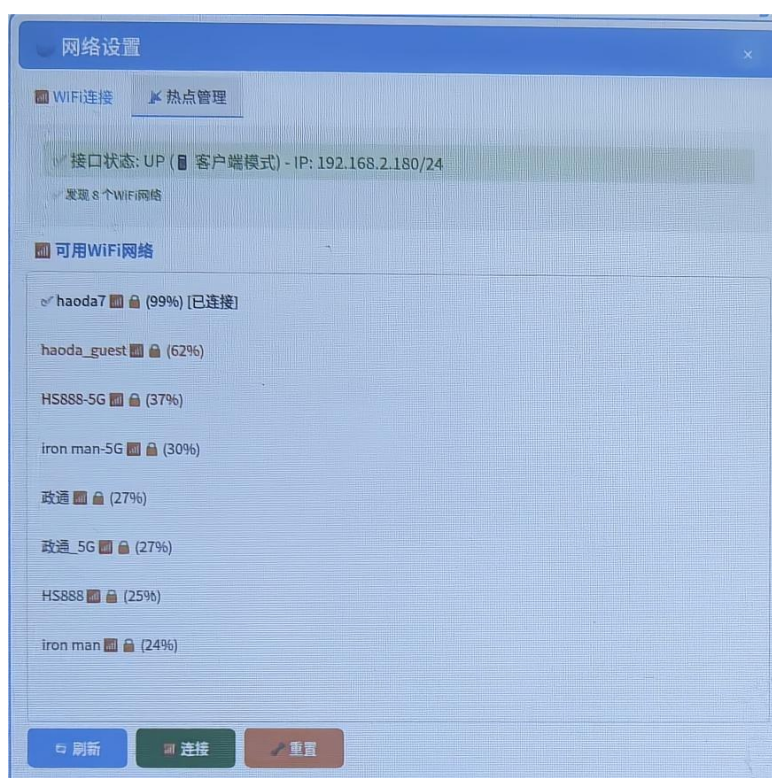
由于好搭 AI 派的许多功能都需要连接网络，所以在正常情况下，推荐用户能够在安全可靠的网络情况下进行使用，并使用 IP 网络进行连接。

3.1 IP 网络连接

点击好搭 AI 派右上角的【网络设置】按钮。



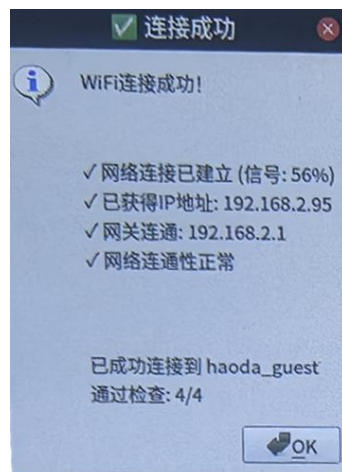
点击后会出现可以连接的 WiFi 网络。（如果是新设立的网络，需等待数秒，才会在界面中刷新出来；右侧热点管理是指使用好搭 AI 派开启一个热点，而非热点连接到好搭 AI 派。若使用热点进行连接，请等待 WiFi 连接部分刷新）



选择需要连接的 WiFi，点击连接按钮。注意，主控开机后会自动连接之前连接到的 WiFi。这里我以连接 haoda_guest 为例进行连接。



与常规连接 WiFi 方法一致，输入 WiFi 密码，点击确认按钮即可。
正常连接 WiFi 提示如下。



请确保好搭 AI 派和编程电脑使用同一个网络。

点击软件右上角的【未连接】按钮，选择 IP 网络连接，如下图所示：



点击后界面会出现一个提示框，要求输入好搭 AI 派的 IP 地址；此时可以查

看主控右上角，得到一个 IP 地址，例如 192.168.2.140。那么则在下方图中的四个方框中分别输入 192、168、2、140 并点击【确认】按钮，无需输入【.】。

请输入IP地址【IP地址显示在好搭AI派右上角】×

0

0

0

0

每段输入 0-255 之间的数字

取消 确定

+连接后，右上角界面会变成【IP 设备（1）】，如下图所示：



四、程序运行

4.1 打开范例程序

打开范例代码【1-外设接口使用 5.串口打印】。

这一段程序下载运行后，会在好搭 AI 派的屏幕上显示打印的消息。



4.2 运行下载

打开范例代码后，无需连接各类数据线，IP 网络连接后，直接点击软件右上角的【运行】按钮即可。程序会默认下载到【根目录】。

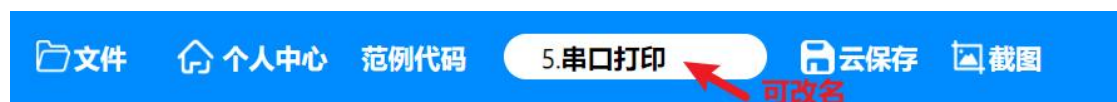
文件已保存到【根目录】
若要保存到其它文件夹，
请点击【文件管理】勾选该文件夹

我们可以在好搭 AI 派的界面中，找到这个程序：

我们也可以点击软件右上角的文件管理，找到刚刚下载的这个范例程序：



如果需要更改下载的程序名称，可以在软件左上角，云保存左侧的文本框中更改程序名称，如下图所示：

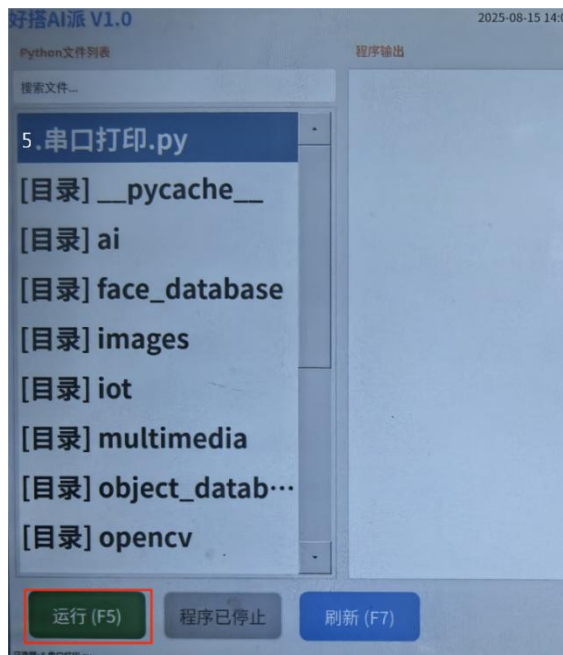


4.3 运行效果

找到刚刚下载的程序，选择刚刚下载的程序。

选中程序后，发现好搭 AI 派左下角的运行按钮变为绿色。

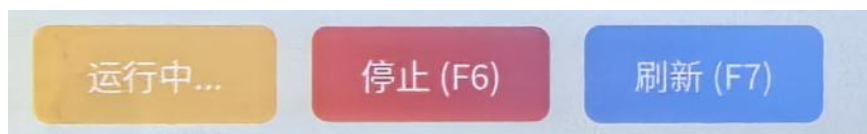
点击左下角【运行】按钮，如下图所示：



程序运行后会在右侧显示打印的串口消息 Hello world! 和 Hello haohaodada, 并显示[成功] 程序运行完成。

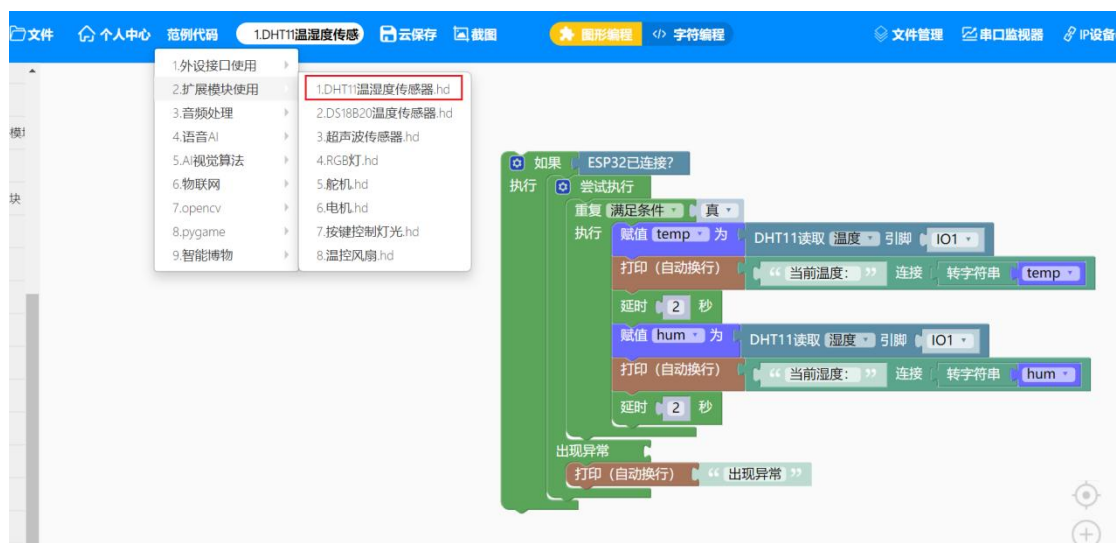
运行后左下角如图所示，可以点击【停止】按钮停止当前程序。

如果程序从好搭 Block 软件下载后没有显示，则可以点击【刷新】按钮。



再次尝试其他程序，进行巩固练习。

这次选择打开范例代码【1-外设接口使用 1.DHT11 温湿度传感器】。



将 DHT11 温湿度传感器连接到好搭 AI 派的 IO1 引脚。如果刚刚已经进行过 IP 网络连接，此时直接点击软件右上角【运行】按钮，运行下载即可。

将好搭 AI 派右下角的开关拨动到左侧，确保外设正常连接。

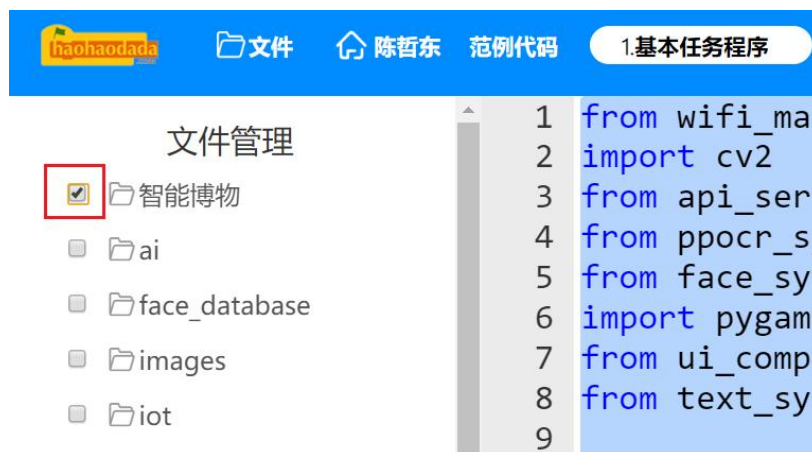
下载后，选择程序，点击【运行(F5)】，好搭 AI 派就会显示当前温度和湿度。



4.4 文件管理

如果需要下载到某个文件夹中，在点击软件右上角的【运行】按钮前，先点击【文件管理】按钮，勾选要下载到的文件夹，再点击【运行】按钮。

如果你想删除某个程序，则可以勾选程序，点击文件管理左下角的【删除】按钮。注意请不要把整个文件夹删除。



需要注意的是，智能博物比赛中的基本任务程序，必须下载到【智能博物】文件夹中。每次下载前，请点击文件管理，勾选对应的文件夹。

五、程序保存

5.1 本地保存

根据需要修改作品名称，然后在菜单栏中的个人中心中，选择保存（图形文件）或项目另存为，选择保存路径即可。



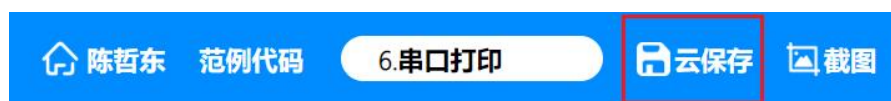
5.2 云保存

登录账号

需要先登录自己的好好搭搭账号，在个人中心中登录，如果没有账号可以免费注册



登录成功后，点击云保存即可保存在自己的账号中。



查看保存项目

在菜单栏的项目中心中-我的项目，即可查看刚才保存的文件。项目还可以分享出来。分享后的项目，所有人都可以在最新项目中进行查看。

官方例程

最新项目

我的项目

请输入作品名称

项目 陈哲东

新建项目

打开项目

保存 (图形文件)

项目另存为

项目中心

作业中心

班级管理



播放视频与音频

🕒 2025-08-04

 陈哲东

分享



按键大模型对话

🕒 2025-08-02

 陈哲东

分享

基础操作

一、界面布局

好搭 AI 派的界面包含以下部分：

①左侧的文件目录区，可以选择相应的程序

②左下角的绿红蓝三个按钮：**【运行】**按钮可以运行当前程序；**【停止】**按钮可以停下当前正在运行的程序；**【刷新】**按钮用于程序下载到好搭 AI 派或者在好搭 Block 使用文件管理修改文件后，刷新界面显示。

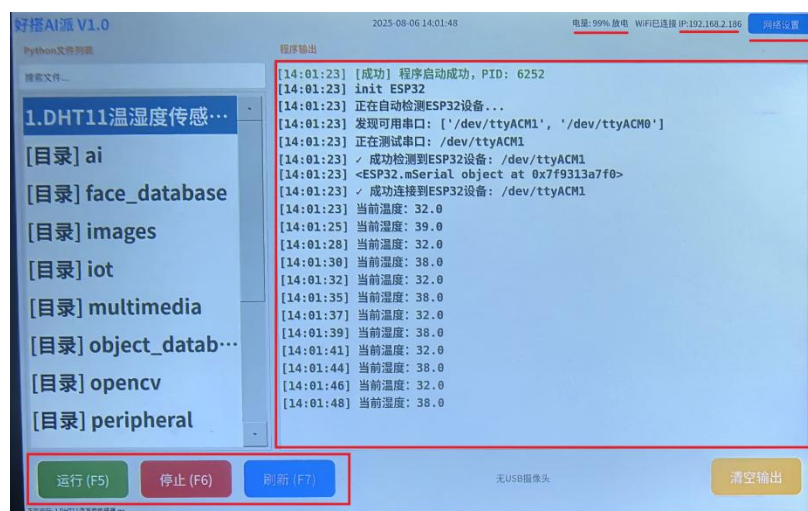
③右上角有**【网络设置】**按钮，可以修改网络。修改网络后，IP 地址会改变，用于 IP 网络连接。

④IP 地址左侧，有电量显示，并且会显示**充电/放电**状态。可以通过这个状态观察是否正在充电。

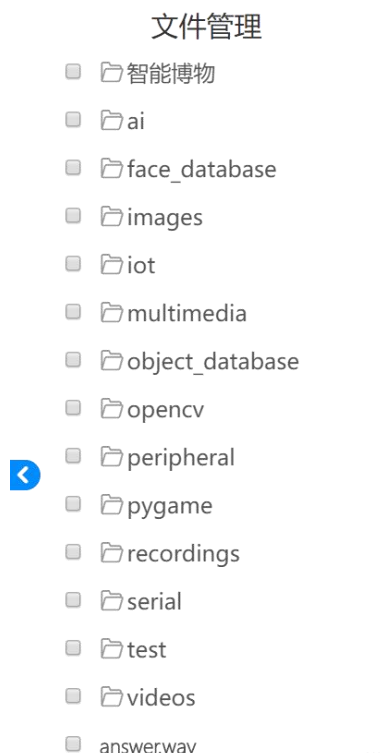
⑤屏幕右侧中央是消息显示区。所有程序运行的结果和串口消息都会在这里显示，是我们获取程序运行进度和情况的主要信息来源。

⑥屏幕右下角是**【清空输出】**按钮，可以清空消息显示区。

⑦屏幕正下方会显示摄像头连接状态，如下图就是摄像头未连接的状态。



二、文件目录



我们可以在好搭 **Block** 的软件管理处，或者在好搭 **AI** 派左侧，看到对应的文件目录。其中整个目录称作【根目录】，例如最下方的 **answer.wav**，就处于【根目录】下。【根目录】中包含众多文件夹，每个文件夹各有相应的功能。推荐程序下载到对应的文件夹中，好搭 **AI** 派的界面会更加清楚整洁。

【**智能博物**】文件夹包含了关键 **py** 文件和其他文件，智能博物比赛的基本任务程序必须下载到该文件夹中才能正常运行。

【**images**】文件夹主要用来放置图片。图片支持 **png**、**jpg**、**jpeg**、**bmp** 等常见格式。

【**recordings**】文件夹主要用来存放音频，包括需要播放的音频、录制的音频、语音合成生成的音频等。音频支持 **mp3**、**wav** 等常见格式。

【**videos**】文件夹主要用来存放视频文件。视频文件支持 **mp4** 格式。

【**test**】文件夹主要用来存放测试程序。

良好的文件放置习惯可以帮助我们更好地学习和使用人工智能。

三、充电

好搭 AI 派内置 5000mAh 电池，满足设备一定时间的移动使用需求。当电量低于 10%时，好搭 AI 派如果没有正在使用，则会自动关机。请及时给好搭 AI 派充电。充电时请注意好搭 AI 派右下角的开关按钮，请将按钮拨动到右侧。

四、添加扩展

好搭 AI 派编写程序需要一些扩展库作支撑。

添加扩展

在官方库中加载所有扩展库。

第一次安装软件后，常用扩展库会自动加载。

智能博物 2025 主要是智能博物比赛基本任务相关指令；

扩展板外接包含外设接口及各类传感器、执行器、显示器设备的指令。

语音 AI 模块包含语音合成、语音识别、文本翻译、自然语言模型等功能；

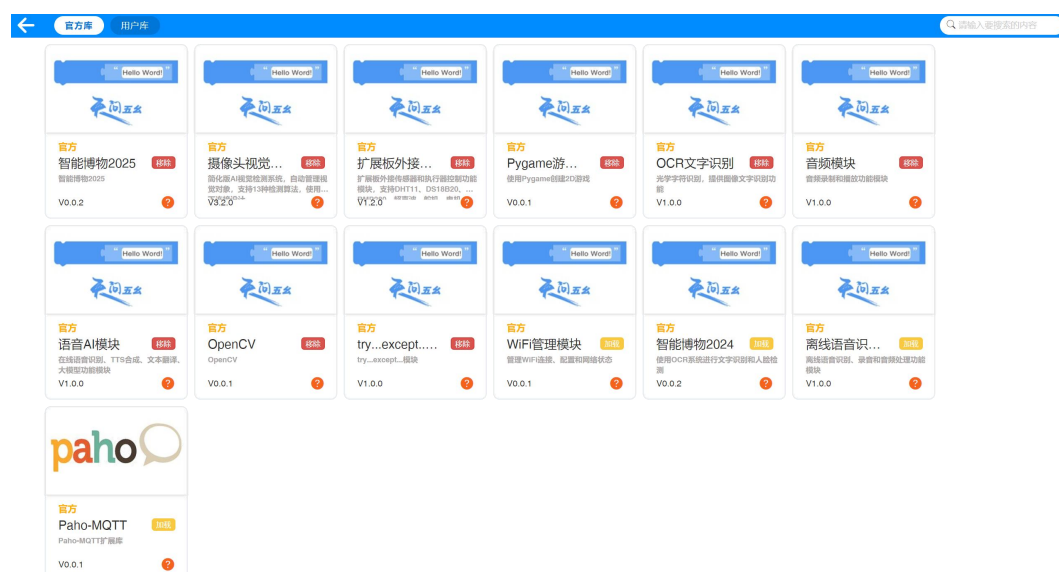
音频模块主要是音频的录制与播放；

摄像头视觉系统 V3 包含各类人工智能视觉识别算法；

Pygame 游戏模块包含界面设置、图像渲染、音频播放、输入管理等功能；

OpenCV 包括图片、音频、视频画面的处理相关的指令；

OCR 文字识别支持摄像头进行文字识别功能；



外设接口使用

好搭 AI 派提供了丰富的外设接口，包括 2 路电机接口，可用于连接和驱动电机；2 路 USB 接口，方便与其他设备进行数据通信；8 路 IO 接口，支持 GPIO、ADC、PWM 等功能，可灵活用于各种功能扩展。

一、GPIO

GPIO，是通用型数字输入输出接口的简称，可以通过软件配置设置引脚高低电平输出，也可以读取外部设备的电平状态，如 LED 控制、数字信号采集等。

好搭 AI 派的 8 个 IO 接口，IO1-IO8，均支持 GPIO 功能。

指令学习

ESP32已连接?

这条指令用于判断是否已经启用外设接口，通常和判断指令一起使用。好搭 AI 派右下角有开关，拨动到右侧是充电模式，拨动到左侧才会启用外设接口。

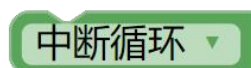
该指令只是为了逻辑更严谨判断报错信息，不影响程序本身。



这条指令是 `try...except` 指令，是异常处理机制的核心组件。`try` 指令一般包裹可能引发错误的代码块（如文件操作、网络请求、计算错误等），当出现异常时，立刻中断当前流程并提供配套的处理逻辑。点击蓝色小齿轮，还可以添加 `finally` 子句，防止资源泄露。一般异常后我们会打印异常信息。



这条指令是 `while True:`，重复执行指令。



这条指令可以让程序在特定条件下跳出当前循环。



当我们涉及到使用外接设备时，可以先编写这样一个框架，用于确保整个程序运行的稳定性。



这条指令可以读取 `IO1` 引脚的数字值。点击小倒三角下拉，可以选择 `IO` 引脚。单按键、限位开关、光电开关、磁感应模块等，都可以读取引脚数字值。

`GPIO` 包括 `IO1-IO8`。好搭 AI 派下方八个 `IO` 口，最左侧是 `IO8`，最右侧是 `IO1`。



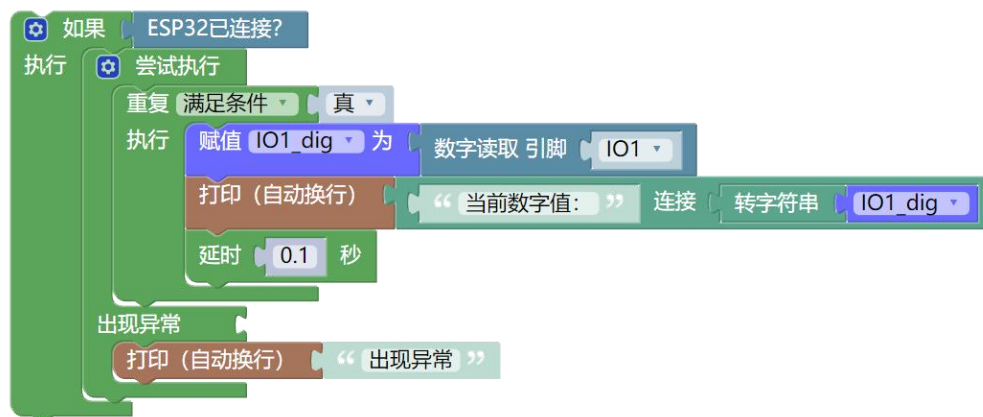
这条指令可以设置 `IO1` 引脚的数字值，设置高电平或低电平。例如 `IO0` 引脚连接 `LED` 灯，写入高电平，`LED` 点亮；写入低电平，`LED` 熄灭。

程序编写

示例 1：读取数字值

打开范例代码 模数输入输出-1.读取数字值。

将好搭 AI 派右下角开关拨动到左侧，在 IO1 引脚连接单按键模块。



```
程序输出
[16:47:46] 当前数字值: 0.0
[16:47:46] 当前数字值: 0.0
[16:47:47] 当前数字值: 0.0
[16:47:47] 当前数字值: 1.0
[16:47:47] 当前数字值: 1.0
[16:47:47] 当前数字值: 1.0
[16:47:47] 当前数字值: 1.0
[16:47:47] 当前数字值: 1.0
[16:47:47] 当前数字值: 1.0
[16:47:48] 当前数字值: 1.0
[16:47:48] 当前数字值: 1.0
[16:47:48] 当前数字值: 1.0
[16:47:48] 当前数字值: 1.0
[16:47:48] 当前数字值: 0.0
[16:47:48] 当前数字值: 0.0
[16:47:48] 当前数字值: 0.0
[16:47:49] 当前数字值: 0.0
[16:47:49] 当前数字值: 0.0
[16:47:49] 当前数字值: 0.0
[16:47:49] 当前数字值: 0.0
[16:47:49] [系统] 程序异常退出 (代码: 15)
[16:47:49] [系统] 程序已停止
```

当按键被按下，好搭 AI 派串口打印 当前数字值: 1.0

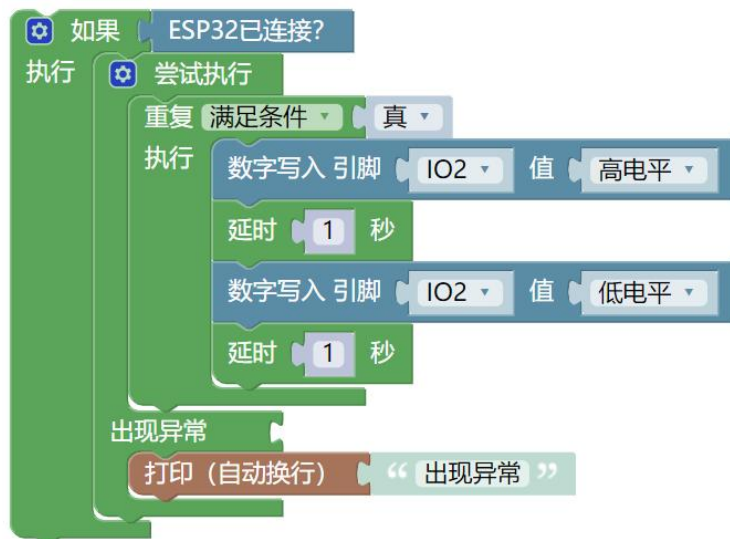
当按键松开时，好搭 AI 派串口打印 当前数字值: 0.0

每隔 0.1s 刷新一次。

示例 2：写入数字值

打开范例代码 模数输入输出-2.写入数字值。

将好搭 AI 派右下角开关拨动到左侧，在 IO2 引脚连接 LED 模块。

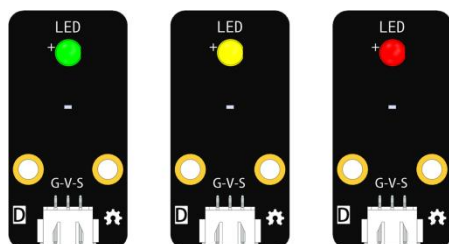
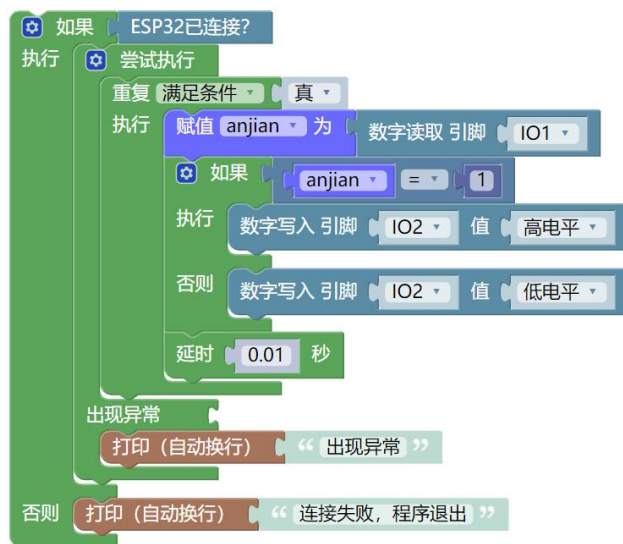


写入高电平后，LED 会点亮；写入低电平后，LED 会熄灭。
最终 LED 呈亮一秒灭一秒的闪烁状态。

示例 3：按键控制灯光

打开范例代码 传感与控制-7.按键控制灯光。

将好搭 AI 派右下角开关拨动到左侧，在 IO2 引脚连接 LED 模块，在 IO1 引脚连接单按键模块。



当按键被按下时，LED 灯点亮；

当按键松开后，LED 灯熄灭。

二、ADC

ADC，模数转换器，是一种将连续变化的模拟信号转换为离散的数字信号的电子元器件，是模拟读取的关键环节。

好搭 AI 派的 ADC 是 12 位，转换值范围为 0-4095。

指令学习



这条指令用于读取 IO1 引脚的模拟值。例如亮度传感器、声音传感器、土壤湿度传感器等，这类不带单位的传感器，一般都使用模拟读取指令。

程序编写

示例 1：读取模拟值

打开范例代码 模数输入输出-3.读取模拟值。

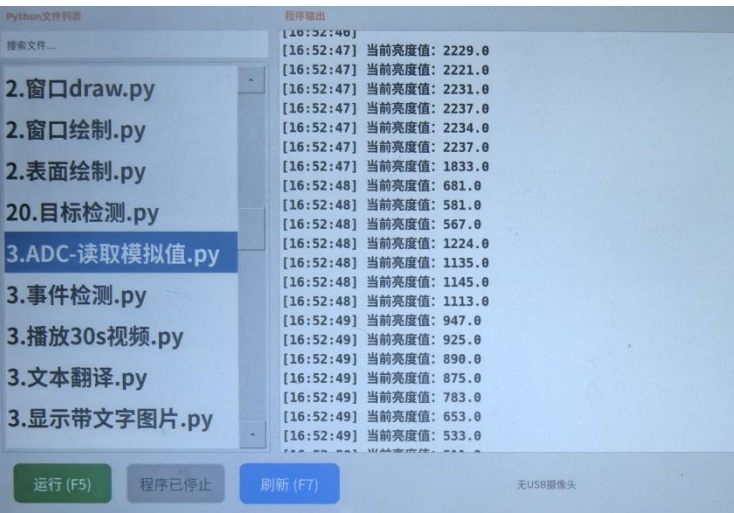
将好搭 AI 派右下角开关拨动到左侧，在 IO2 引脚连接亮度传感器。



我们以使用手机手电筒强光照射和自然光条件下进行对照：

自然光条件下的亮度模拟值为 500-1200;

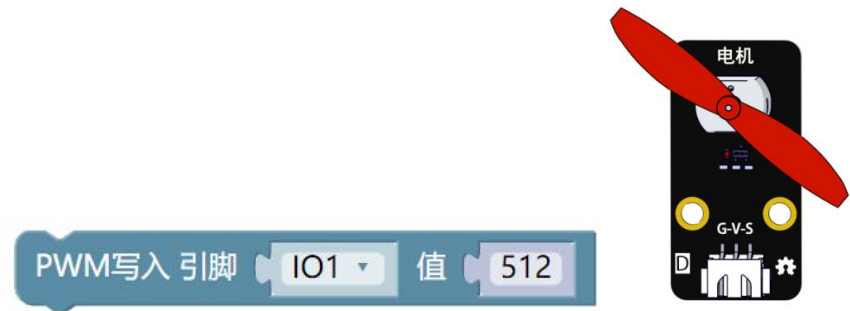
手机手电筒强光照射下的亮度模拟值为 2000+。



三、PWM

PWM（脉宽调制）是一种通过调节数字信号的脉冲宽度来模拟模拟量输出的技术手段，一般用于小风扇模块调速、LED 调光等。

指令学习



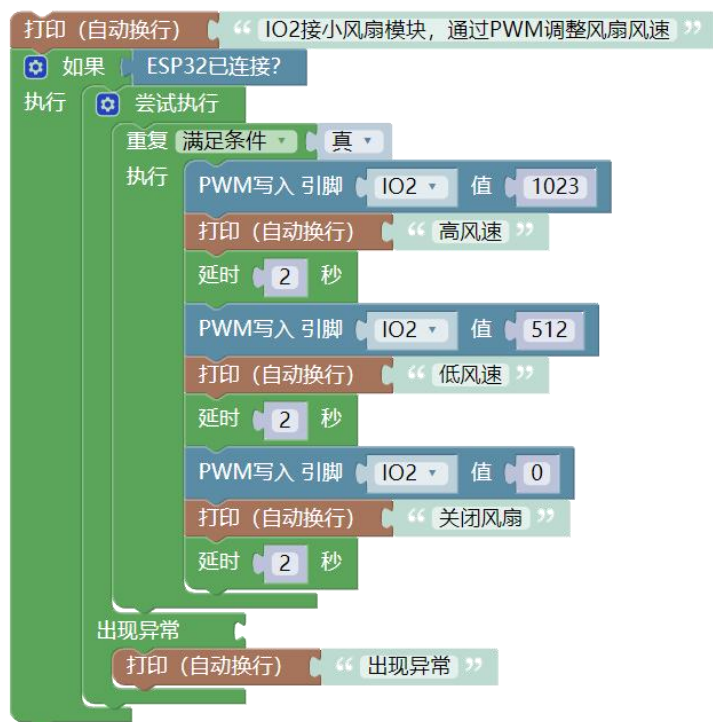
这条指令用于设置 IO1 引脚的 PWM，设置范围在 0-1023。例如小风扇模块，也称小电机模块，通过电机的转动来带动风扇叶的转动，就可以通过 PWM 控制来调整速度。

程序编写

示例 1：写入模拟值

打开范例代码 模数输入输出-4.写入模拟值。

将好搭 AI 派右下角开关拨动到左侧，在 IO2 引脚连接小风扇模块。



程序运行后，小风扇模块会在高风速、低风速、停止三种状态间切换，切换间隔为 2s。

一、传感器

指令学习



这条指令可以读取 IO1 引脚的 DHT11 温湿度传感器的温度值和湿度值。
DHT11 传感器，可以用来同时测量温度和湿度，温度的测量范围为 0-50℃，精度 $\pm 2^{\circ}\text{C}$ ；湿度的测量范围为 20-90%RH，精度 $\pm 5\%\text{RH}$ 。



这条指令可以读取 IO1 引脚的 DS18B20 防水型温度传感器的温度值。这种温度传感器可以用来对水体温度进行定量的检测，固有测温分辨率 0.5°C 。支持多点组网功能，多个 DS18B20 可以并联在一根三线上，实现多点测温。当模块与好搭 AI 派连接使用时，需要再额外连接上拉扩展模块。DS18B20 温度传感器检测获取的数值精度在小数点后四位。



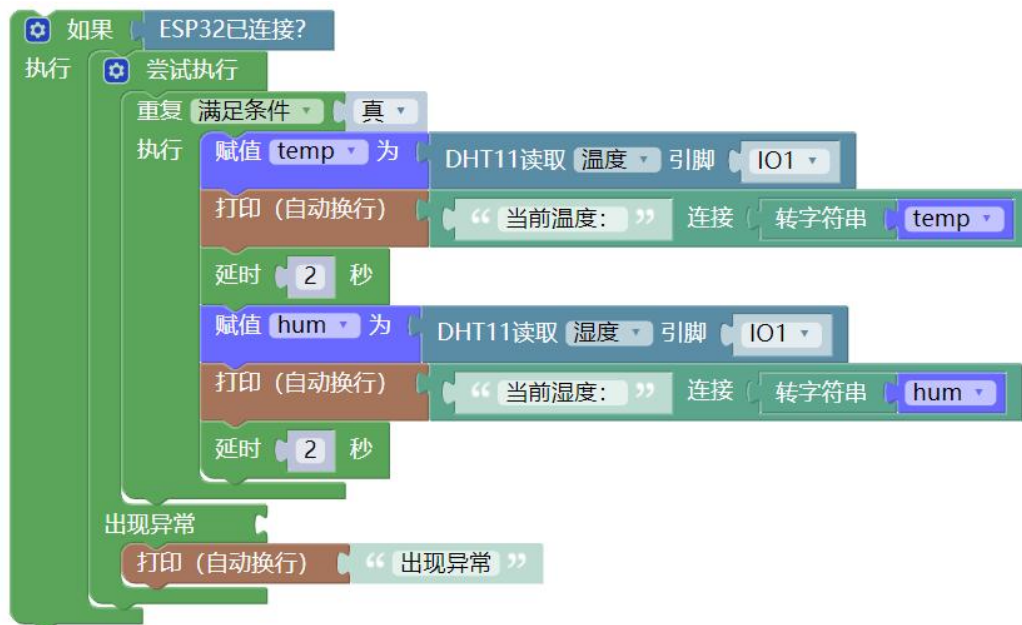
这条指令可以读取 IO1 引脚超声波传感器的数值。超声波传感器的有效探测距离范围为 4-400cm。该模块内部将发出 8 个 40kHz 周期电平并检测回波；一旦检测到有回波信号则输出回响信号。回响信号的脉冲宽度与所测的距离成正比。

由此通过发射信号到回响信号的时间间隔可以计算得到距离。

程序编写

示例 1：DHT11 温湿度传感器

打开范例代码 传感与控制-1.DHT11 温湿度传感器。



在 IO1 引脚连接 DHT11 温湿度传感器模块。

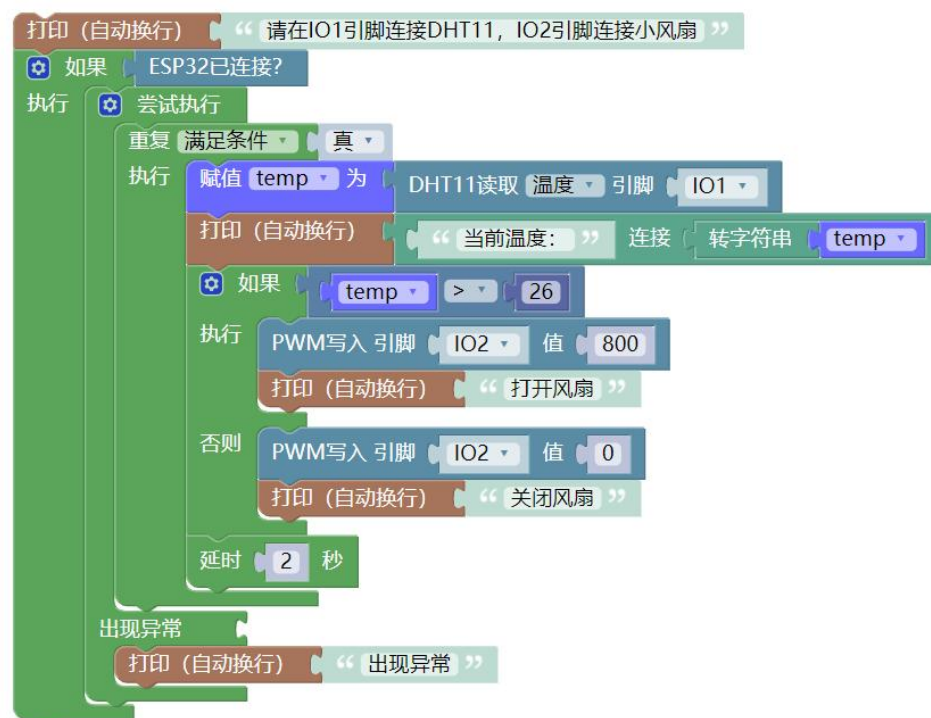
好搭 AI 派的屏幕上会轮流刷新温度值和湿度值。

其中温度值的单位是℃，湿度值的单位是%RH。

示例 2：温控风扇

打开范例代码 传感与控制-8.温控风扇。

在 IO1 引脚连接 DHT11 温湿度传感器，在 IO2 引脚连接小风扇模块。



当温度值高于 26 摄氏度时，风扇就会打开；当温度值低于 26 摄氏度时，风扇就会关闭。尝试用风扇对准 DHT11 温度传感器，可以观察到温度下降。

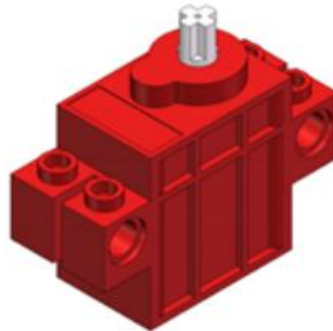
二、执行器

指令学习



这条指令可以控制灰色舵机角度的转动，转动范围在 0-180 度。

舵机是一种位置（角度）伺服的驱动器，适用于那些需要角度不断变化并可以保持的控制系统。目前，在高档遥控玩具，如飞机、潜艇模型、遥控机器人中已经得到了普遍应用。套件中一般配备灰色的乐高兼容专用舵机。



这条指令可以控制红色电机的转动，电机转速在-1023-1023。

正装电机，速度大于 0 正转；

反装电机，速度小于 0 正转；

可以根据实际情况选择对应指令。

电机，也叫“马达”，是依据电磁感应原理将电能转换为机械能的一种装置，很多常用电器和机械的动力源都是各种各样的电机。套件配备的红色专用电机就是一种直流电机，能够将输入的直流电能转换为机械能，同时兼容乐高积木，可以带动积木进行旋转。

程序编写

示例 1：舵机转动

打开范例代码 传感与控制-5.舵机。

在 IO1 引脚连接灰色舵机模块，并将乐高件连接在舵机上，方便观察。



舵机会在 0、90、180 三个角度之间来回切换。

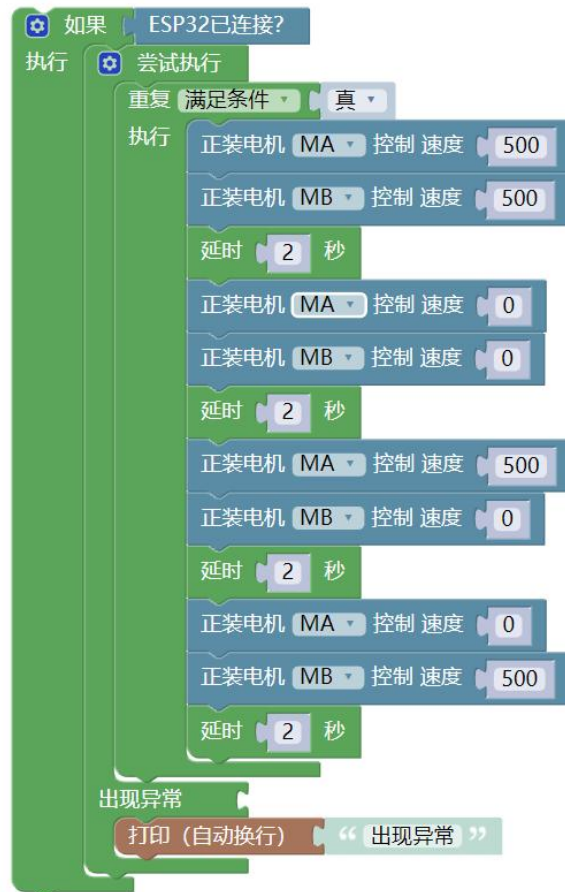
示例 2：电机转动

打开范例代码 传感与控制-6.电机。

电机的接口不在 8 个 IO 口，而是旁边的 MA、MB 电机端口。

将两个红色电机连接到 MA、MB 引脚，并将乐高件连接在电机上，方便观察两个电机的转动方向和速度。

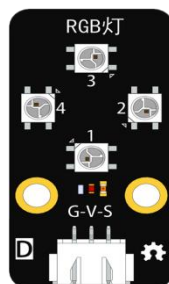
程序运行后，可以看到，两个电机的转速变化。用户可以尝试自行修改程序，将速度更改为负值，再次观察电机转动的变化。



三、显示器

WS2812 是一种集成了电流控制芯片的低功耗的 RGB 三色灯。其中 R 代表红色，G 代表绿色，B 代表蓝色，可实现 256 级亮度显示，支持 16,777,216 (256×256×256) 种颜色的全真色彩显示。

这种 RGB 灯不仅包括 4 灯灯珠的小模块，还包括 7 灯、60 灯的长条灯带。灯带的初始序号从 0 开始，可以自由设置任意一个灯珠的颜色和亮灭。



指令学习

WS2812初始化 引脚 LED总数量

这条指令可以初始化 RGB 灯模块，设置引脚并设置灯珠的总数量。

WS2812设置单个LED 引脚 LED序号 红 绿 蓝

这条指令可以设置某个灯珠的具体颜色。其中序号从 0 开始，一直到灯珠总数量-1。红绿蓝的 RGB 值取值范围在 0-255 之间。如上方的指令，就是设置序号 0，也就是第一个灯珠，颜色为红色。

WS2812设置所有LED 引脚 红 绿 蓝

这条指令可以设置所有灯珠都为同一种颜色。

WS2812清除所有LED 引脚

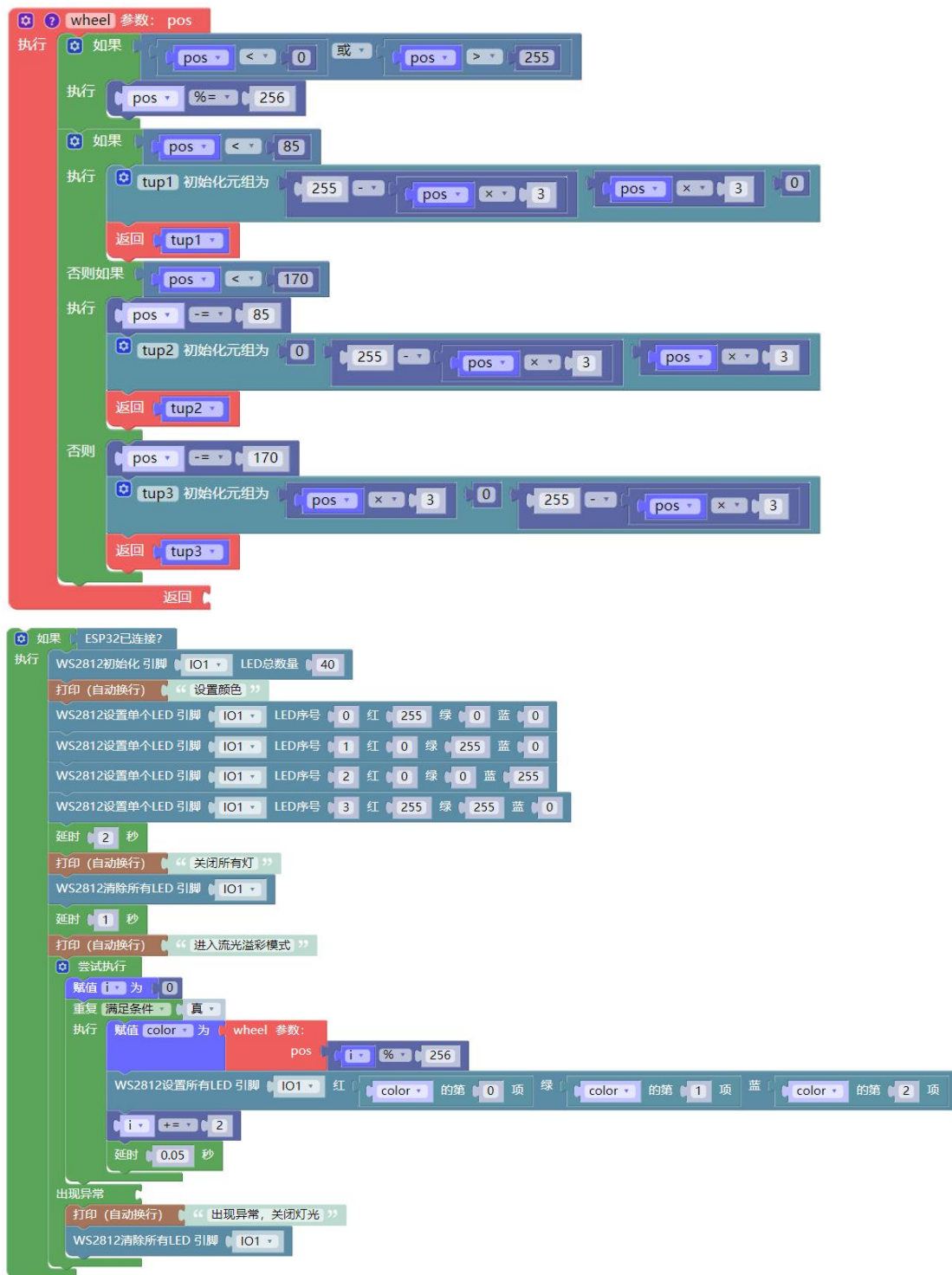
这条指令可以设置所有灯珠都熄灭。

程序编写

示例 1：流光溢彩

打开范例代码 传感与控制-4.RGB 灯。

在 IO1 引脚连接 RGB 灯带。注意，连接时需要额外连接上拉扩展模块。



RGB 灯灯带，前四个灯珠一开始会分别显示红、绿、蓝、黄四种颜色；2s 后，灯光会关闭；再过 1 秒后，会进入流光溢彩模式。

音频处理

音频是人类在日常生活里面使用非常广泛的信息获取方式，它的本质就是声音信息的采集、存储和回放。好搭 AI 派自带录音器，并配备双扬声器，提供良好的音频播放效果。接下来我们通过学习录音器和播放器，了解如何进行音频的录制与播放。

一、录音器的使用

录音基础知识

录音器，泛指具有录制声音功能的设备或系统，其核心在于捕捉声波并转换为可存储的电信号。

麦克风是录音器的核心组件，负责将声音信号转换为电信号；而完整的录音系统还需声卡等辅助模块处理信号以实现高质量录音。

好搭 AI 派的录音器支持 8、16、44.1、48、96kHz 的采样率，支持单声道与立体声。采样率越高，录制的音频质量越高。

用户可以将录制好的音频保存到对应的文件夹中。

指令学习

赋值 recorder 为 初始化录音器 采样率 16000 声道数 1

这条指令用于初始化录音器并命名 recorder，设置录音器的采样率和声道。其中采样率可以填写 8000、16000、44100、48000、96000。采样率越高，录制的音频质量越高。声道数可以填写 1 或 2，其中 1 表示单声道，2 表示立体声。

录音器 recorder 设置默认输出目录 目录路径 recordings

这条指令可以设置音频文件保存的目录。好搭 AI 派自带文件夹 recordings，哪怕不使用该条指令，录制的音频文件也会默认保存到该文件夹中。如果希望保

存到其他地方，请使用该指令对路径进行修改。



这条指令可以对音频进行录制，录制时间为 5s，文件路径 file_path 为 recordings/test_5sec.wav。指令执行后即开始录制，5s 后自动停止录制。



这条指令可以让录音器在自由时点开始录制。



这条指令可以让录音器在自由时点停止录制，并把音频数据存储到变量 audio_data 中。



这条指令可以让录音器把音频数据保存并生成文件，文件路径 file_path 为 recordings/manual_recording.wav。

程序编写

示例 1：录制 5s 音频

打开范例代码 音频处理-1.录音 5s。

将程序下载到好搭 AI 派中并运行。



实际演示效果如下图所示：

```

[15:42:55] === 示例1: 录制5秒音频 ===
[15:42:55] 采样率可以取8、16、44.1、48、96kHz
[15:42:55] 声道数1表示单声道, 2表示立体声
[15:42:55] 适合预设时长的录音场景, 完全自动化, 指令开始即录音
[15:42:55] 可用的音频设备:
[15:42:55] 0: rockchip,es8388-codec: dailink-multicodecs ES8323.7-0011-0
(hw:0,0) (输入声道: 2)
[15:42:55] 1: sysdefault (输入声道: 128)
[15:42:55] 2: samplerate (输入声道: 128)
[15:42:55] 3: speexrate (输入声道: 128)
[15:42:55] 4: pulse (输入声道: 32)
[15:42:55] 5: upmix (输入声道: 8)
[15:42:55] 6: vdownmix (输入声道: 6)
[15:42:55] 8: default (输入声道: 32)
[15:42:55] 开始录制 5 秒音频...
[15:42:55] 录音中... 请说话
[15:43:00] 录音完成!
[15:43:00] 音频文件已保存: recordings/test_5sec.wav
[15:43:00] 文件信息: 80000 样本, 5.00 秒
[15:43:00] 录音成功! 文件保存在: recordings/test_5sec.wav

```

音频录制后会保存在对应的文件夹中。我们可以通过文件管理来查看录制的音频, 也可以通过稍后学习的播放器, 来播放刚刚录制的音频。

示例 2: 按键控制录音

打开范例代码 音频处理-4.按键控制录音。在 IO1 引脚连接单按键模块, 将好搭 AI 派右下角的开关拨到左侧。(按键知识请回顾 GPIO)

将程序下载到好搭 AI 派中并运行。



实际演示效果：按键每 10ms 进行一次检测，当按键被按下后，查看好搭 AI 派的信息显示界面，当显示“录音中，请说话”后，可以说话开始录音，按键松开后结束录音。录音文件会保存到 recordings/test_button.wav。

需要注意的是，按下按键后，请稍作等待，再开始说话，以此确保录音内容的完整性。

二、播放器的使用

音频播放基础知识

音频播放器是一种用于存储、管理和播放数字音频文件的设备或软件，结合扬声器将数字信号转换为可听的声音。

好搭 AI 派的音频播放器，支持播放 WAV 格式与 MP3 格式的音频文件，配合双扬声器，可以提供良好的音频播放效果。

好搭 AI 派的左上角，有长条型的音量调节按钮。

指令学习

赋值 player 为 初始化播放器

这条指令用于初始化播放器并命名为 player。

播放器 player 播放文件 文件路径 “ recordings/recording.wav ”

这条指令用于播放指定文件路径的音频。通常这条指令配合变量一起使用，如下图所示：

赋值 success 为 播放器 player 播放文件 文件路径 “ recordings/test_5sec.wav ”

播放器 player 清理资源

这条指令主要用于清理音频播放器内部的缓冲数据，清理缓冲队列，释放解码资源，重置播放状态（会中断播放进度），但不会涉及删除音频文件。

程序编写

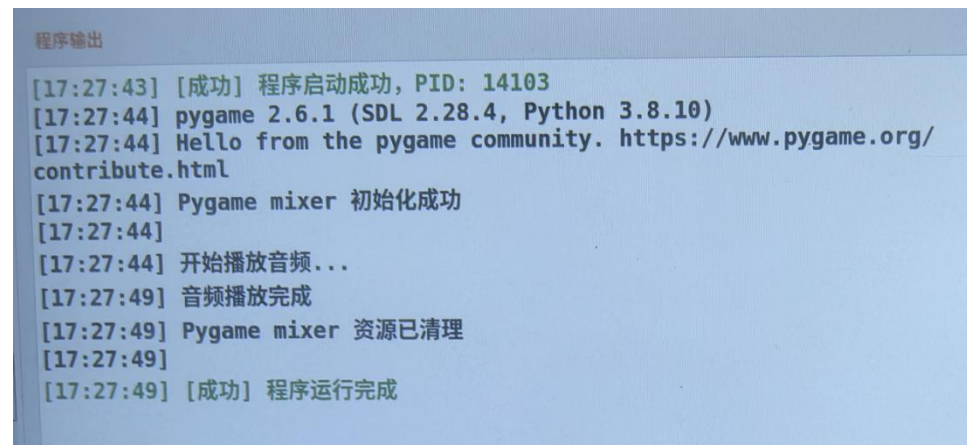
示例 1：音频播放

打开范例代码 音频处理-2.音频播放

之前运行了范例代码 音频处理-1.录音 5s，生成了一个名为 `test_5sec.wav` 的音频。现在我们就来播放它。

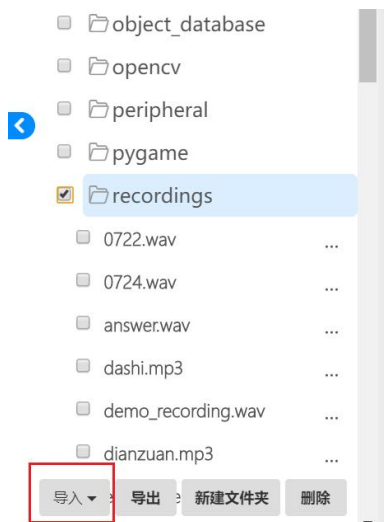


实际演示效果如下图所示：



当然我们也可以通过播放器，播放其他音频文件，而非刚刚录制的音频。比如说我们想要播放一段新闻的音频，可以提前将音频文件存储到文件夹中。

点击好搭 Block 右上角的文件管理，勾选 `recordings` 文件夹，点击左下角的导入按钮，将需要播放的新闻音频文件导入到文件夹中。注意，音频文件的命名必须为英文，格式为 `wav` 或 `mp3`。导入后，修改程序中的文件名称，即可播放想要播放的音频文件。

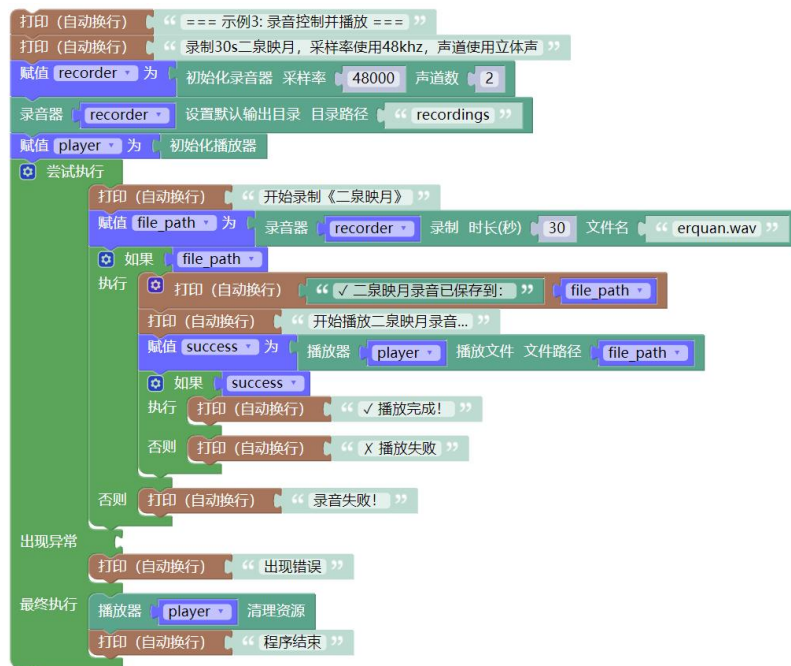


示例 2：音频录制并播放

打开范例代码 音频处理-5.音频录制并播放。

我们也可以分开录制音频、播放音频，也直接录制音频并进行播放。

播放器的文件路径，也可以直接填充变量 `file_path`。（多个音频注意区分）



语音 AI

语音 AI 作为人工智能领域的核心技术之一，已渗透到现代生活的方方面面。它通过多种技术模块的协同作用，实现了人机交互的智能化与自然化。接下来，我们通过对语音合成、语音识别、文本翻译、大语言模型等内容进行学习，深化对语音 AI 的理解，并编写程序进行一定的人工智能应用。

一、在线语音合成

在线语音合成（TTS，Text to Speech）是一种基于云端服务的实时技术，能够将文字信息转换为自然流畅的人声语音。

好搭 AI 派的在线语音合成功能，可以将文本信息通过 TTS 合成音频，并通过播放器进行播放，支持播放中文和英文。

指令学习

赋值 `voice_api` 为 初始化语音API API地址 `“http://www.haohaodada.com/project/voiceAI/ApiZNB...”`

这条指令用于初始化语音 AI 功能的 API 地址。所有语音 AI 功能都需要用该指令。与之相关的还有下方的 API 客户端认证指令。需要注意的是，好搭 AI 派需要联网后，方可使用该功能。下文语音 AI 功能均默认已联网，不再赘述。

赋值 `token_result` 为 API客户端 `voice_api` 获取认证 用户名 `“username”` 密码 `“*****”`

这条指令需要输入通过认证的好好搭搭账号。输入经过认证的用户名和密码就可以使用语音 AI 相关的功能。为保护用户隐私权益，密码输入后默认隐藏；鼠标在密码框内单击可以再次查看。账号如需认证，请联系售后人员。

赋值 `audio_data` 为 API客户端 `voice_api` TTS合成 文本 `“欢迎使用智能博物系统”` 保存路径(可选) `“recordings/output.wav”`

这条指令可以将文本转化成音频，并将音频文件保存到指定的文件夹中。一般我们会将音频文件保存到 `recordings` 文件夹方便管理，所以在保存路径上一般填写 `recordings/` 文件名称，文件名称一般使用英文或拼音。

程序编写

示例 1：语音合成

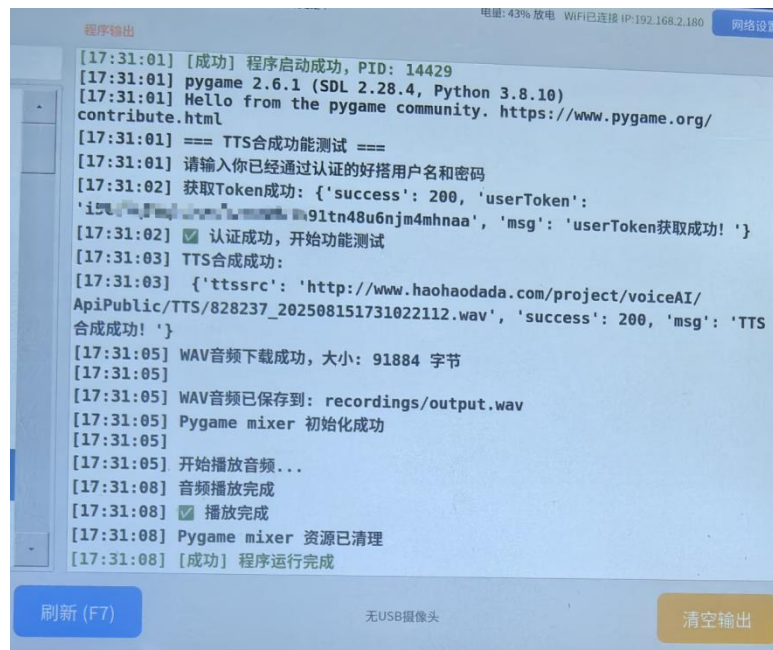
打开范例代码 语音 AI-1.语音合成。

修改用户名和密码，需要合成的文本，然后运行程序。

账号在通过认证后，通过 TTS 合成音频文件并播报。可以通过查看好搭 AI 派的打印信息来观测程序运行进度。



输出结果如下所示：



二、在线语音识别

ASR，自动语音识别技术，是指通过将人类语音转换为文字实现人机交互。在线语音识别是 ASR 技术的应用场景之一。

用户可以通过识别好搭 AI 派的音频文件，将音频转化成文本信息。

指令学习



这条指令用于识别指定文件路径的音频文件，并通过在线语音识别技术，将音频转化成文本，并赋值给变量 `recognition_text`。

路径中的文本连接指令，在【文本】类别指令中。

程序编写

示例 1：语音识别

打开范例代码 语音 AI-2.语音识别。

修改用户名和密码。

语音识别需要识别一段音频。用户可以预先录制一段音频，例如运行范例代码 音频处理-1.录音 5s，进行录音并生成文件 `recordings/test_5sec.wav`。

如果需要进行实时语音识别，请参考自然语言处理部分操作方法。

语音识别的内容会打印显示在好搭 AI 派上。



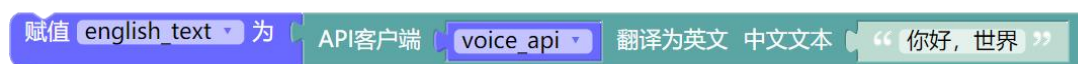
```
程序输出
[17:34:32] Hello from the pygame community. https://www.pygame.org/
contribute.html
[17:34:32] ===语音识别功能测试 ===
[17:34:32] 使用语音识别将音频转化成文字并打印出来
[17:34:32] 这里的音频使用提前录音好的文件，文件名为test_5sec.wav
[17:34:32] 具体可以先运行范例2. 录音与播放-1. 录音5s
[17:34:32] 请输入你已经通过认证的好搭用户名和密码
[17:34:33] 获取Token成功: {'success': 200, 'userToken':
'qcnd59la' + 'fgubvma9pjf2is5dmjr1', 'msg': 'userToken获取成功! '}
[17:34:33] ✅ 认证成功，开始功能测试
[17:34:33] 音频文件大小: 160044 字节
[17:34:33] 正在识别音频文件: recordings/test_5sec.wav
[17:34:37] 语音识别API调用成功:
[17:34:37] {"success":
200, "result": "\u4f60\u597d\u4e0b\u6765\u8981\u6d4b\u8bd5\u4e00
\u6bb5\u6587\u4ef6\u3002", "msg": "\u77ed\u8bed\u9684\u522b\u529
f\u4e00\u52c9"}
[17:34:37] API状态码: 200
[17:34:37] 识别结果: '你好，接下来要测试一段文件。'
[17:34:37] 消息: 短语语音识别成功!
[17:34:37] ✅ 识别成功: 你好，接下来要测试一段文件。
[17:34:37] 语音识别内容: 你好，接下来要测试一段文件。
[17:34:38] [成功] 程序运行完成
```

三、文本翻译

文本翻译，是指将一种语言的文本翻译成另一种语言的技术。

好搭 AI 派的文本翻译功能，支持将中文翻译成英文。

指令学习



这条指令用于将一串中文文本翻译成英文文本，并赋值给变量 `english_text`。

程序编写

示例 1：文本翻译

打开范例代码 语音 AI-3.文本翻译。

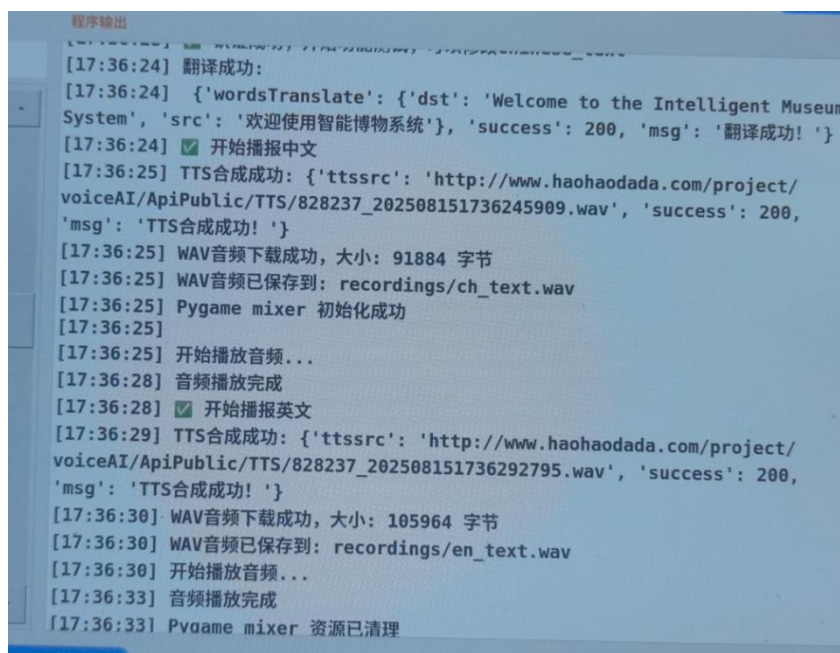
修改用户名和密码。

红框内就是需要翻译的文本和翻译指令。中文文本保存到变量 `chinese_text`，英文文本保存到变量 `english_text`。

接下来通过播放器，结合语音合成功能，分别播放中文文本和英文文本。



输出结果如下所示：



四、大语言模型

大语言模型（Large Language Model，简称 LLM）是指使用大量文本数据训练的深度学习模型，使得该模型可以生成自然语言文本或理解语言文本的含义。LLM 是自然语言处理 NLP 中的一种尖端分支技术，通过预学习通用语言模式实现文本生成、多轮对话等复杂功能。

好搭 AI 派支持的大语言模型微调自阿里云的大语言模型，支持和大模型进

行基础的对话并获取回复文本。

指令学习



这条指令会向大模型提问文本框中的内容，并将回答的内容赋值给变量 llm_answer。

程序编写

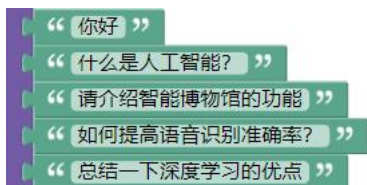
示例 1：大模型对话-文本

打开范例代码 语音 AI-4.大模型对话一。

修改用户名和密码。



这个程序会和大模型展开对话，询问大模型以下五个问题。



程序运行后，显示区效果如下（中间部分省略）：

```
程序输出
[09:36:30] [成功] 程序启动成功, PID: 10733
[09:36:30] === LLM大模型功能测试 ===
[09:36:30] 该测试输入使用固定文本, 输出使用串口打印
[09:36:30] 获取Token成功:
[09:36:30] {'success': 200, 'userToken': 'f2uvvess9lm2gvc5mo4pw29qp2nduw0fvph9dc3u', 'msg': 'userToken获取成功!'}
[09:36:30] 认证成功成功, 开始测试LLM功能
[09:36:31] 测试问题: 你好
[09:36:33] 大模型调用成功:
[09:36:33] {'success': 3, 'msg': '获取成功!', 'answer': '你好! 很高兴见到你。今天是2025年8月7日, 星期四。有什么我可以帮你的吗?'}
[09:36:33] API状态码: 3
[09:36:33] 消息: 获取成功!
[09:36:33] 大模型回答成功
[09:36:33] 回答内容: 你好! 很高兴见到你。今天是2025年8月7日, 星期四。有什么我可以帮你的吗? ...
[09:36:33] 成功 - 回答长度: 38 字符
[09:36:33] 回答: 你好! 很高兴见到你。今天是2025年8月7日, 星期四。有什么我可以帮你的吗?
[09:36:33] -----
[09:36:33] 测试问题: 什么是人工智能?
[09:36:40] 大模型调用成功:
```

```
集的能力。'
[09:37:07] API状态码: 3
[09:37:07] 消息: 获取成功!
[09:37:07] 大模型回答成功
[09:37:07] 回答内容: 根据提供的资料, 深度学习的主要优点可以总结为以下几点: 1. **卓越的性能表现**: 在图像识别、语音识别、自然语言处理、游戏等众多复杂任务上, 深度学习模型的准确性远超传统机器学习方法。 2. ...
[09:37:07] 成功 - 回答长度: 466 字符
[09:37:07] 回答: 根据提供的资料, 深度学习的主要优点可以总结为以下几点: 1. **卓越的性能表现**: 在图像识别、语音识别、自然语言处理、游戏等众多复杂任务上, 深度学习模型的准确性远超传统机器学习方法。 2. **强大的数据扩展能力**: 深度学习模型能随着训练数据量的增加而显著提升性能, 通常数据越多, 效果越好。这与传统机器学习算法(需要更复杂的手段来提升精度)形成对比。 3. **自动特征学习(减少特征工程)**: 深度学习能够自动从原始数据中学习和提取有用的特征, 大大减少了人工设计特征的复杂性和工作量, 避免了因特征选择不当带来的问题。 4. **良好的适应性与迁移性**: 深度学习技术易于适应不同领域和应用。通过迁移学习, 预训练好的模型可以轻松应用于同一领域内的新任务(如图像分类模型做目标检测), 有效节省训练时间和资源。此外, 不同领域的基础深度学习技术(如语音识别和自然语言处理)也具有高的可转换性。 5. **可扩展性**: 深度学习模型能够处理海量数据, 并且可以在分布式计算系统上进行训练, 使其具备处理大规模数据集的能力。
[09:37:07] -----
[09:37:07] === LLM测试完成 ===
[09:37:07] [成功] 程序运行完成
```

可以看到，好搭 AI 派会联网调用大模型，并在 3s 左右给出回答。这个大模型联网，但未处于深度思考模式。

示例 2：大模型对话-语音

刚刚的大模型范例，都是文本的输入与输出。

我们可以让其更智能，修改程序，语音输入语音输出。

打开范例代码 语音 AI-6.大模型对话三。

修改用户名和密码。

这个程序主要包含以下部分：

①进行用户 API 认证

②程序开始后就开始音频录入，持续 8s

③录入的音频通过语音识别功能，生成对应文本

④向大模型提问这串文本，大模型会进行回复

⑤大模型回复的文本信息通过语音合成功能，生成音频并播放



示例 3：按键大模型对话

刚刚的示例 2，只能提问一个问题，并且限制了录音时间，与大模型的交互体验感还不够，所以我们尝试使用按键来进行控制。按下按键开始录音，松开按键停止录音。这样就可以一直与大模型进行简单交互了。

打开范例代码 语音 AI-7.按键大模型对话。

修改用户名和密码。

这个程序的基础逻辑与示例 2 相似，多了按键判断功能。

请将单按键模块连接到好搭 AI 派的 IO1 引脚，并将好搭 AI 派右下角的开关拨动到左侧。

当按键被按下后，开始录音；

当按键松开后，停止录音，之后保存录音文件、进行语音识别、进行大模型提问。

需要注意的是，为了确保录制完整的音频，建议按下按键后，稍等片刻，等待屏幕提示【开始录音】，再说话进行提问



程序运行后，会先显示以下界面，等待按键按下中：

```
程序输出
[10:33:18] === LLM大模型功能测试3 ===
[10:33:18] 该测试输入使用语音识别，输出并播报
[10:33:18] 获取Token成功:
[10:33:18] {'success': 200, 'userToken':
'ttvi0161farlmfcg4elqoj340rc38a4f4vfuu24', 'msg': 'userToken获取成功!'}
[10:33:18] ☒ 认证成功，开始测试LLM功能
[10:33:18] 正在自动检测ESP32设备...
[10:33:18] 发现可用串口: ['/dev/ttyACM1', '/dev/ttyACM0']
[10:33:18] 正在测试串口: /dev/ttyACM1
[10:33:18] ✓ 成功检测到ESP32设备: /dev/ttyACM1
[10:33:18] <ESP32.mSerial object at 0x7f99197370>
[10:33:18] ✓ 成功连接到ESP32设备: /dev/ttyACM1
[10:33:18] 可用的音频设备:
[10:33:18] 0: rockchip,es8388-codec: dailink-multicodecs ES8323.7-0011-0
(hw:0,0) (输入声道: 2)
[10:33:18] 1: sysdefault (输入声道: 128)
[10:33:18] 2: samplerate (输入声道: 128)
[10:33:18] 3: speexrate (输入声道: 128)
[10:33:18] 4: pulse (输入声道: 32)
[10:33:18] 5: upmix (输入声道: 8)
[10:33:18] 6: vdownmix (输入声道: 6)
[10:33:18] 8: default (输入声道: 32)
```

当单按键被按下后，说话进行提问；注意，推荐等“开始录音...”字样出现后再开始说话，不然容易少录入。

这里说了一句“杭州今天天气怎么样”，语音识别成功。

```
程序输出
[10:33:18] 5: upmix (输入声道: 8)
[10:33:18] 6: vdownmix (输入声道: 6)
[10:33:18] 8: default (输入声道: 32)
[10:34:56] 开始录音... (采样率: 16000Hz, 声道: 1)
[10:34:57] 录音开始! 按 Ctrl+C 停止录音...
[10:34:57]
[10:34:59] 正在停止录音...
[10:34:59] 录音完成! 录制时长: 2.37 秒
[10:34:59] 音频文件已保存: recordings/LLM4.wav
[10:34:59] 文件信息: 37948 样本, 2.37 秒
[10:34:59] 录音成功! 文件保存在: recordings/LLM4.wav
[10:34:59] 音频文件大小: 75940 字节
[10:34:59] 正在识别音频文件: recordings/LLM4.wav
[10:35:01] 语音识别API调用成功:
[10:35:01] {"success":
200,"result":"u676d\u5dde\u4eca\u5929\u5929\u6c14\u600e\u4e48\u6837\u4eff1f",
"msg":"u77ed\u8bed\u97f3\u8bc6\u522b\u6210\u529f\u4eff01"}
[10:35:01] API状态码: 200
[10:35:01] 识别结果: '杭州今天天气怎么样?'
[10:35:01] 消息: 短语音识别成功!
[10:35:01] ☒ 识别成功: 杭州今天天气怎么样?
[10:35:01] 测试问题: 杭州今天天气怎么样?
[10:35:04] 大模型调用成功:
```

识别成功后向大模型提问，大模型回答并生成了音频进行播报。

```
程序输出
[10:35:04] 消息: 短语音识别成功!
[10:35:01] ☒ 识别成功: 杭州今天天气怎么样?
[10:35:01] 测试问题: 杭州今天天气怎么样?
[10:35:04] 大模型调用成功:
[10:35:04] {'success': 3, 'msg': '获取成功!', 'answer': '杭州今天天气多云, 气温27°C至37°C, 体感闷热, 有持续高温。'}
[10:35:04] API状态码: 3
[10:35:04] 消息: 获取成功!
[10:35:04] ☒ 大模型回答成功
[10:35:04] 回答内容: 杭州今天天气多云, 气温27°C至37°C, 体感闷热, 有持续高温...
[10:35:04] TTS合成成功: {'ttsrsrc': 'http://www.haohaodada.com/project/voiceAI/ApiPublic/TTS/828237_202508071035045594.wav', 'success': 200,
'msg': 'TTS合成成功!'}
[10:35:04]
[10:35:05] WAV音频下载成功, 大小: 287724 字节
[10:35:05]
[10:35:05] WAV音频已保存到: recordings/answer.wav
[10:35:05] Pygame mixer 初始化成功
[10:35:05]
[10:35:05] 开始播放音频...
[10:35:14] 音频播放完成
[10:35:14] 回答: 杭州今天天气多云, 气温27°C至37°C, 体感闷热, 有持续高温。
[10:35:14] Pygame mixer 资源已清理
```

AI 视觉算法

AI 视觉算法是基于计算机视觉与人工智能技术，使机器能够模拟人类视觉系统进行图像理解的核心方法

好搭 AI 派的视觉算法主要是通过外置摄像头进行视觉识别，包含 AprilTag 标签检测、黑线检测、色块检测、颜色识别、人脸识别、二维码识别、车牌识别、目标检测、姿态检测、自定义物体识别、人脸计数、图像分类等多种算法，支持人脸信息数据库和自定义物体信息数据库。

一、摄像头与算法基础

好搭 AI 派支持外接摄像头。摄像头会在窗口显示画面，并通过 AI 视觉算法对画面进行相应的识别与处理。

指令学习



这两条指令用于初始化视觉系统。指令会自动检测摄像头并打开，并设置显示摄像头窗口的分辨率。一般选择标准或高清；1920*1080 则是全屏。

两条指令根据实际选择一条即可。

开始连续检测

开始单帧检测

这两条指令是摄像头检测的必要指令，用于设置检测方式。

单帧检测：每次处理独立的一帧图像，不需要考虑帧间关联性，比如静态图像分类、单次识别等。

连续检测：处理视频流，需要利用帧间信息进行跟踪、预测或行为分析，比如目标跟踪、运动分析等。

AprilTag 标签检测、黑线检测、色块检测、二维码识别、图像分类、人脸识别、目标检测单次等，适合单帧检测。

单帧检测常用写法如下：



目标检测（带跟踪）、姿态检测（动态分析）、人流计数（动态去重）、车牌识别（动态场景）、颜色识别等则推荐使用连续检测。

连续检测常用写法如下：



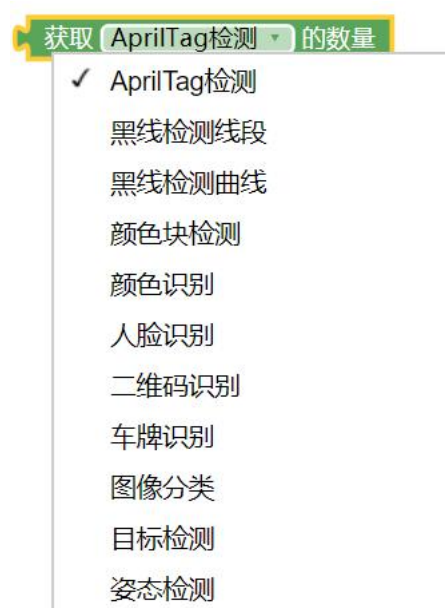
下方两条指令，则是用于程序最后，停止检测与清理系统资源。

停止检测

清理系统资源



这条指令用于设置各类算法启用，共 12 种。



这条指令用于获取摄像头识别到对应算法检测的结果数量。其中物体识别只能检测一个；人流计数检测多个；其余 10 种算法可以有 0/1/更多的检测结果。

刷新检测结果

这条指令用于刷新检测结果。注意该指令并不是指刷新摄像头画面中的内容，而是刷新算法检测的具体结果，一般默认每帧刷新一次检测的结果。

二、AprilTag 检测

AprilTag 是一种基于视觉的基准标记系统，可以利用 AprilTag 来获取标签相对于摄像头的位置、距离及坐标，主要用于物体定位、姿态估计和空间校准领域。

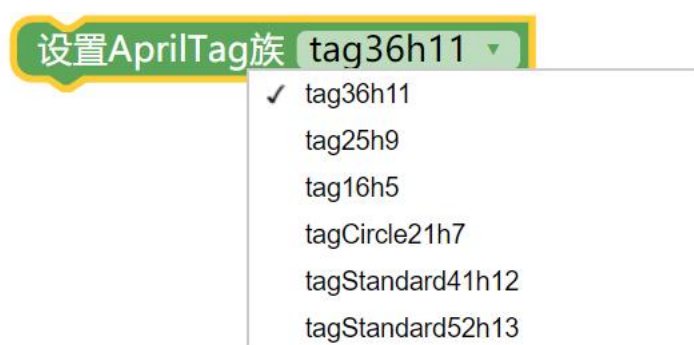
AprilTag 标签对旋转、缩放、光照变化具有强适应性，识别速度显著快于传统二维码，尤其适合动态场景的实时处理，例如货架标签定位、传送带物料位姿检测与机械臂抓取引导、AR 技术中虚拟物体在实体标签位置的精准叠加等。

AprilTag 有非常多的种类，比如常见的 tag16h5，共有 30 种；又比如 tag36h11，足足有 587 种。

可以点击下方链接下载 AprilTag 标签。

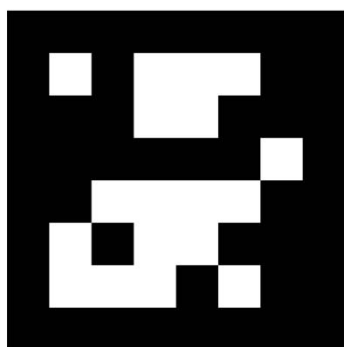
<https://haohaodada.com/ueditor/php/upload/file/20220411/1649644441188210.pdf>

指令学习



这条指令可以设置 AprilTag 识别的种类，默认是 tag36h11 族。如果需要识别其他种类的标签，则需要使用这条指令更改识别的族。

如下图，这是一个 tag36h11 族的标签，ID 是 27。





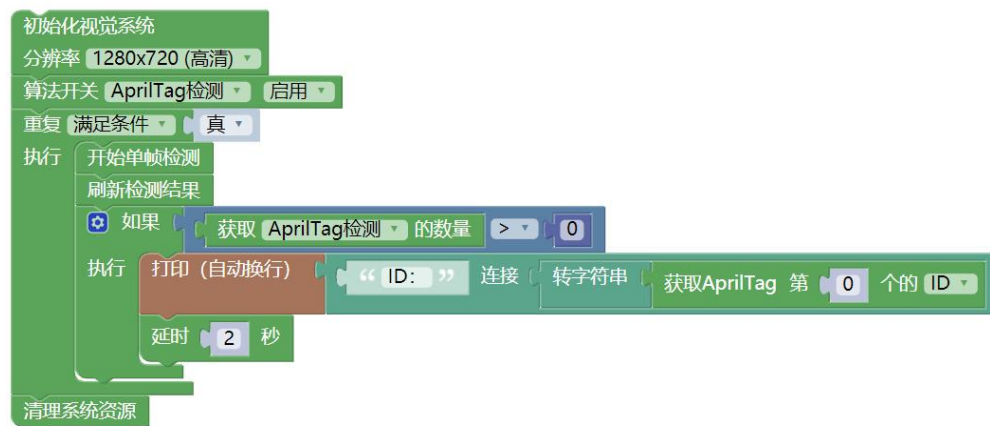
这条指令可以获取 AprilTag 识别到标签的 ID、族、坐标等各类信息。

程序编写

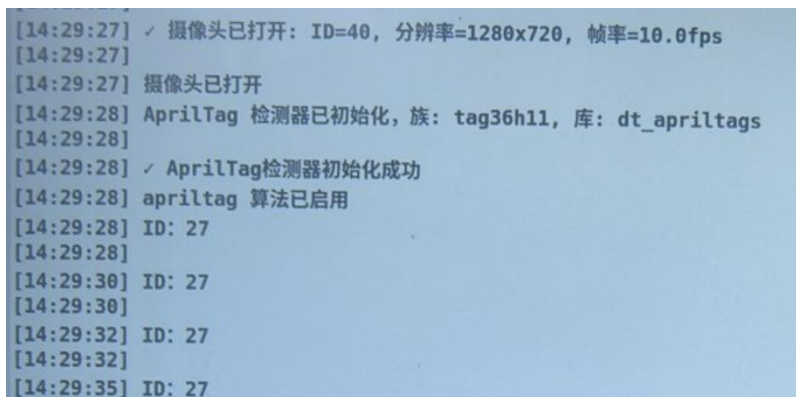
示例 1：标签识别

打开范例代码 AI 视觉算法-01.标签识别。

连接外置摄像头，并调整摄像头的位置与方向。



识别到标签后，具体效果显示如下：



示例 2：超市自助结账

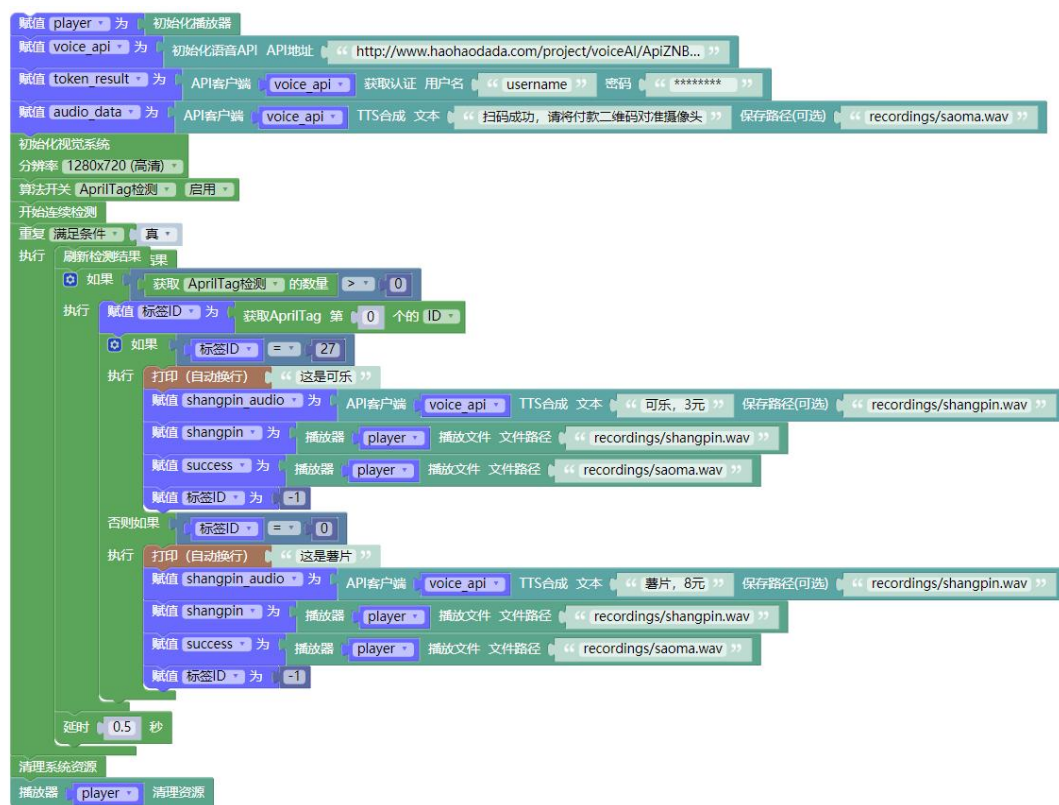
我们在一些超市、便利店中，经常看到有可以自助结账的装置。我们给几样商品贴上 Apriltag 标签，模拟商品的标签，结合摄像头与语音合成功能，制作一个简易的超市自助结账系统。

打开范例代码 AI 视觉算法-02.标签识别-超市自助收银。

输入用户名和密码。

当检测到标签 ID=27 时，语音播报，可乐，3 元，扫码成功，请将付款二维码对准摄像头；

当检测到标签 ID=0 时，语音播报，薯片，8 元，扫码成功，请将付款二维码对准摄像头。

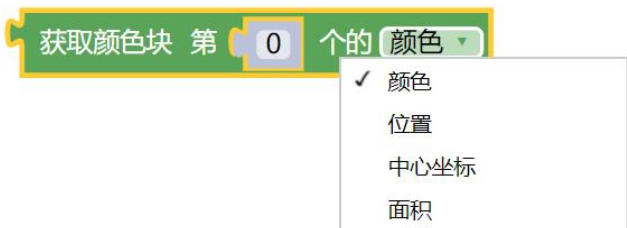


三、色块检测

指令学习



这条指令可以设置色块识别的位置和对应的颜色。默认有黑、白、红、绿、蓝、黄六种颜色。

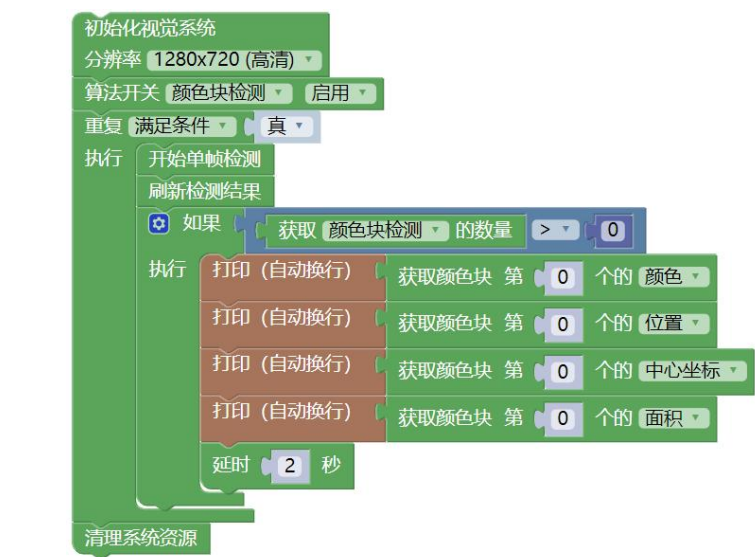


这条指令可以获取色块的颜色、位置、中心坐标和面积。

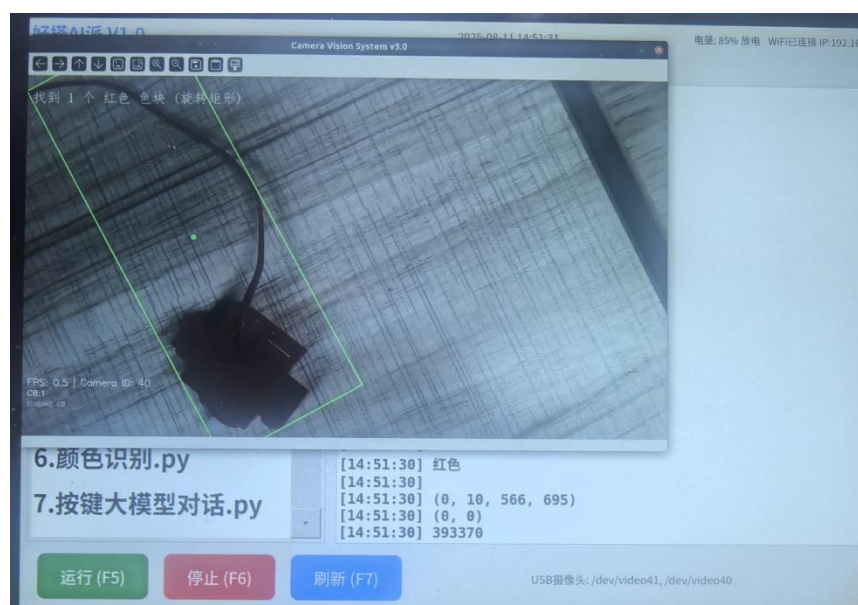
程序编写

示例 1：色块识别

打开范例代码 AI 视觉算法-06.色块识别。
连接外置摄像头，并调整摄像头的位置与方向。

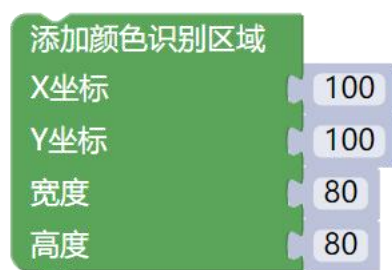


具体显示效果如下所示，识别到红色色块：



四、颜色识别

指令学习

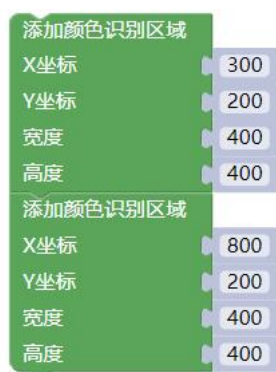


这条指令可以设置颜色识别具体的识别区域。其中 XY 坐标（100,100）表示颜色识别区域左上角的坐标；宽度和高度表示识别区域的大小。

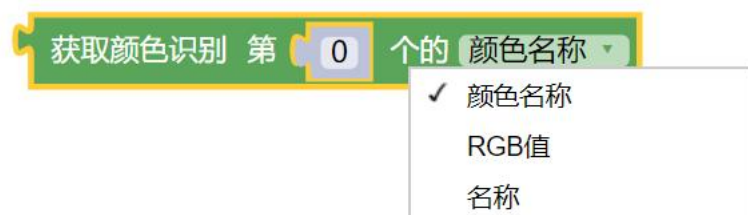
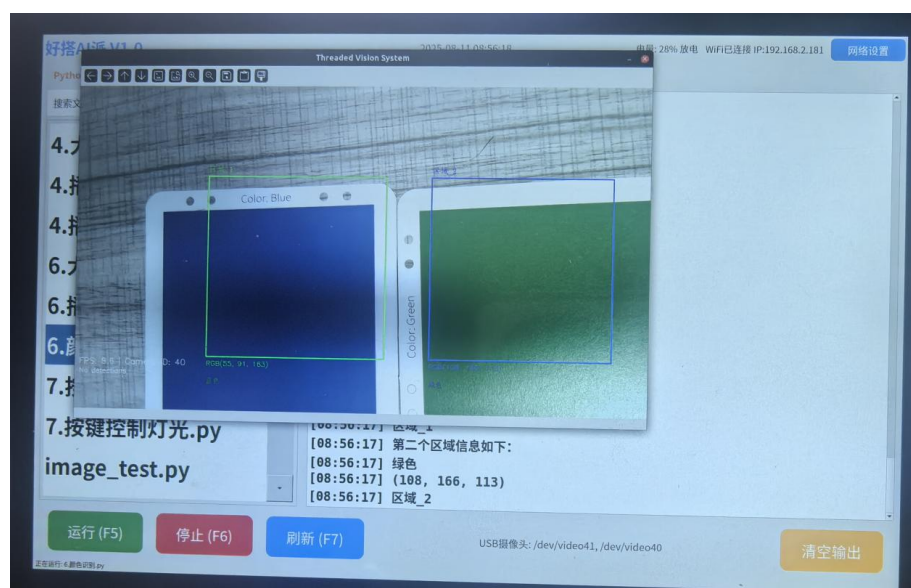
整个好搭 AI 派的最大识别区域是宽度 1280，高度 720；即代表 X 坐标+宽度 ≤ 1280 ；Y 坐标+高度 ≤ 720 。（如果摄像头分辨率设置 1280*720）

如果需要同时对两个区域进行颜色检测，则先设置的区域是第 0 个结果，是区域_1，后设置的区域是第 1 个结果，表示区域 2。

如下图，设置了两个区域：



最终效果如下所示：



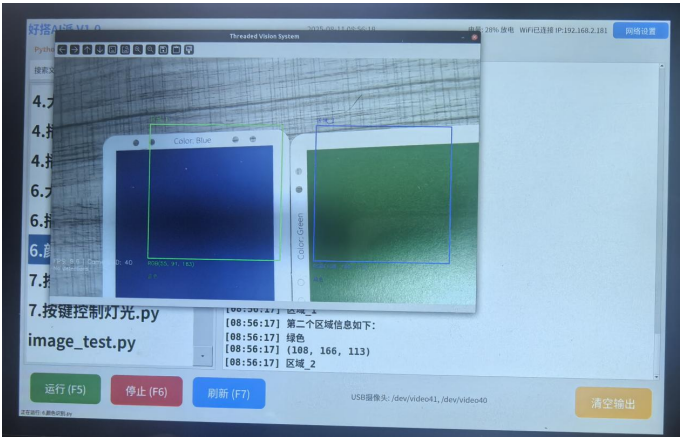
这条指令可以获取对应区域颜色名称、RGB 值和名称。其中识别到的第一个区域，名称叫做区域_1，是识别到的第 0 个结果；识别到的第二个区域，名称叫做区域_2，是识别到的第 1 个结果。摄像头会根据识别结果，将识别的颜色分类到六种颜色中，红黄蓝绿黑白，并输出具体的 RGB 值。

程序编写

示例 1：颜色识别

打开范例代码 AI 视觉算法-04.颜色识别。

连接摄像头，运行程序。



效果如图所示，区域_1 是蓝色，RGB 值是（8，53，146）；区域_2 是绿色，RGB 值是（96，160，101）。

示例 2：实际应用

某新能源电池工厂需对锂电池极片的涂层颜色进行质量检测，要求识别绿色

（合格）与蓝色（不合格），并实现自动分拣，日均处理量达 20 万片。

为了将颜色通过数据进行量化，我们使用标准的 RGB 颜色空间到亮度 (Luminance) 的转换公式，也称灰度转换公式：

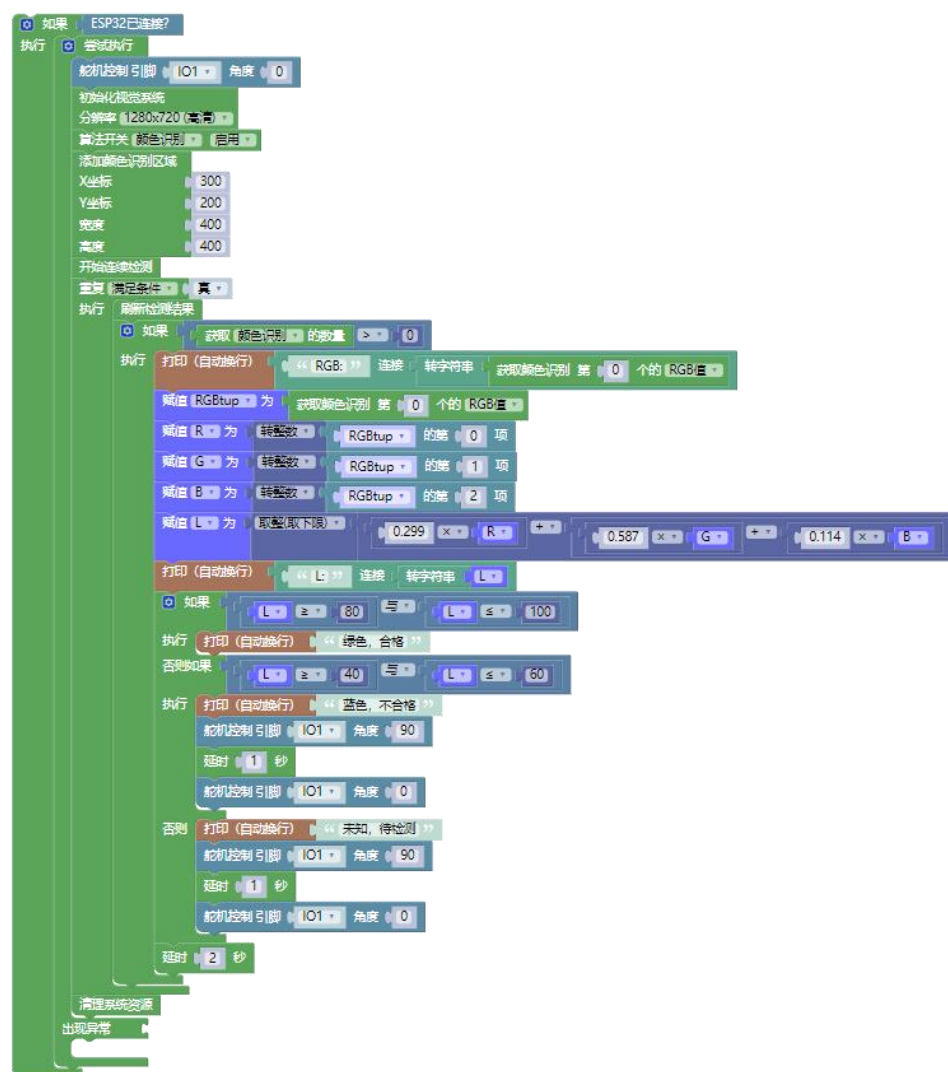
$$L = 0.299 * R + 0.587 * G + 0.114 * B$$

待检测的两种涂层颜色，蓝色的 $L \approx 50$ ，绿色的 $L \approx 90$ ，我们以 ± 10 为区间，即 L 的值在 80-100 之间，正常通过检测；在 40-60 之间，判定为不合格；在其他区间，则先分拣，等待其他处理。

为实现自动分拣，我们简单使用舵机；舵机转动到 0 度表示合格，转动到 90 度表示不合格或待检测。实际场景则有机臂气缸推物体，或是动态抓取。

打开范例代码 AI 视觉算法-05.颜色识别-自动分拣。

连接摄像头，运行程序。绿色表示合格，其余颜色不合格，舵机转动。



五、人脸识别

指令学习

获取人脸识别的 ID

这条指令可以获取识别到的人脸是人脸数据库中的第几个人脸。点击下拉按钮，可以选择获取识别到的人脸的置信度和位置。

学习新人脸

使用这条指令可以根据当前摄像头学习新人脸。

学习新人脸并返回结果

赋值 人脸信息 为 学习新人脸并返回结果
打印 (自动换行) 人脸信息

使用这条指令则可以获取学习人脸后返回的信息，学习新人脸后返回结果如下所示：

这是学习了一个新人脸，ID 是 19。

```
[10:44:39] {'success': True, 'face_id': 19, 'message': '成功添加新的人脸，分配ID: 19'}
```

如果你再次学习同样的人脸，系统会进行提示。

```
[10:44:44] {'success': False, 'face_id': 19, 'message': '该人脸已经注册过，ID: 19'}
```

使用指定图像学习人脸 图像数据

使用指定图像学习人脸并返回结果 图像数据

从文件加载图像 文件路径

“ image.jpg ”

这两条指令则是使用指定的图像进行人脸学习。配合图像获取类别的从文件加载图像指令一起使用。

程序编写

示例 1：人脸学习 1

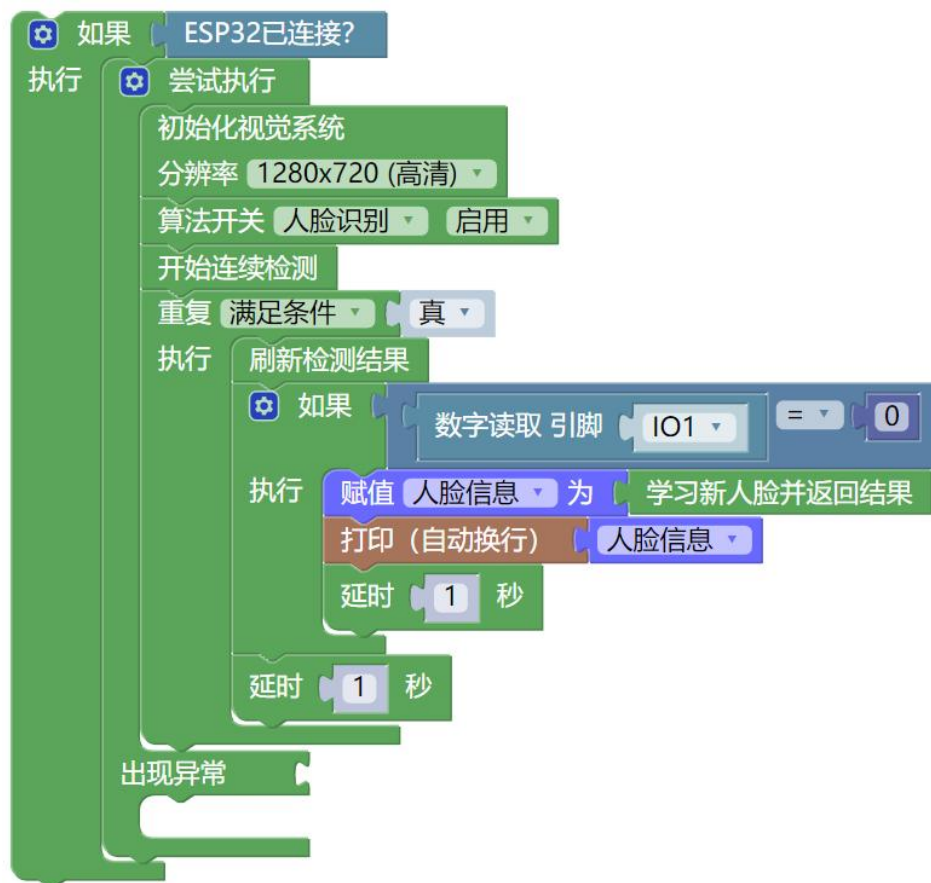
打开范例代码 AI 视觉算法-08.人脸学习 1。

连接外置摄像头，并调整摄像头的位置与方向。

将人脸对准摄像头，按下连接在 IO1 的单按键进行人脸学习。

板载按键，松开是 1，按下是 0。

学习后记录下对应的人脸 ID。这次学习的人脸信息为 19。



示例 2：人脸学习 2

打开范例代码 AI 视觉算法-09.人脸学习 2。



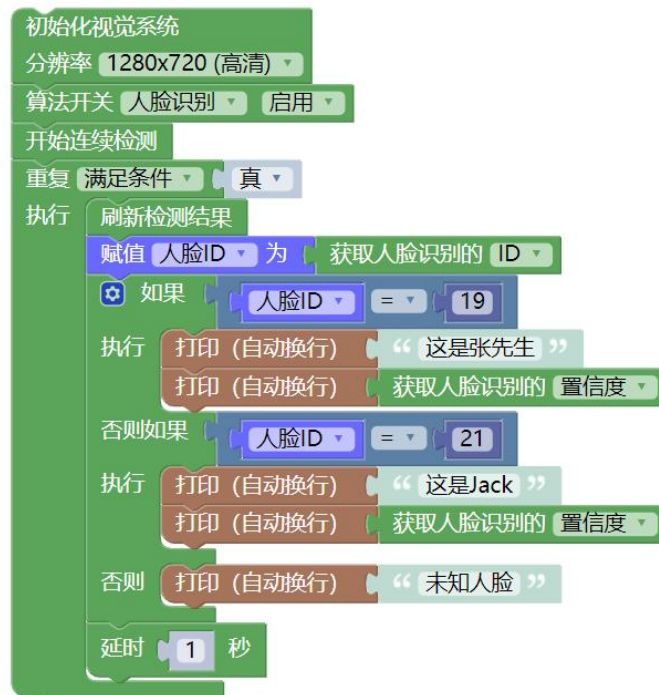
点击运行后，程序会读取图片并直接学习对应人脸。

这次学习的人脸 ID 为 21。

示例 3：人脸识别

打开范例代码 AI 视觉算法-10.人脸识别。

刚刚学习的人脸对应 ID 是 19 和 21，记录后填写到程序中，屏幕显示如下。



```
[13:32:04] 这是张先生
[13:32:04] 0.9061625657030786
[13:32:05] 未知人脸
[13:32:06] 未知人脸
[13:32:07] 未知人脸
[13:32:08] 这是张先生
[13:32:08] 0.8732578618018495
```

六、物体识别

指令学习

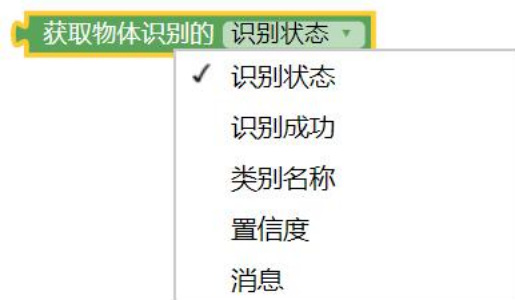


这条指令与人脸学习类似，可以进行物体学习，并将识别的 000 结果添加到新类别/原有类别。上方指令是将识别到的物体添加到“苹果”类别。

两条指令一条返回结果，一条不返回。



与人脸识别类型，我们可以使用这条指令，通过指定图片添加物体识别类别。我们可以给同一个类别学习多张图片。



这条指令可以获取学习后的物体，识别时的状态、名称、置信度等。

程序编写

示例 1：物体识别学习

打开范例代码 AI 视觉算法-11.物体识别学习。

本案例以识别水杯为例进行介绍。

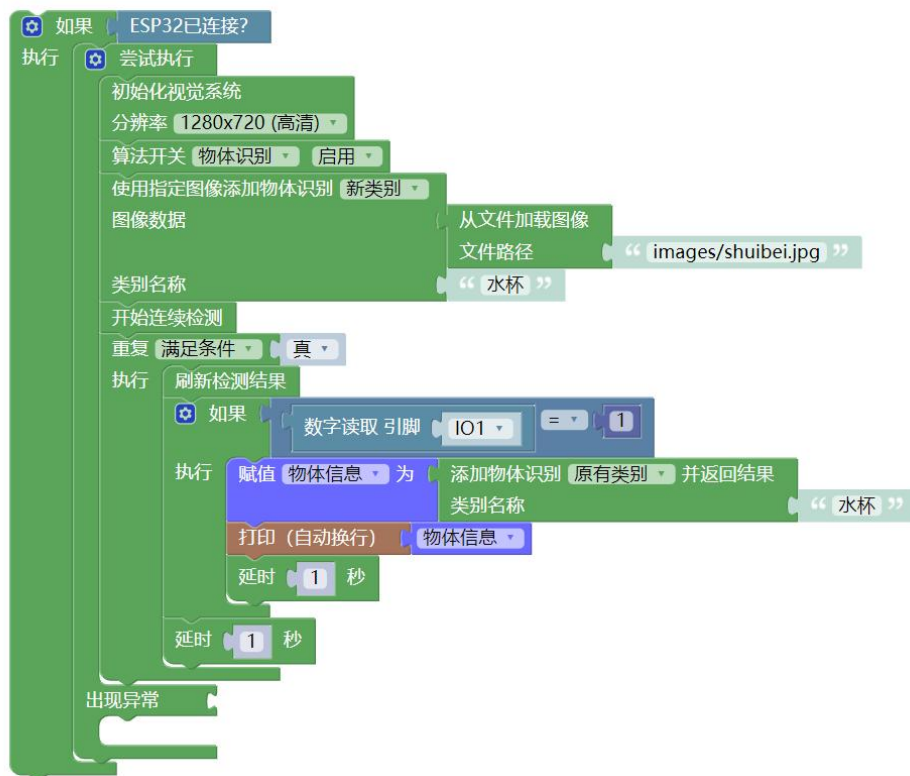
我先用手机给水杯拍了一张照片，命名并导入，如下所示：



连接外置摄像头，在 IO1 引脚连接单按键模块，运行程序。

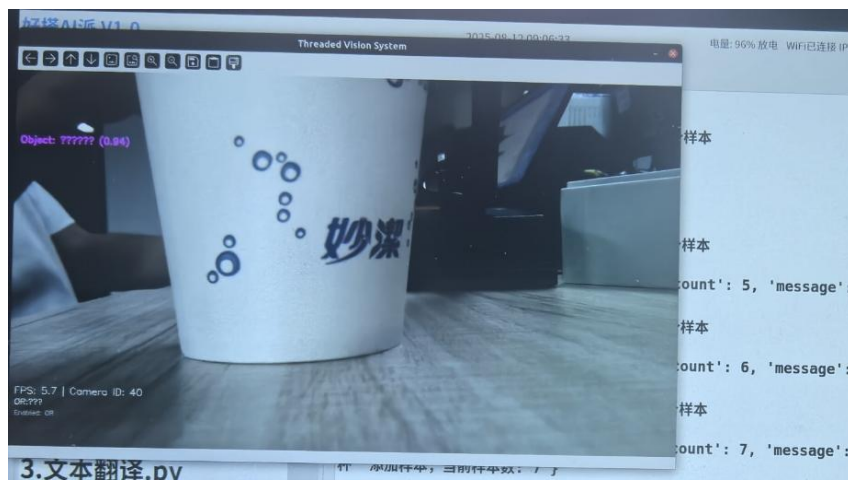
调整摄像头的位置与方向，按下外接按键给要识别的水杯拍照，直到屏幕上出现以下字样，说明学习成功。

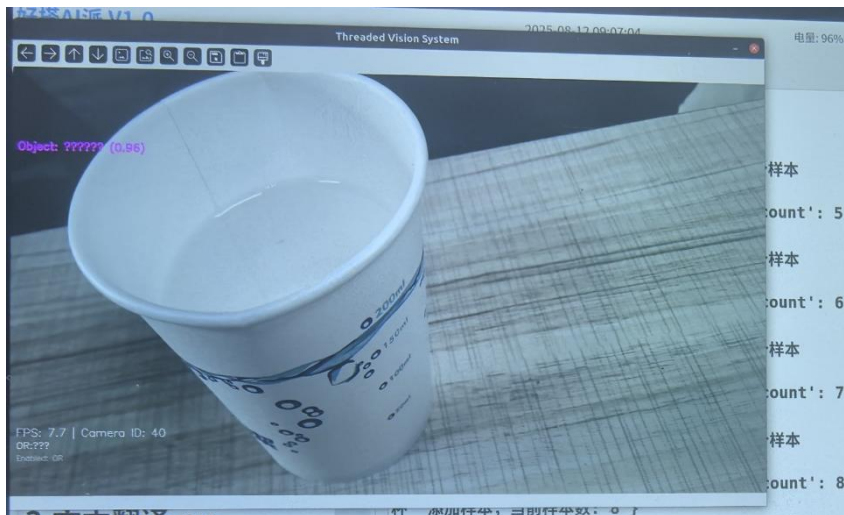
```
[09:05:26] {'success': True, 'samples_count': 8, 'message': '成功为类别 "水杯" 添加样本, 当前样本数: 8'}
```



这里对多个方向的水杯进行了学习。

具体学习情况如下：





示例 2：物体识别

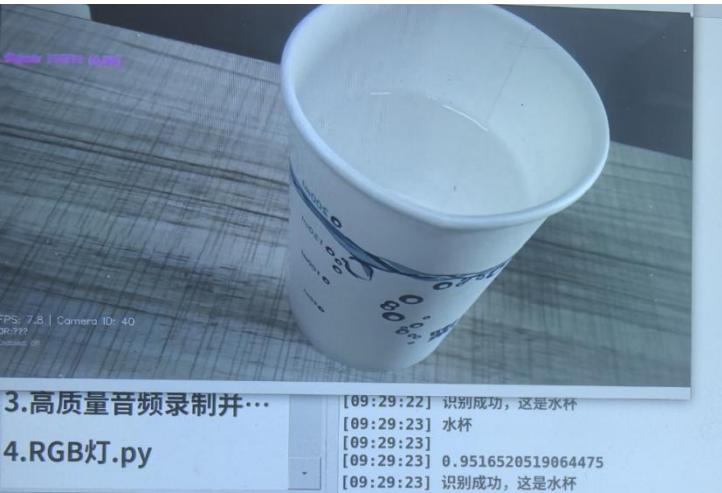
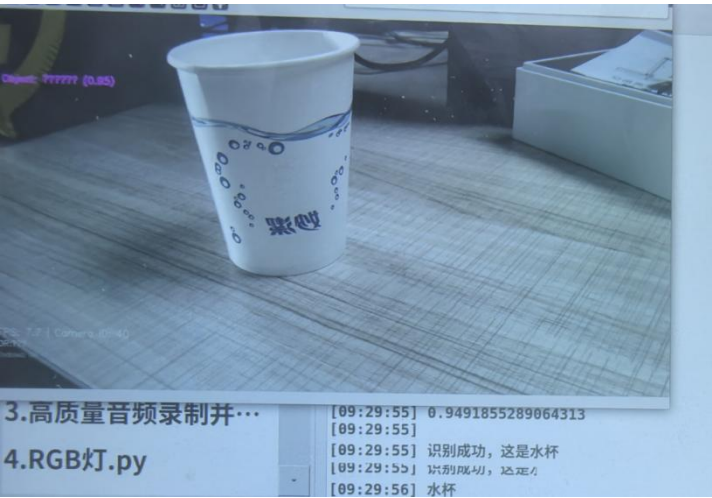
打开范例代码 AI 视觉算法-12.物体识别。

以刚刚学习过的水杯为例，进行物体识别。

一般我们获取类别名称和置信度即可。

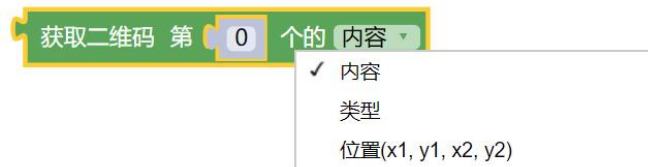


具体识别效果如下，可以看到，各个方向都能正常识别：



七、二维码检测

指令学习



这条指令可以获取识别到的二维码对应的内容。

目前类型只支持 **QRCODE** 类型的二维码。

常规的二维码、收款码、付款码等均能正常识别。

二维码内容目前只支持英文，不支持中文。

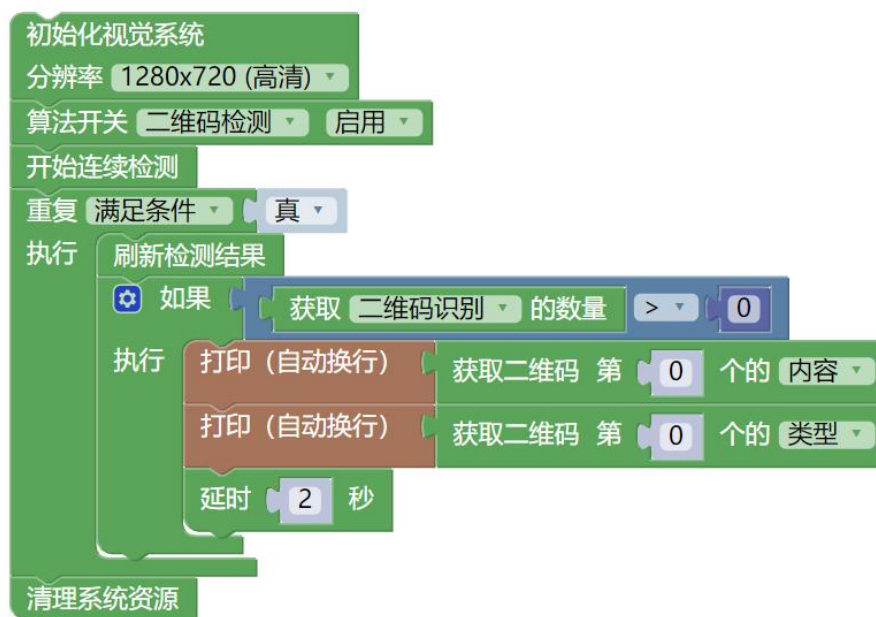
程序编写

示例 1：二维码识别

打开范例代码 **AI 视觉算法-03.二维码识别**。

连接外置摄像头，运行程序代码。

展示二维码，调整摄像头，对二维码进行识别。





以识别上方二维码为例，识别结果如下：

识别内容为：<http://haohaodada.com/new/index.php>

```
[08:48:58] [成功] 程序启动成功, PID: 2913
[08:48:59] AprilTag 库已加载: dt_apriltags
[08:48:59]
[08:49:00] [错误] INFO:color_block_detector:PIL可用, 支持中文字体渲染
[08:49:04] 视觉系统初始化完成
[08:49:04]
[08:49:04] ✓ 使用备用摄像头 ID: 40
[08:49:04]
[08:49:05] ✓ 摄像头已打开: ID=40, 分辨率=1280x720, 帧率=10.0fps
[08:49:05]
[08:49:05] 摄像头已打开
[08:49:05] ✓ 二维码检测器初始化成功
[08:49:05] qr_code 算法已启用
[08:49:05] ✓ 后台检测已启动
[08:49:12] http://haohaodada.com/new/index.php
[08:49:12] QRCODE
[08:49:13] http://haohaodada.com/new/index.php
[08:49:13] QRCODE
[08:49:14] http://haohaodada.com/new/index.php
[08:49:14] QRCODE
[08:49:15] http://haohaodada.com/new/index.php
[08:49:15] QRCODE
[08:49:16] [系统] 程序异常退出 (代码: 9)
[08:49:16] [系统] 程序已停止
```

八、车牌识别

指令学习

获取车牌 第 0 个的 车牌号码

✓ 车牌号码

置信度

位置(x1, y1, x2, y2)

这条指令可以获取识别到的车牌对应的内容、置信度和位置。

如果画面中出现多个车牌，则会分别返回到第 0-N 个结果中。

程序编写

示例 1：车牌识别

打开范例代码 AI 视觉算法-13.车牌识别。

连接外置摄像头，运行程序代码。

展示车牌图片，调整摄像头，对车牌进行识别。

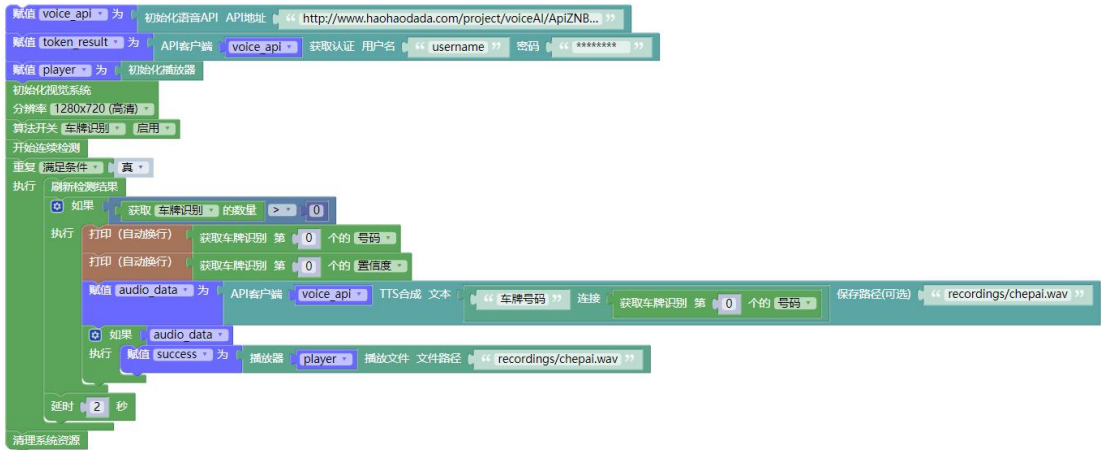
本范例先后对以下车牌进行识别，检测车牌识别算法的性能：



车牌识别算法包含两层模型，一层模型用于获取图片中车牌的具体位置，一层模型用于识别车牌的具体号码。

目前暂不支持同一张图片中有多个车牌号码。

可以使用语音合成相关代码，编写程序，播报车牌号码。



双层车牌也可以直接识别。



九、人流计数

指令学习



首先需要注意的是，人流计数并不能获取检测结果数量，即不需要使用检测结果数量>0 类似的指令。这里的检测必须使用连续检测。

人流计数一共可以提供两个参数，这两个参数含义如下：

进入人数 `get_people_counter_in`:

表示检测周期内从监控区域外部进入内部的人数。该数值通过分析视频流中人体移动方向判定。

离开人数 `get_people_counter_out`:

统计周期内从监控区域内部离开到外部的人数。其技术实现与进入人数检测对称，但移动方向判断逻辑相反。

通过处理这两个参数，我们可以获得另外两个重要参数：

总通过人数:

累计所有经过监控区域的人次总和（进入+离开）。该指标适用于评估区域活跃度，比如常用于商场客流量分析。

净流量:

计算进入与离开人数的差值（进入-离开），正值表示区域人数增加，负值表示减少。该指标对空间容量监控具有重要意义。

程序编写

示例 1：人流计数

打开范例代码 AI 视觉算法-16.人流计数。

连接外置摄像头，运行程序代码。

调整摄像头识别画面，使其对准行人走过的区域。



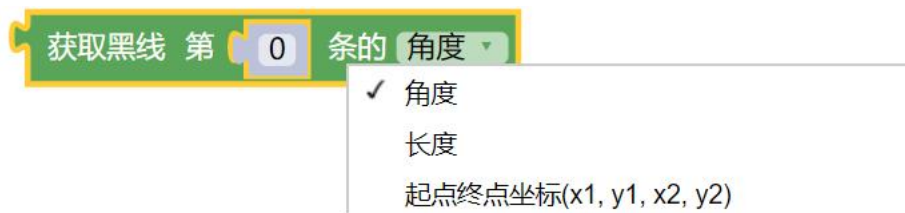
注意只有检测到一个人走过去才会进行计数。所以尽可能将摄像头安装在具有一定高度的位置，确保检测到一个人而不是人半身的衣服之类。

识别画面如下所示：



十、黑线检测

指令学习



这条指令可以获取检测到的线条的角度、长度和起点终点坐标。一般来说，

摄像头检测到画面中的线条不止一条。

程序编写

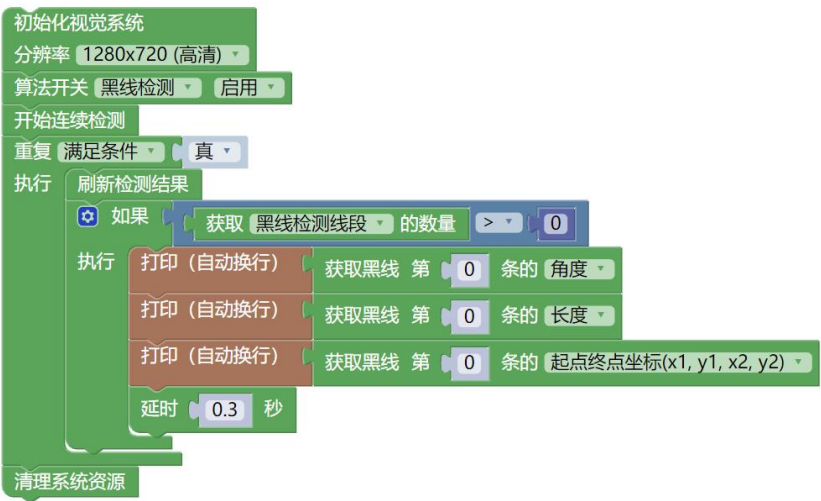
示例 1：黑线检测

打开范例代码 AI 视觉算法-07.黑线检测。

连接外置摄像头，运行程序代码。

调整摄像头识别画面。

线条检测要求的帧率较高，尽量减少延时。

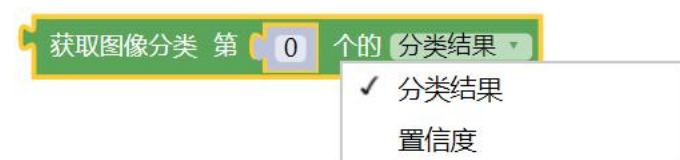


检测结果如下所示，屏幕上出现三处线条检测，分别代表 L0、L1、L2，摄像头和串口显示区均会显示线条的长度、角度、起点终点坐标。



十一、图像分类

指令学习



好搭 AI 派的图像分类算法，是指基于 ResNet 模型的实时图像分类系统，通过摄像头捕获视频流并进行分类，能够识别 1000 种常见物体类别。查看识别结果，发现会显示置信度前三的信息。

```
[11:49:09] {'apriltag': [], 'black_line': {}, 'color_block': {},
'color_recognition': {}, 'face_recognition': {}, 'qr_code': [],
'license_plate': {}, 'object_recognition': {}, 'people_counter': {}},
'image_classification': {'success': True, 'predictions': [{'rank': 1,
'class_id': 647, 'class_name': 'n03733805 量杯', 'score':
0.18311162292957306, 'confidence': '18.31%'}, {'rank': 2, 'class_id': 899,
'class_name': 'n04560804 水壶', 'score': 0.13653822243213654, 'confidence':
'13.65%'}, {'rank': 3, 'class_id': 503, 'class_name': 'n03062245 鸡尾酒调酒
器', 'score': 0.13653822243213654, 'confidence': '13.65%'}], 'error':
None}, 'object_detection': {}, 'pose_detection': {}, 'timestamp':
1755143349.3240564}
[11:49:11] {'apriltag': [], 'black_line': {}, 'color_block': {},
'color_recognition': {}, 'face_recognition': {}, 'qr_code': [],
'license_plate': {}, 'object_recognition': {}, 'people_counter': {}},
'image_classification': {'success': True, 'predictions': [{'rank': 1,
'class_id': 438, 'class_name': 'n02815834 烧杯', 'score':
0.17876815795898438, 'confidence': '17.88%'}, {'rank': 2, 'class_id': 899,
'class_name': 'n04560804 水壶', 'score': 0.11510591953992844, 'confidence':
'11.51%'}, {'rank': 3, 'class_id': 631, 'class_name': 'n03690938 乳液',
'score': 0.0993955209851265, 'confidence': '9.94%'}], 'error': None},
'object_detection': {}, 'pose_detection': {}, 'timestamp':
1755143351.3875458}
```

程序编写

示例 1：图像分类

打开范例代码 AI 视觉算法-15.图像分类。

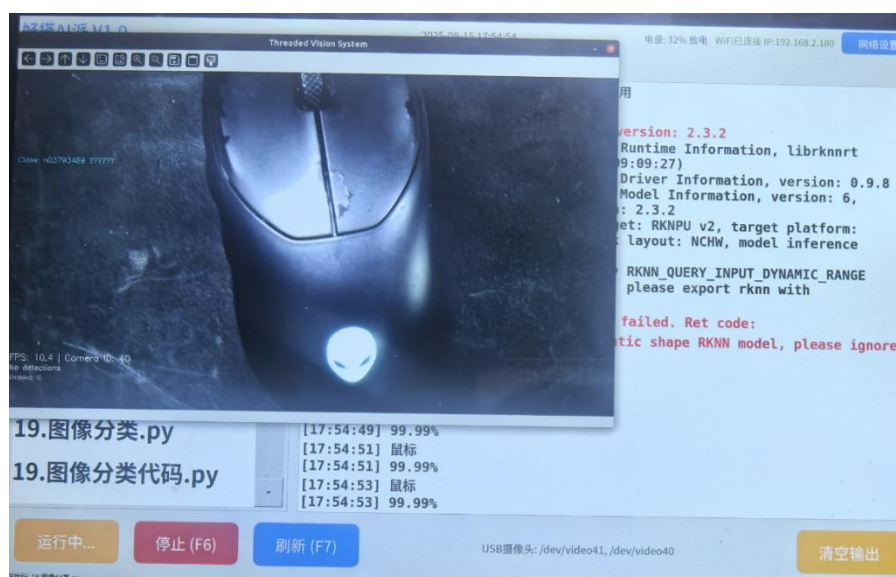
连接外置摄像头，运行程序代码。

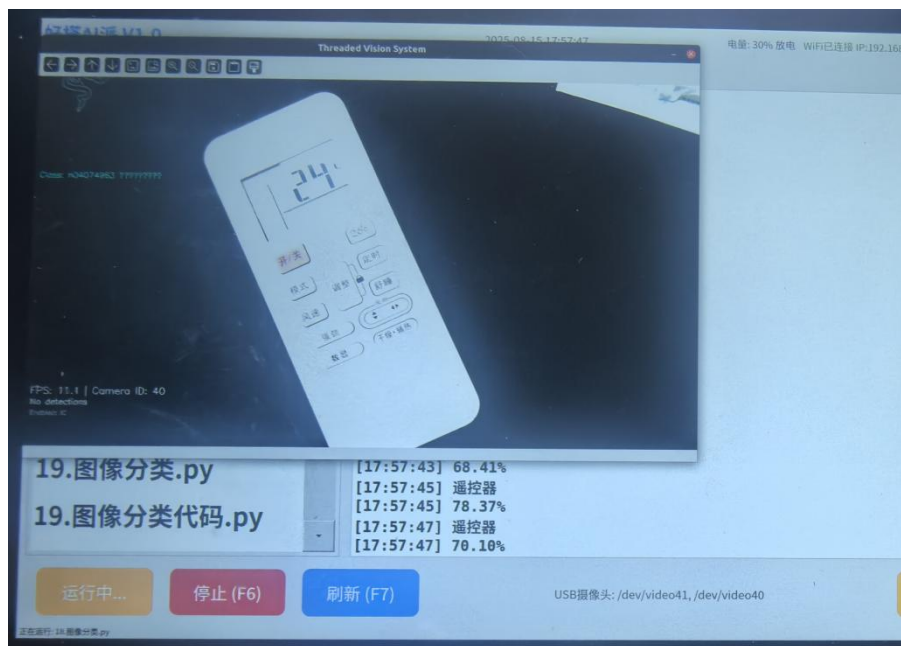
展示图像照片或者实物，调整摄像头，对图像进行识别。



图像分类加载模型时间较长，请耐心等待。

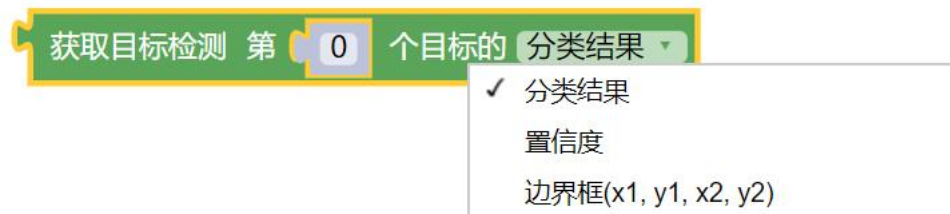
对鼠标和空调遥控器进行识别测试，测试结果如下：





十二、目标检测

指令学习



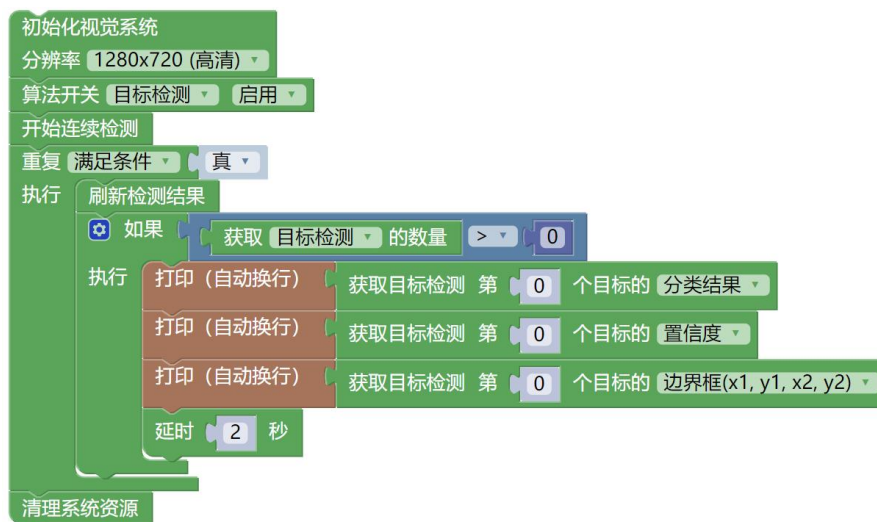
好搭 AI 派的目标检测算法，与图像分类算法类似，只不过只返回单个结果。

程序编写

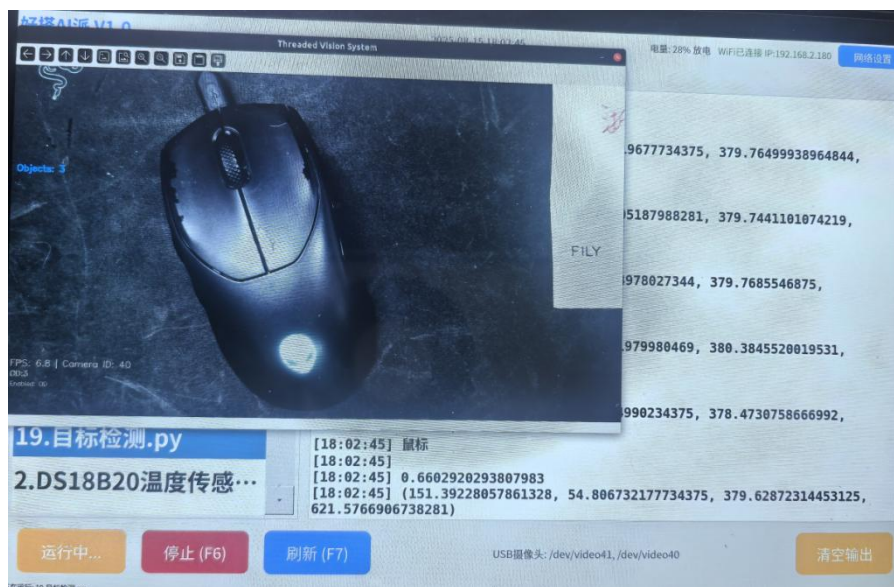
示例 1：目标检测

打开范例代码 AI 视觉算法-17.目标检测 并运行

展示图像照片或者实物，调整摄像头，对图像进行识别。



识别结果如图所示。检测出置信度 0.66，边界框如下图，与图像分类算法类似，但是能确认目标物体的具体位置。



十三、姿态检测

指令学习

获取姿态检测 第

 个人的 边界框(x1, y1, x2, y2) 坐标

✓ 边界框(x1, y1, x2, y2)
 整体置信度
 关键点(x, y, confidence)

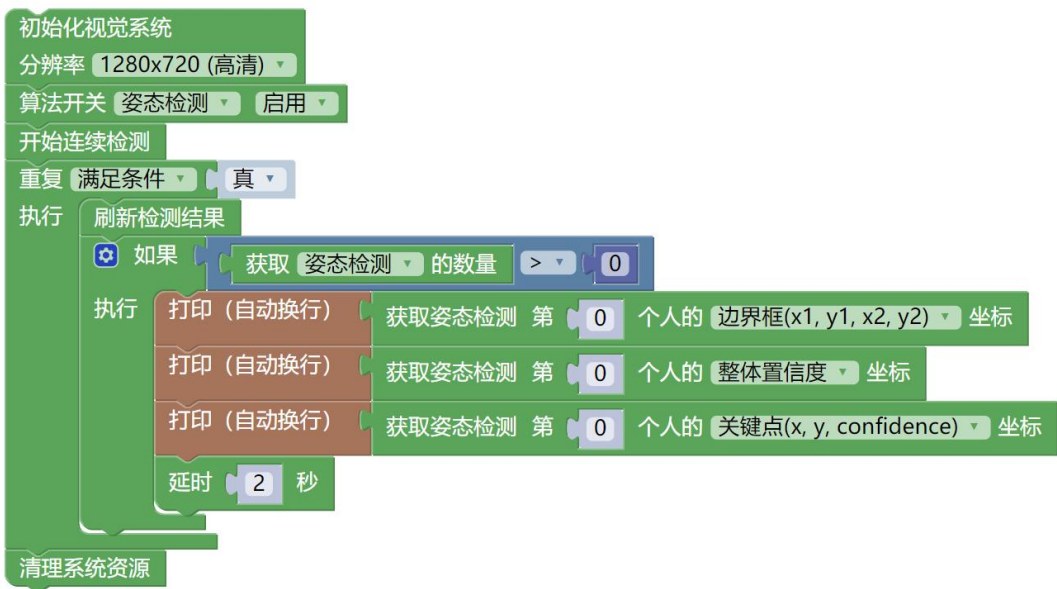
好搭 AI 派的姿态检测算法，可以检测人体的关键点与其位置。

程序编写

示例 1：姿态检测

打开范例代码 AI 视觉算法-18.姿态检测 并运行

调整摄像头，对人体进行识别，可以识别出关键点坐标。



十四、文字识别

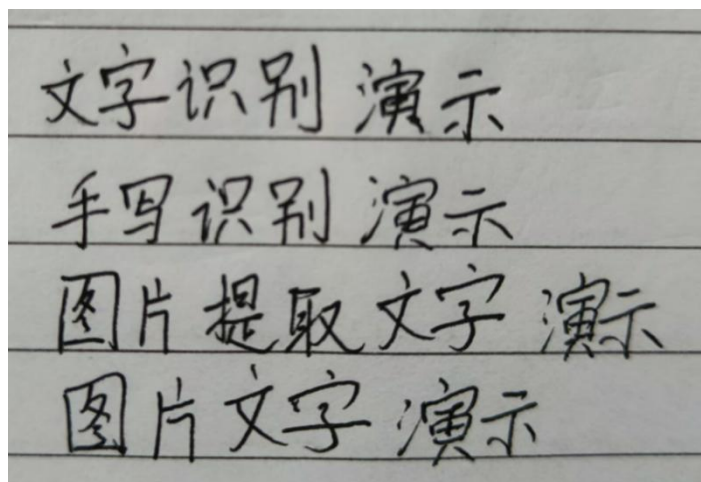
指令学习



这条指令可以对图像进行便捷识别，并获取识别到的文字。

程序编写

示例 1：便捷识别文字



使用手写文字图片，将图片导入到 images 文件夹中，命名为 text_rec.png
打开范例代码 AI 视觉算法-19.文字识别-便捷 并运行。



运行结果如下所示，可以正常识别：

```
[09:07:11] --> Init runtime environment
[09:07:11] I RKNN: [09:07:11.021] RKNN Runtime Information, librknrt
version: 2.3.2 (429f97ae6b@2025-04-09T09:09:27)
[09:07:11] I RKNN: [09:07:11.021] RKNN Driver Information, version: 0.9.8
[09:07:11] I RKNN: [09:07:11.022] RKNN Model Information, version: 6,
toolkit version: 2.3.2(compiler version: 2.3.2
(e045de294f@2025-04-07T19:48:25)), target: RKNPU v2, target platform:
rk3588, framework name: ONNX, framework layout: NCHW, model inference
type: static_shape
[09:07:11] W RKNN: [09:07:11.039] query RKNN_QUERY_INPUT_DYNAMIC_RANGE
error, rknn model is static shape type, please export rknn with
dynamic_shapes
[09:07:11] done
[09:07:11] Model-/home/cxdz/jupyter/lib/./model/ppocrv4_rec.rknn is rknn
model, starting val
[09:07:11] 文字识别系统初始化成功
[09:07:11] [错误] W Query dynamic range failed. Ret code:
RKNN_ERR_MODEL_INVALID. (If it is a static shape RKNN model, please ignore
the above warning message.)
[09:07:11] 文字识别系统资源已清理
[09:07:11]
[09:07:11] 文字识别演示 手写识别演示 图片提取文字演示 图片文字演示
[09:07:12] [成功] 程序运行完成
```

示例 2：OCR 识别器

同样使用上方的手写文字图片，这次使用 OCR 识别器进行识别。

打开范例代码 AI 视觉算法 20.文字识别-OCR 识别器 并运行。



识别结果如下所示：

```
[09:07:11] --> Init runtime environment
[09:07:11] I RKNN: [09:07:11.021] RKNN Runtime Information, librknrt
version: 2.3.2 (429f97ae6b@2025-04-09T09:09:27)
[09:07:11] I RKNN: [09:07:11.021] RKNN Driver Information, version: 0.9.8
[09:07:11] I RKNN: [09:07:11.022] RKNN Model Information, version: 6,
toolkit version: 2.3.2(compiler version: 2.3.2
(e045de294f@2025-04-07T19:48:25)), target: RKNPU v2, target platform:
rk3588, framework name: ONNX, framework layout: NCHW, model inference
type: static_shape
[09:07:11] W RKNN: [09:07:11.039] query RKNN_QUERY_INPUT_DYNAMIC_RANGE
error, rknn model is static shape type, please export rknn with
dynamic_shapes
[09:07:11] done
[09:07:11] Model-/home/cxdz/jupyter/lib/./model/ppocrv4_rec.rknn is rknn
model, starting val
[09:07:11] 文字识别系统初始化成功
[09:07:11] [错误] W Query dynamic range failed. Ret code:
RKNN_ERR_MODEL_INVALID. (If it is a static shape RKNN model, please ignore
the above warning message.)
[09:07:11] 文字识别系统资源已清理
[09:07:11]
[09:07:11] 文字识别演示 手写识别演示 图片提取文字演示 图片文字演示
[09:07:12] [成功] 程序运行完成
```

便捷识别和 OCR 识别器性能接近；其中便捷识别内部自动管理资源，调用后立即释放，适合单次识别任务；OCR 识别器适合需要长期保持识别服务的场景，同时返回了更多内容，方便调试；使用时根据实际情况使用其中一种即可。

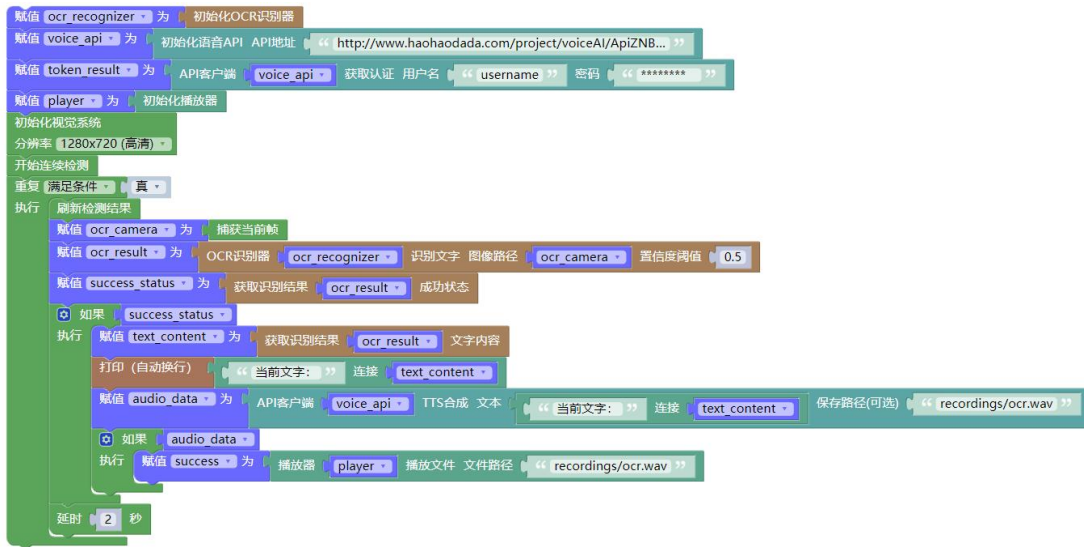
示例 3：拍照文字识别

接下来对实物进行拍照并识别其中的文字。

打开范例代码 AI 视觉算法-21.实时文字识别 并运行。

修改用户名和密码。

摄像头会识别文字并播报当前文字，每隔 2s 识别一次。



物联网

物联网（IOT）是指通过信息传感设备，按照约定协议，将任何物体与网络相连接，实现智能化识别、定位、跟踪、监控和管理的网络系统。

这里主要介绍物联网技术中的 MQTT。

MQTT

物联网协议分为两大类，一类是传输协议，一般负责子网内设备间的组网及通信，如 IPV4，IPV6 等；另一类是通信协议，负责设备通过互联网进行数据交换及通信，如 MQTT、HTTP 等。通信环境不同，相应支持的协议也不同。



MQTT 协议就是一种基于 TCP/IP 协议的轻量级消息发布/订阅传输协议，适用于低功耗、低速网络和低处理能力的设备，常用于物联网中对传感器数据进行发布和订阅，实现实时监测和远程控制等功能。

MQTT 发布/订阅模型是一种发布或订阅型协议，能实现“一对多”通信。

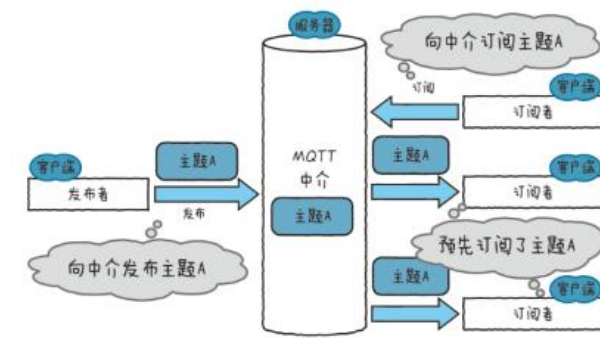
MQTT 发布/订阅模型有四个重要组成部分：发布者、订阅者、中介、主题。

发布者（Publisher）将消息发布到主题；发布者一次只能向一个主题发布消息，发布消息时无需关注订阅者是否在线。

中介（Broker）负责承担起转发 MQTT 通信的服务器的作用，接收发布者的消息，并将信息分发给已订阅该主题的任何订阅者；

订阅者（Subscriber）通过订阅主题消息接收消息，一次可以订阅多个主题。

主题（Topic）是 MQTT 进行消息路由的基础，使用/进行分层，如 topic/light



指令学习

MQTT 初始化	
客户端ID	“ haoda ”
用户名	“ ”
密码	“ ”
服务器地址	“ broker.emqx.io ”
服务器端口	1883
保活时间	60

这条指令是一条设置参数的通用类型 MQTT 指令。

客户端 ID：必须填写，内容随意，全局唯一，不同设备客户端 ID 不可相同。

用户名：客户端连接服务器端的账号 user，使用 BIOT 时默认不填写

密码：客户端连接服务器端的密码 Password，使用 BIOT 时默认不填写

服务器地址：本地服务器或云服务的 IP 地址，填写内容详见下文

服务器端口：port，默认为 1883，不同平台可能有区别

保活时间：keepalive，通过客户端定期发送消息来实现，以确保连接保持活动状态；如果网络稳定，一般设置 30/60；如果应用对实时性要求高，需要及时获取和相应消息，一般设置 0s，以此快速检测到连接丢失。

指令默认设置相关参数并连接 MQTT。

MQTT 发送消息 主题	“ haoda/mqtt/msg ”	消息	“ haohaodada ”
--------------	--------------------	----	----------------

这条指令用来向某个主题发送 MQTT 消息。消息类型为文本，主题格式则按照服务器端的具体要求，用 / 隔开。



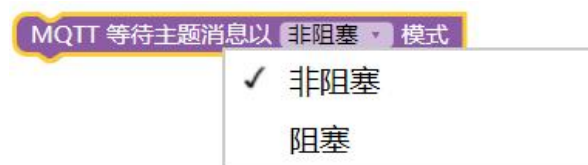
当从某个主题订阅到 MQTT 消息时，可以使用这条指令，client、userdata、msg 属于进阶数据，正常情况下可以不作使用。

MQTT 接收的消息 (字典)

这条指令表示 MQTT 接收的消息。消息的格式是字典，包含三个内容，主题 topic，消息内容 payload，消息传输的服务质量 qos。

如下图，topic 名称为 topic/rgb；消息内容为 close；消息传输服务质量为 0（表示至多分发一次）。

```
{'topic': 'topic/rgb', 'payload': 'close', 'qos': 0}
```



这条指令用于 MQTT 订阅消息，分为阻塞和非阻塞模式。

阻塞模式：一般放在重复执行中，阻止其他代码的执行，直到接收到 MQTT 消息为止。收到消息后会继续执行后续操作。

非阻塞模式：用于检查是否有新的 MQTT 消息，但不会阻塞代码的执行。如果有则处理该消息，如果没有则立即返回，剩余代码继续执行。

如果代码在接收到 MQTT 消息时需要立即做出响应，并且不需要同时执行其他操作时，需要用到阻塞模式；其余时候使用非阻塞模式。

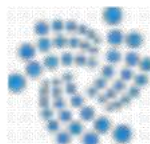
BIOT

MQTT 连接的服务器可以是本地服务器，也可以是云服务器。

这里先对如何使用本地服务器进行介绍。

BIOT 是一款基于 MQTTV3.1 版本，可以快速建立 MQTT 本地服务器的软件。

安装好搭 Block 后，BIOT 会自动安装到电脑上并在桌面出现快捷方式，具体图标如下。双击打开 BIOT，一个标准的 MQTT 服务器就在本地计算机搭建完成了，可以使用 MQTT 客户端与服务器进行通讯。

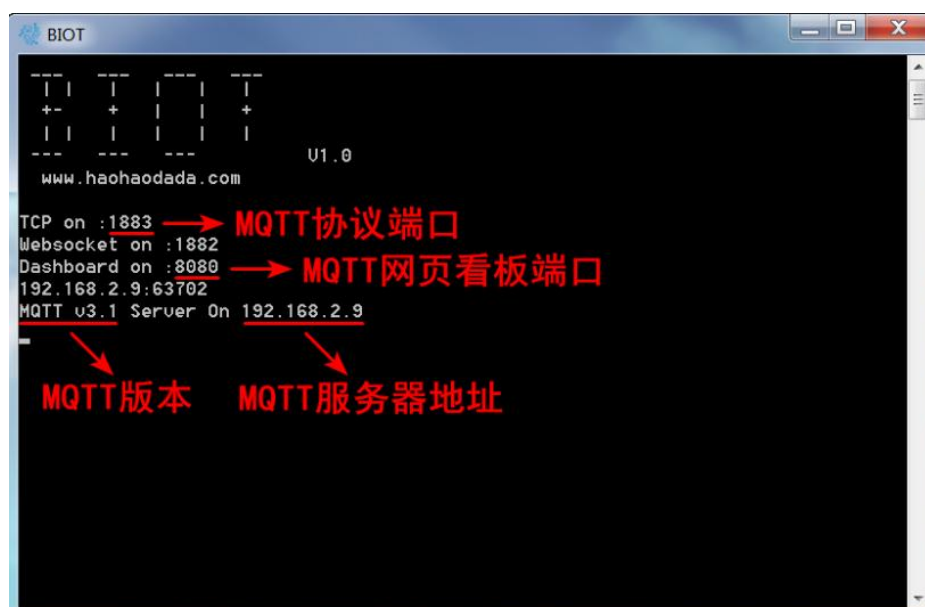


打开 BIOT 后，会出现以下界面；在这个界面下，我们可以看到 MQTT 本地服务器地址、MQTT 协议端口以及稍后会出现的 MQTT 消息相关内容。

以下图为例，我们可以获取到：

服务器地址 server : 192.168.2.9

MQTT 端口 port : 1883



与此同时，网页端会自动打开一个页面，用于显示 MQTT 消息的具体内容，我们称之为网页看板。

网页看板有三种参考地址：

参考地址一：<http://127.0.0.1:8080>（BIOT 启动后会自动启动浏览器打开此地址）

参考地址二：<http://localhost:8080>

参考地址三：[http://计算机局域网 IP 地址:8080](http://计算机局域网IP地址:8080)（以上图为例，局域网设备可以访问 <http://192.168.2.9:8080>）



接下来我们通过本地服务器，展示如何发布和订阅 MQTT 消息。

程序编写

示例 1：MQTT 发布-本地服务器

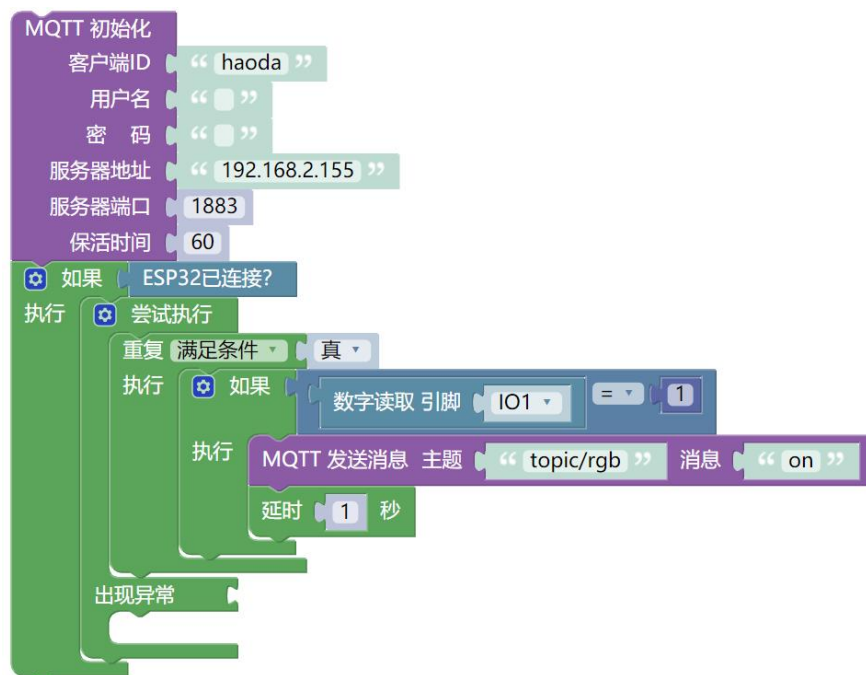
打开并运行 BIOT，快速创建一个本地服务器。



运行后界面如上所示，服务器的 IP 地址为 192.168.2.155。

打开范例代码 物联网-2.MQTT 发布

修改服务器地址和主题，将单按键连接到 IO1，运行程序



运行后，MQTT 服务器会接收到消息，BIOT 界面会显示。

```
<< OnConnect client connected haoda: {FixedHeader:{Remaining:21 Type:1 Qos:0 Dup:false Retain:false} AllowClients:[ Topics:[ ReturnCodes:[ ProtocolName:[77 81 84 84] Qoss:[ Payload:[ Username:[ Password:[ WillMessage:[ ClientIdentifier:haoda TopicName: WillTopic: PacketID:0 Keepalive:60 ReturnCode:0 ProtocolVersion:4 WillQos:0 ReservedBit:0 CleanSession:false WillFlag:false WillRetain:false UsernameFlag:true PasswordFlag:true SessionPresent:false}
< OnMessage modified message from client haoda topic/rgb on
< OnMessage modified message from client haoda topic/rgb on
< OnMessage modified message from client haoda topic/rgb on
< OnMessage modified message from client haoda topic/rgb on
```

好搭 AI 派的串口消息显示区，也会显示相应内容：

```
程序输出
[14:07:31] [成功] 程序启动成功, PID: 25243
[14:07:31] init ESP32
[14:07:31] 正在自动检测ESP32设备...
[14:07:31] 发现可用串口: ['/dev/ttyCH341USB1', '/dev/ttyCH341USB0']
[14:07:31] 正在测试串口: /dev/ttyCH341USB1
[14:07:31] ✓ 成功检测到ESP32设备: /dev/ttyCH341USB1
[14:07:31] <ESP32.mSerial object at 0x7f9ca59580>
[14:07:31] ✓ 成功连接到ESP32设备: /dev/ttyCH341USB1
[14:07:36] 发送成功: 消息 on 到主题 topic/rgb
[14:07:36]
[14:07:37] 发送成功: 消息 on 到主题 topic/rgb
[14:07:39] 发送成功: 消息 on 到主题 topic/rgb
[14:07:40] 发送成功: 消息 on 到主题 topic/rgb
```

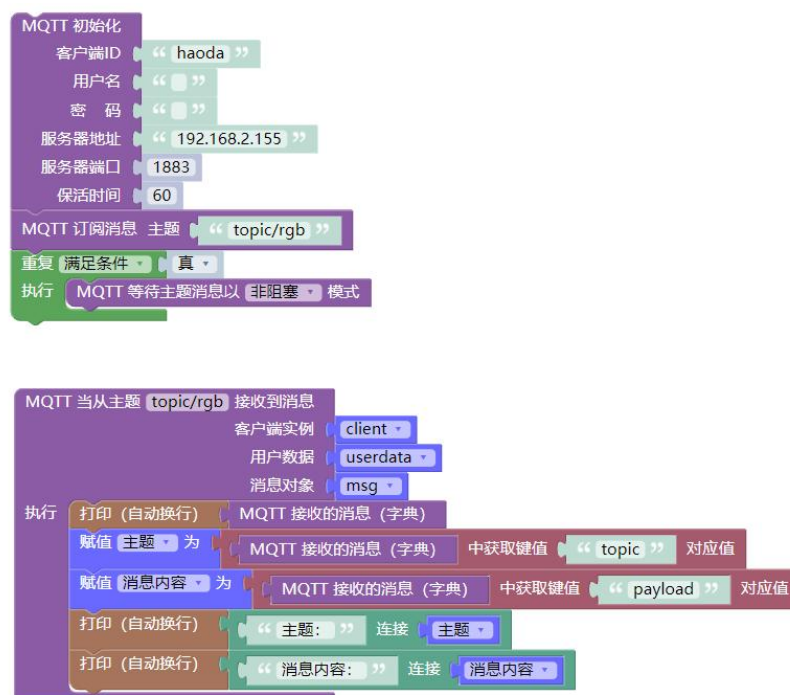
示例 2: MQTT 订阅-本地服务器

打开范例代码 物联网-3.MQTT 订阅，修改服务器地址和主题并运行。

BIOT 照常打开，保证服务器正常运作。

使用其他物联网设备或者 MQTTX，向服务器主题 topic/rgb 发送消息。

查看好搭 AI 派的 MQTT 订阅端的具体内容。

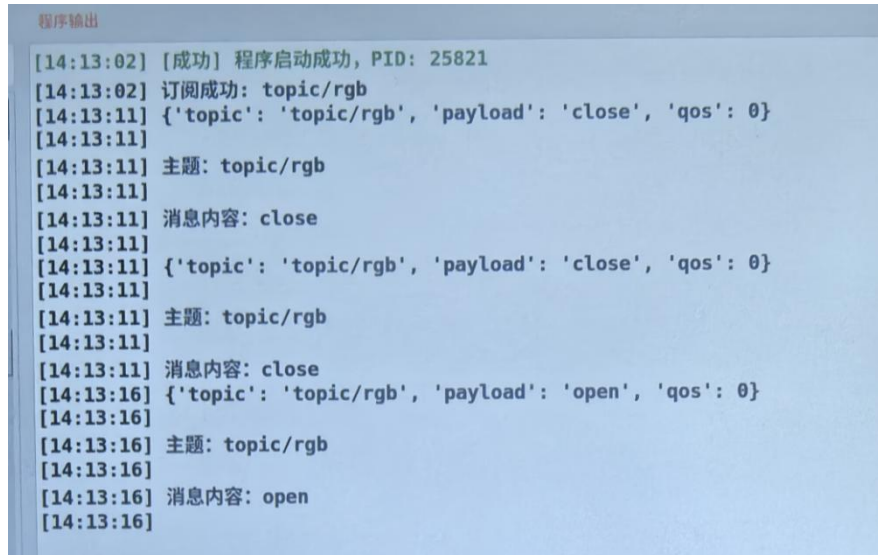


BIOT 服务器端界面如下所示：

```
<< OnDisconnect client disconnected haoda: reader: EOF
<< OnConnect client connected haoda: {FixedHeader:{Remaining:21 Type:1 Qos:0 Dup:false Retain:false} AllowClients:[] Topics:[] ReturnCodes:[] ProtocolName:[77 81 84 84] Qos:[] Payload:[] Username:[] Password:[] WillMessage:[] ClientIdentifier:haoda TopicName: WillTopic: PacketID:0 Keepalive:60 ReturnCode:0 ProtocolVersion:4 WillQos:0 ReservedBit:0 CleanSession:false WillFlag:false WillRetain:false UsernameFlag:true PasswordFlag:true SessionPresent:false}
< OnMessage modified message from client d2jb6j4o27klj6p4vlg topic/rgb close
< OnMessage modified message from client d2jb6j4o27klj6p4vlg topic/rgb close
< OnMessage modified message from client d2jb6j4o27klj6p4vlg topic/rgb open
```

好搭 AI 派消息界面如下所示。

可以看到，已经成功接收到消息，主题是 `topic/rgb`，消息内容是 `open`。



程序输出

```
[14:13:02] [成功] 程序启动成功, PID: 25821
[14:13:02] 订阅成功: topic/rgb
[14:13:11] {'topic': 'topic/rgb', 'payload': 'close', 'qos': 0}
[14:13:11]
[14:13:11] 主题: topic/rgb
[14:13:11]
[14:13:11] 消息内容: close
[14:13:11]
[14:13:11] {'topic': 'topic/rgb', 'payload': 'close', 'qos': 0}
[14:13:11]
[14:13:11] 主题: topic/rgb
[14:13:11]
[14:13:11] 消息内容: close
[14:13:16] {'topic': 'topic/rgb', 'payload': 'open', 'qos': 0}
[14:13:16]
[14:13:16] 主题: topic/rgb
[14:13:16]
[14:13:16] 消息内容: open
[14:13:16]
```

OpenCV

OpenCV 是一个跨平台计算机视觉和机器学习开源库，支持支持 Linux 、 Windows 、 Android 、 Mac OS 等系统，提供图像处理、视频分析、人脸识别、物体识别等功能。

这里主要介绍好搭 AI 派的图像处理功能与视频处理功能。

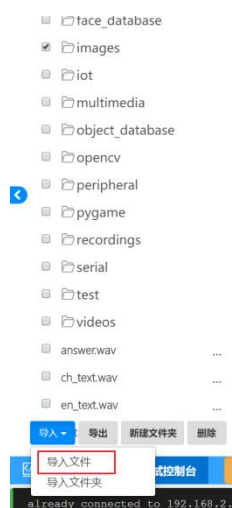
一、图像处理

好搭 AI 派的图像处理功能，包括图片显示、图片绘制、图像调整、图像截取、图像旋转、高斯模糊、边缘检测等功能。

指令学习



这条指令用于读取好搭 AI 派中的图片文件。如上方的第二条指令，就是到文件夹中 `images` 中读取图片 `example.jpeg`。我们需要提前将这些图片文件，通过文件管理导入到文件夹中。一般来说，推荐将图片都导入到文件夹 `images` 中，方便管理，具体如下图所示。





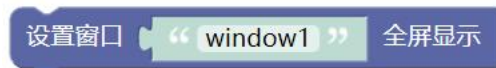
这条指令表示读取到的图片最终需要显示到某个窗口中。



这条指令可以创建一个窗口。在 OpenCV 中，窗口（Window）是用于显示图像、视频帧或可视化结果的图形界面元素。它提供了一个简单的 GUI 接口，无需依赖其他库即可实现图像显示和基础交互。

其中 `windows1` 是这条指令创建窗口的名称。

窗口可以同时创建多个。



这条指令可以调整窗口的大小，横是宽，竖是高。窗口最大可以调整为 1920×1080 。此时该指令和另一条指令设置窗口全屏显示的效果相同。



OpenCV 的窗口系统基于事件驱动模型，上方的等待按键按下指令，即 `waitKey()` 是维持事件循环（如窗口刷新、鼠标/键盘输入）的唯一方法。没有它会导致：窗口无法响应任何交互，图像显示卡死（窗口线程阻塞）。

其中第一条指令是等待 5000 毫秒，同时处理事件；等待任意按键按下指令，则是检测到任意按键被按下后，就继续下一步。常用写法有以下两种：



这种写法表示延时 5000ms，在这 5000ms 中，事件维持，例如常见的图片显示，5000ms 后触发事件，运行下一条程序。



这种写法表示每隔 25ms 进行一次事件检测，当检测 q 按键时就中断循环，否则会一直保持原事件循环。

这里的 q 按键并不是指外接的按键模块，而是指键盘按键输入。我们可以通过 USB 接口连接一个外置键盘。选中想要关闭的窗口，按下 q 按键，即可关闭；也可以右上角手动关闭。

这里的按键事件，一般用于 OpenCV 的图片处理和视频处理，方便快速退出图片或正在播放的视频，但并非停止整个程序。

而使用单纯的延时指令，如 `time.sleep`，会冻结整个窗口。



第一条指令可以关闭所有窗口，是最后清理 OpenCV 资源的常见写法。如果只希望关闭部分窗口，则可以使用第二条指令。

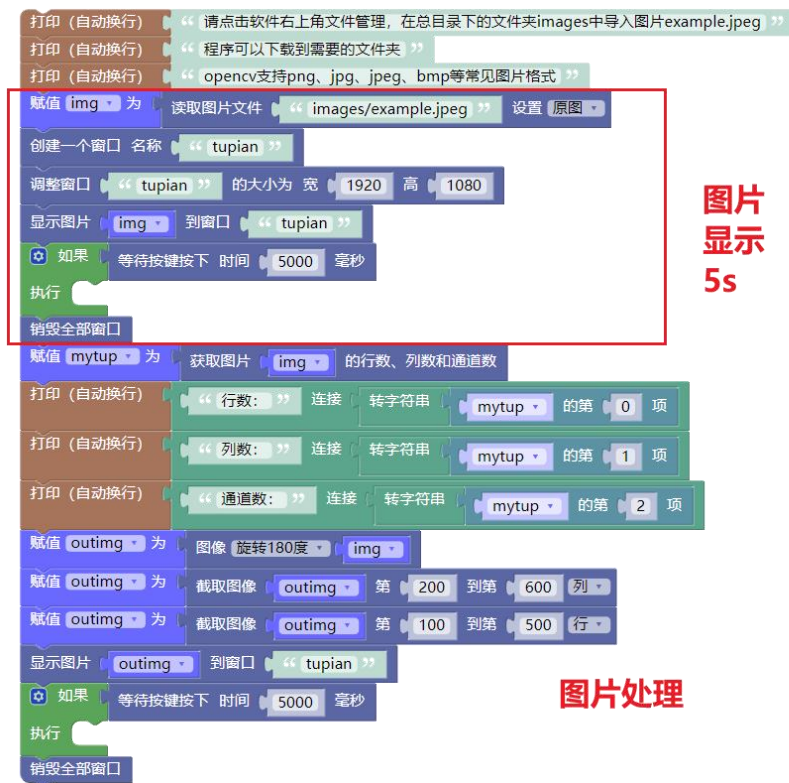
程序编写

示例 1：图片显示

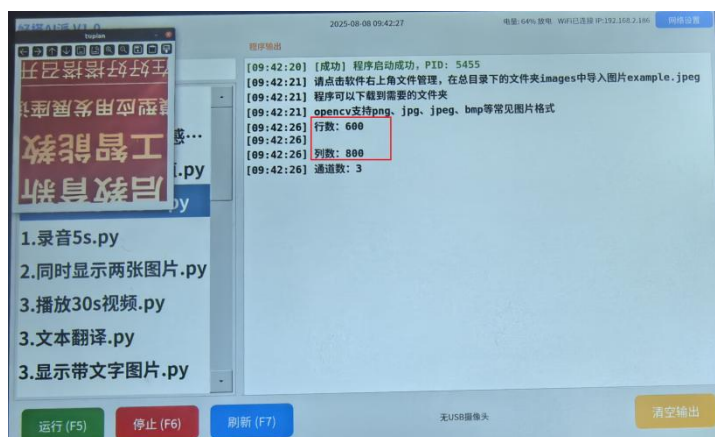
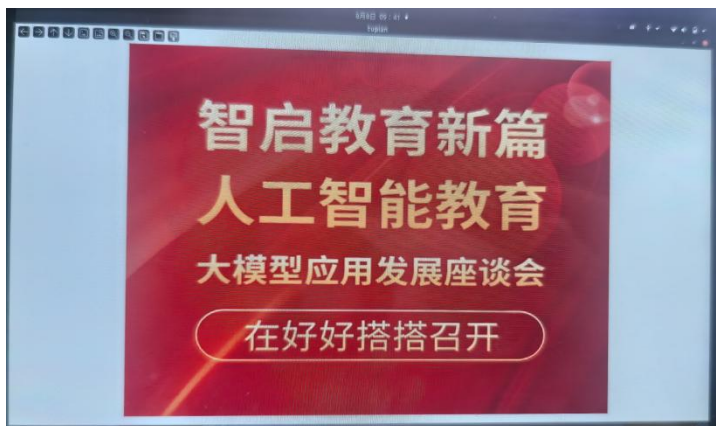
打开范例代码 OpenCV-1.图片显示与处理。

我们可以使用好搭 AI 派内部自带的图片，也可以通过点击好搭 Block 软件右上角的文件管理，在 images 文件夹中导入自己想要显示的图片，并进行命名。图片支持 png、jpg、jpeg、bmp 等常见格式。

程序先显示 5s 图片，接着对图片进行了处理。串口打印的行数列数代表着图像的横宽像素点，表明图片大小。1920*1080 是窗口大小，可以自行调整。



具体显示效果如下：



示例 2：多张图片显示与按键关闭

打开范例代码 OpenCV-2.同时显示两张图片。

我们不仅可以更改图片显示窗口的大小，还可以更改显示的位置。



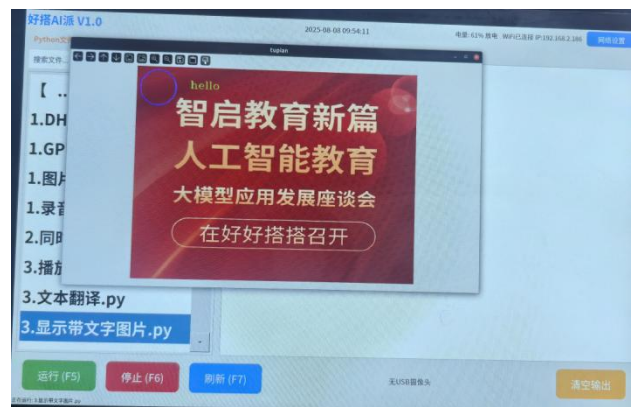
具体显示效果如下所示：



示例 3：图片绘制

打开范例代码 OpenCV-3.显示带文字图片。

我们可以在图片上绘制图案与文本。具体效果如下所示：



二、视频处理

好搭 AI 派的视频处理包括视频的输入捕获与输出、帧处理等。摄像头相关使用请参考 AI 视觉算法 部分。

指令学习

赋值 vd 为 创建VideoCapture对象

VideoCapture 是计算机视觉和视频处理领域常用的技术，用于通过视频捕捉设备（如摄像头、摄像机）捕获视频数据。这条指令类似初始化，创建了一个 VideoCapture 对象。

关闭VideoCapture对象 vd

这条指令用于关闭 VideoCapture 对象。

使用VideoCapture对象 vd 打开视频文件 “ video.mp4 ”

这条指令用于打开某个视频文件。需要注意的是，OpenCV 在读取打开视频时，默认只处理视频流，即图像帧，音频流需要我们使用其他方法进行播放，比如调用 pygame 库的音频播放功能。具体可以查看范例代码 6.播放视频与音频。

从VideoCapture对象 vd 中抓取下一帧 grab 以及状态 ret

这条指令是 VideoCapture 处理视频流的关键指令，主要包含两个变量，grab 代表读取的图像帧，用来在窗口上显示；ret 则表示状态，一般如下图所示使用：



这里事件检测间隔时间不宜过短，25-30ms 比较合适，不然视频会卡顿。

程序编写

示例 1：视频播放

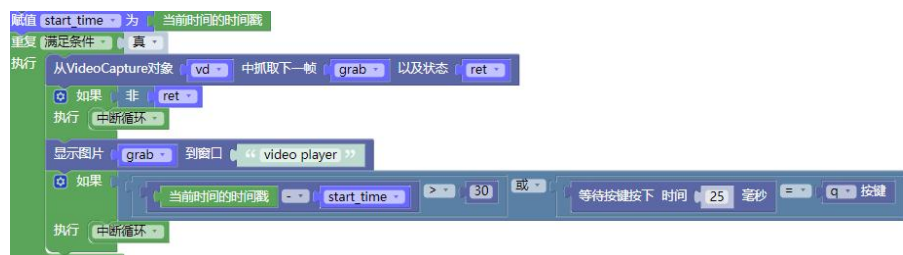
打开范例代码 [OpenCV-4.播放视频](#)。

我们可以使用好搭 AI 派内部自带的视频，也可以自行导入视频。推荐将视频导入到文件夹 `videos` 中。

程序运行后即可播放视频。选中视频窗口，点击 `q` 按键退出视频。



如果希望视频播放一定时长后关闭，可以使用时间戳指令，具体写法如下图所示。如下图，就是视频播放 30s。具体可以查看范例代码 [5.播放 30s 视频](#)。



示例 2：播放视频与音频

打开范例代码 [OpenCV-6.播放视频与音频](#)。

这个程序不仅调用了 `OpenCV` 的视频播放，还调用了语音 `AI` 的语音合成功能，以及 `pygame` 的声音播放功能。

`pygame` 音频播放相关的指令请跳转 [pygame](#) 部分进行学习。

打印 (自动换行) “ 请在videos文件夹中导入视频文件 ”

打印 (自动换行) “ 请在recordings文件夹中导入音频文件 ”

打印 (自动换行) “ 请修改视频和音频的名称 ”

打印 (自动换行) “ 请输入通过认证的用户名和密码 ”

赋值 voice_api 为 初始化语音API API地址 “ http://www.haohaodada.com/project/voiceAI/ApiZNB...”

赋值 token_result 为 API客户端 voice_api 获取认证 用户名 “ username ” 密码 “ ***** ”

如果 非 token_result

执行 打印 (自动换行) “ ✖ 认证失败, 无法测试LLM功能 ”

赋值 player 为 初始化播放器

赋值 recorder 为 初始化录音器 采样率 16000 声道数 1

语音合成 参数:

文本内容 “ 马上为您播放 ”

播放视频 参数:

videoname “ example.mp4 ”

playername “ example.mp3 ”

playtime 30

播放视频 参数: videoname, playername, playtime

执行 初始化Pygame

初始化录音器

创建 声音 sound1 文件名 “ recordings/ ” 连接 playername

赋值 vd 为 创建VideoCapture对象

使用VideoCapture对象 vd 打开视频文件 “ videos/ ” 连接 videoname

如果 非 使用VideoCapture对象 vd 是否初始化完成

执行 打印 (自动换行) “ 无法打开视频 ”

否则 创建一个窗口 名称 “ video player ”

调整窗口 “ video player ” 的大小为 宽 1620 高 1080

赋值 start_time 为 当前时间的时间戳

声音 sound1 播放

重复 满足条件 真

执行 从VideoCapture对象 vd 中抓取下一帧 grab 以及状态 ret

如果 非 ret

执行 中断循环

显示图片 grab 到窗口 “ video player ”

如果 当前时间的时间戳 - start_time > playtime 或 等待按键按下 时间 25 毫秒 或 按键

执行 中断循环

声音 sound1 停止

关闭VideoCapture对象 vd

销毁全部窗口

语音合成 参数: 文本内容

执行 赋值 audio_data 为 API客户端 voice_api TTS合成 文本 文本内容 保存路径(可选) “ zhishi.wav ”

如果 audio_data

执行 赋值 success 为 播放器 player 播放文件 文件路径 “ zhishi.wav ”

如果 success

执行 打印 (自动换行) “ ✔ 播放完成 ”

播放器 player 清理资源

Pygame 交互界面

Pygame 是一个基于 Python 的开源 2D 游戏开发库，它基于 SDL（Simple DirectMedia Layer）构建，提供跨平台的游戏开发支持。

一、窗口

Pygame 窗口是游戏的主要显示界面，是用户可见的图形界面容器，其核心功能通过 `pygame.display` 模块实现。我们可以对窗口进行处理并显示部分内容。

指令学习

初始化Pygame

```
import pygame
```

```
pygame.init()
```

这条指令用于 `pygame` 的初始化。使用 `pygame` 相关功能，必须使用这条初始化指令。

初始化窗口 窗口名称 `window1` 宽度 `800` 高度 `600`

```
window1 = pygame.display.set_mode(size=(800,600), flags=0, depth=0)
```

这条指令用于初始化 `pygame` 的窗口，命名为 `windows1` 并设置尺寸大小为 `800*600`。

设置窗口标题 `"Pygame基础窗口"`

```
pygame.display.set_caption('Pygame 基础窗口')
```

这条指令可以重新设置窗口的标题名称。

设置窗口名称后，效果如图所示：



更新表面对象至屏幕

pygame.display.flip()

这条指令可以让窗口的屏幕上更新相应内容，类似与 OLED 屏幕的屏幕更新显示指令。默认显示黑屏，但窗口依然存在。

程序编写

示例 1：窗口显示

打开范例代码 `pygame-1.窗口显示`。

运行后会出现一个黑色的窗口，名称叫 `Pygame 基础窗口`。



二、表面

表面是 Pygame 中的基础绘图单元，相当于内存中的画布，通过 `pygame.Surface()` 创建。窗口是最顶层 Surface，其他 Surface 需要使用 `blit` 指令到窗口才会显示。我们一般先在后台 Surface 绘制图形/文字/图像，最后整体 `blit` 到窗口 Surface 并调用 `pygame.display.flip()` 刷新显示。

指令学习



s1 = pygame.Surface(size=(800, 600))

这条指令用于 pygame 的初始化。使用 pygame 相关功能，必须使用这条初始化指令。



window1.blit(s1,(1,1))

这条指令在 display 中，用于将创建的表面显示在指定的窗口上。注意表面对象的变量名称与窗口的变量名称。



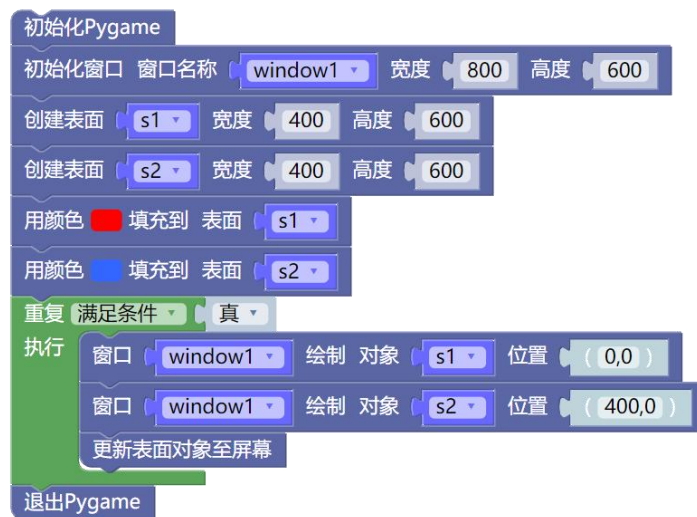
s1.fill((255,0,0))

这条指令可以用红色填充表面 s1。

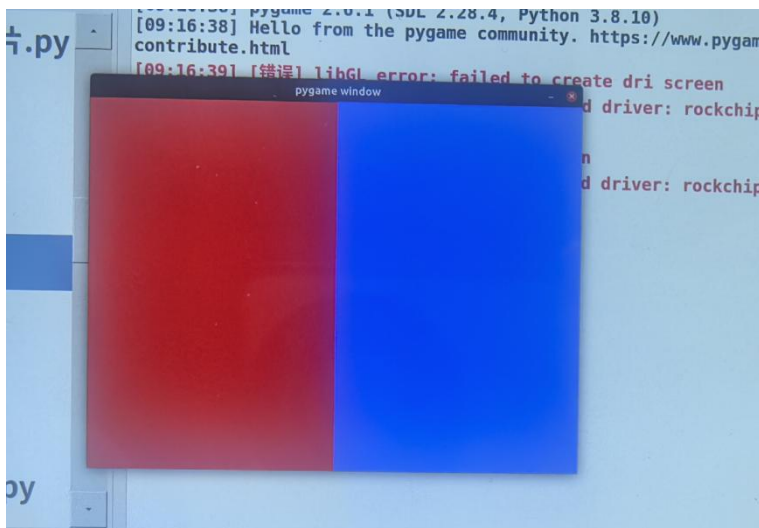
程序编写

示例 1：表面绘制

打开范例代码 pygame-2.表面绘制 并运行。



运行后会出现一个窗口，窗口上有两个表面，如下所示：



三、事件

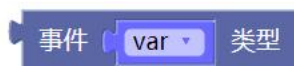
Pygame 中的事件检测，主要是指检测键盘按键和鼠标点击事件并输出信息。

指令学习

窗口事件

pygame.event.get()

这条指令表示好搭 AI 派中窗口检测到的每一个事件。

**var.type**

这条指令表示事件 `var` 的类型。其中变量 `var` 就表示事件。



pygame.ACTIVEEVENT

• • • • •

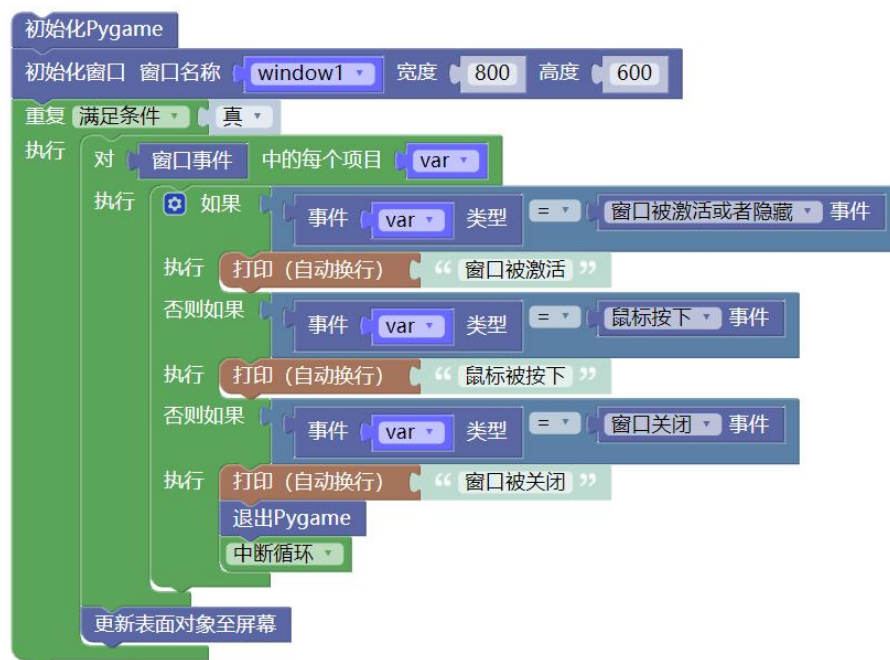
pygame.QUIT

这条指令表示各类事件，包括窗口相关操作、鼠标相关操作、键盘相关操作。其中窗口和鼠标可以直接检测，键盘检测则需要外接 USB 键盘。

程序编写

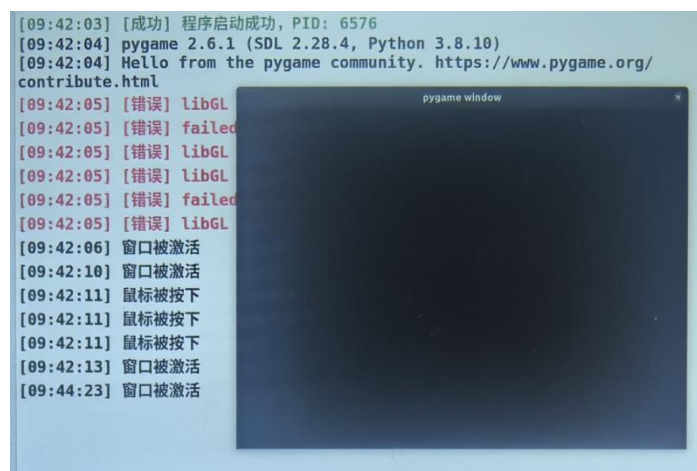
示例 1：事件检测

打开范例代码 `pygame-3.事件检测` 并运行。



当屏幕被激活，或者单击触摸屏幕时，会显示以下信息。

具体效果如下显示：



四、字体

Pygame 中的 font 模块，主要用于加载和渲染字体，并将字体渲染成 surface 对象，主要通过 `pygame.font.Font()` 指令。

指令学习

初始化字体

`pygame.font.init()`

这条指令表示对字体进行初始化，是使用字体的必要指令。



`pygame.font.Font('freesansbold.ttf', 14)`

`font = pygame.font.SysFont('freesansbold.ttf', 14)`

这条指令用于设置字体的类型和大小。可以选择自定义字体或者系统字体。系统字体是从操作系统库中加载，无需额外文件；自定义字体则需要指定文件路径，但可以设置更复杂的属性，如渐变阴影等，适合进阶使用。

通常我们会将字体赋值给变量 `font`。



下载上方程序，串口打印信息如下所示。

Hello from the pygame community.

<https://www.pygame.org/contribute.html> （pygame 网址查看）

['comicansms', 'tlwgtypo', 'dejavuserif', 'urwbookman', 'kalapi', 'rekha',
'tlwgtypewriter', 'dejavusansmono', 'ubuntumono', 'rachana', 'liberationmono',
.....后略。

中文字体则可以使用系统自带的 `/home/cxdz/jupyter/assets/simfang.ttf` 。



```
text = font.render('hello', True, ((255,0,0),1))
```

这条指令用于使用指定字体 `font`，绘制一串文本并赋值给对象 `text`，再通过 `blit` 指令绘制到窗口。

程序编写

示例 1：文本显示

打开范例代码 `pygame-4.字体显示` 并运行。

这里使用了两种类型字体。一种是系统自带的英文字体，一种是自定义的中文字体。



最终显示效果如下所示：

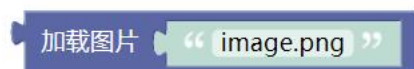


五、图片

Pygame 中的 image 模块，主要用于图像加载、保存等，即 `pygame.image.load('image.png')` 和 `pygame.image.save(o1,'image.png')`，显示图片或者将画面的内容保存为图片。

图片转换处理等功能可以参考 opencv 中的图像处理。

指令学习



`pygame.image.load('image.png')`

这条指令表示加载某张图片，一般配合变量和 `blit` 指令一起使用。加载的格式支持 BMP、GIF、JPEG、LBM、PCX、TGA（未压缩）、TIFF、PNG、SVG 等。



`pygame.image.save(o1,'image.png')`

这条指令表示将某个画面保存成图片。保存的格式仅支持 JPEG、PNG、TGA

程序编写

示例 1：图片显示

打开范例代码 `pygame-5.图片显示` 并运行。



显示结果如下：



六、绘制

Pygame 中的 draw 模块，主要用于基础图形的绘制，包括绘制矩形、圆形、线条等。点的绘制指令在 surface 类别中。

指令学习



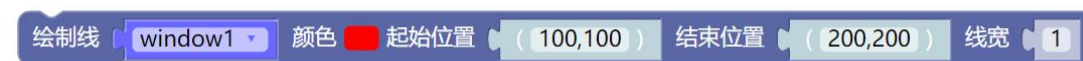
```
pygame.draw.rect(window1,(255,0,0),(0,0,400,600),1)
```

这条指令可以绘制一个 400*600 大小的矩形，矩形左上角从 0,0 开始，绘制矩形的线宽为 1。



```
pygame.draw.circle(window1, (255,0,0), (100,100), 50, 1)
```

这条指令可以绘制一个半径为 50 的圆，圆心坐标为 100,100



```
pygame.draw.line(window1, (255,0,0), (100,100), (200,200), 1)
```

这条指令可以绘制一条直线，并规定了起点与终点的位置。

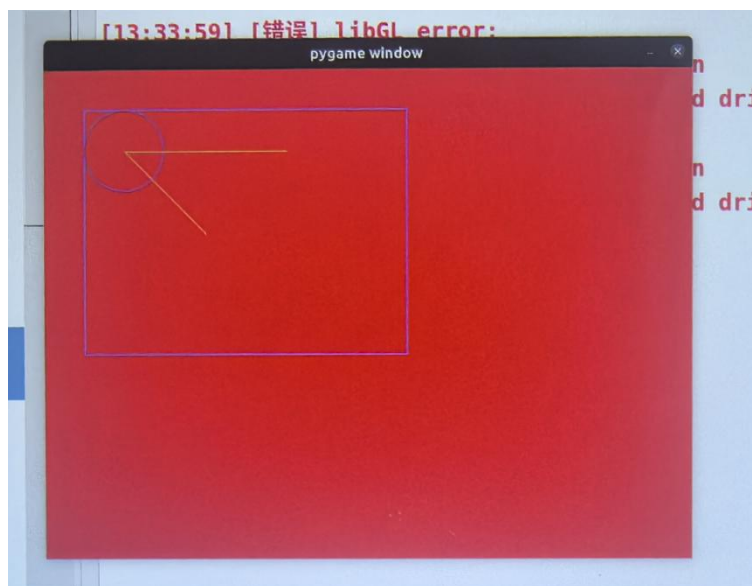
程序编写

示例 1：基础图形绘制

打开范例代码 pygame-6.图形绘制 并运行。



显示结果如下：



七、声音

Pygame 中的 mixer 模块，用于播放音效（如跳跃声、碰撞声），支持 Sound 对象，可同时播放多个短音频文件。

指令学习

初始化混音器

pygame.mixer.init()

使用 mixer 功能必须使用这条指令进行初始化。



sound1 = pygame.mixer.Sound('sound.mp3')

使用这条指令可以设置要播放的音频文件。可以同时设置多个声音。



sound1.play()

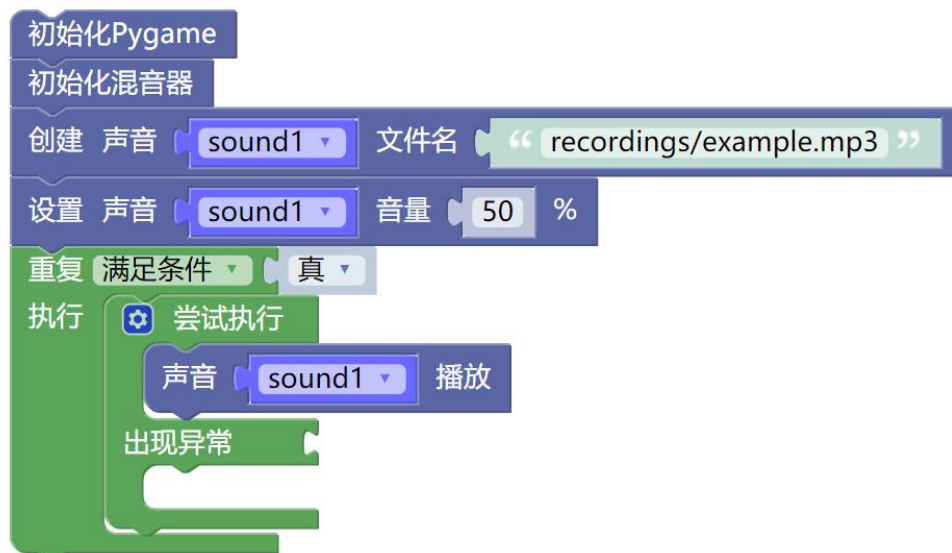
使用这条指令可以播放声音 sound1。如果希望音乐一直播放，需要将这条指令放到重复执行中。

程序编写

示例 1：播放声音

打开范例代码 `pygame-7.播放声音` 并运行。

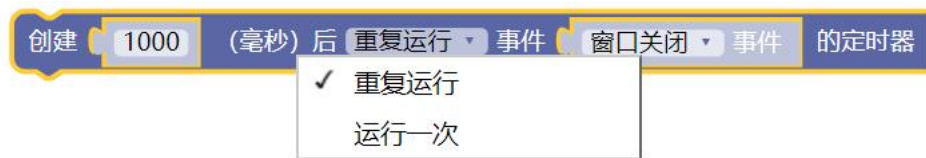
如果播放失败，请查看音频文件是否缺失。



八、定时器

Pygame 中的 time 模块，用于实现定时器和时间获取相关功能。

指令学习



这条指令，每隔 1 秒，向 Pygame 事件队列中添加一个 `pygame.QUIT` 事件。

结合 Pygame 事件处理逻辑，若游戏循环中未对 `pygame.QUIT` 事件做特殊拦截（比如直接退出循环），则会导致每隔 1 秒游戏自动退出（因为 `pygame.QUIT` 事件会触发退出流程）。该指令常用于窗口定时关闭。

获取运行时间（毫秒）

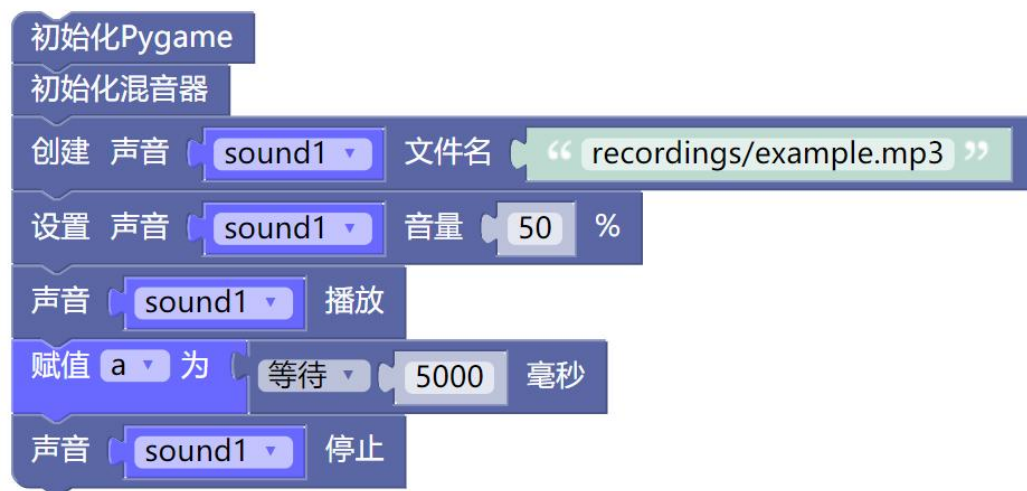
这条指令可以获取当前运行时间。

程序编写

示例 1：播放声音

打开范例代码 `pygame-8.定时器` 并运行。

音乐播放 5s 后停止。



九、音乐

Pygame 中的 `music` 模块，与 `mixer` 相似但不同，主要用于播放背景音乐（如游戏主题曲），支持 MP3、OGG 等格式，适合长时间播放。

指令学习

加载音乐 “ recordings/example.mp3 ”

使用这条指令可以设置要播放的音频文件。

设置音乐 音量 50 %

使用这条指令可以设置音乐播放的音量。

音乐播放

音乐停止

使用这两条指令可以让音乐播放或暂停，一般只执行一次。

音乐是否还在播放？

使用这条指令可以判断音乐是否正在播放。

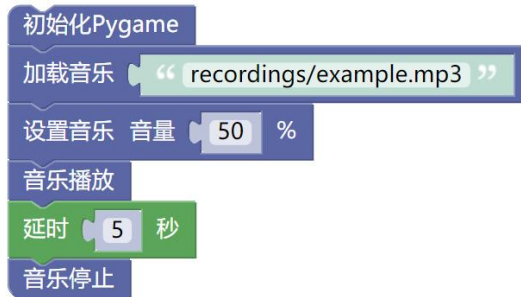
程序编写

示例 1：播放音乐

打开范例代码 `pygame-9.播放音乐` 并运行。

如果播放失败，请查看音频文件是否缺失。

音乐播放 5s 后停止播放。



```
初始化Pygame
加载音乐 “ recordings/example.mp3 ”
设置音乐 音量 50 %
音乐播放
延时 5 秒
音乐停止
```

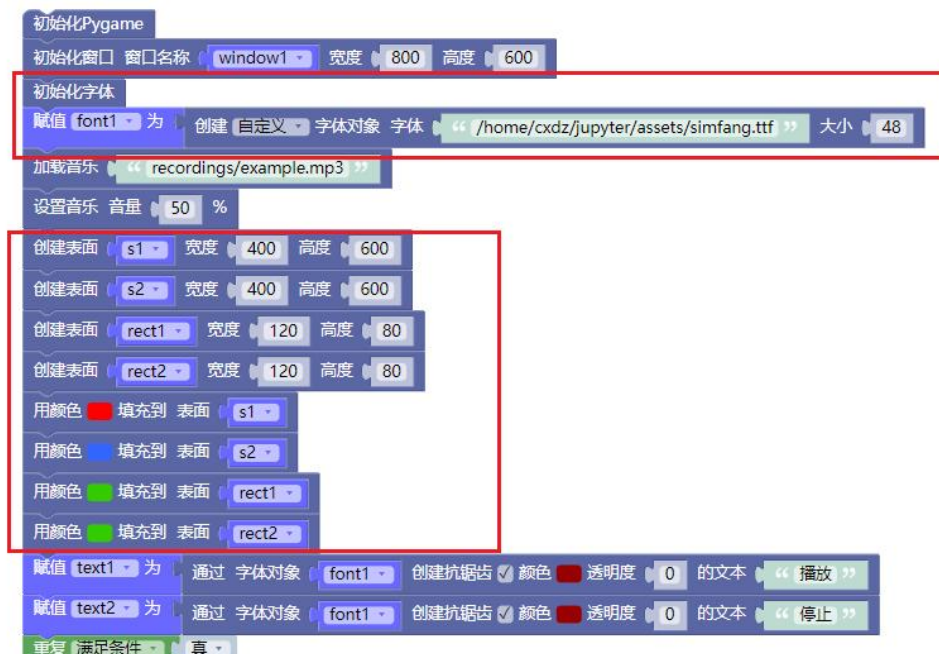
示例 1：按钮控制音乐播放

打开范例代码 `pygame-10.音乐播放-按钮` 并运行。

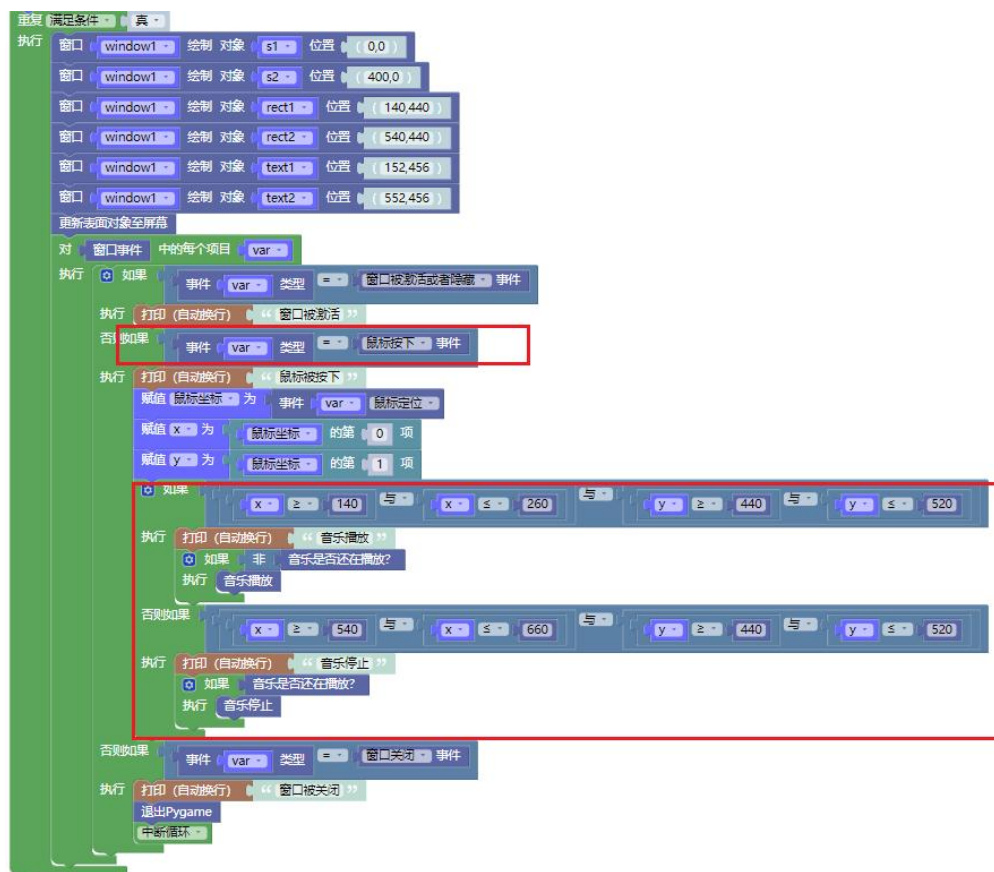
这个程序在窗口界面放置了两个按钮；当判断到鼠标位于某个按钮的区间范围内并按下按键，音乐就会播放。

按钮一 rect1 的位置，左上角坐标 140,440，大小 120*80；

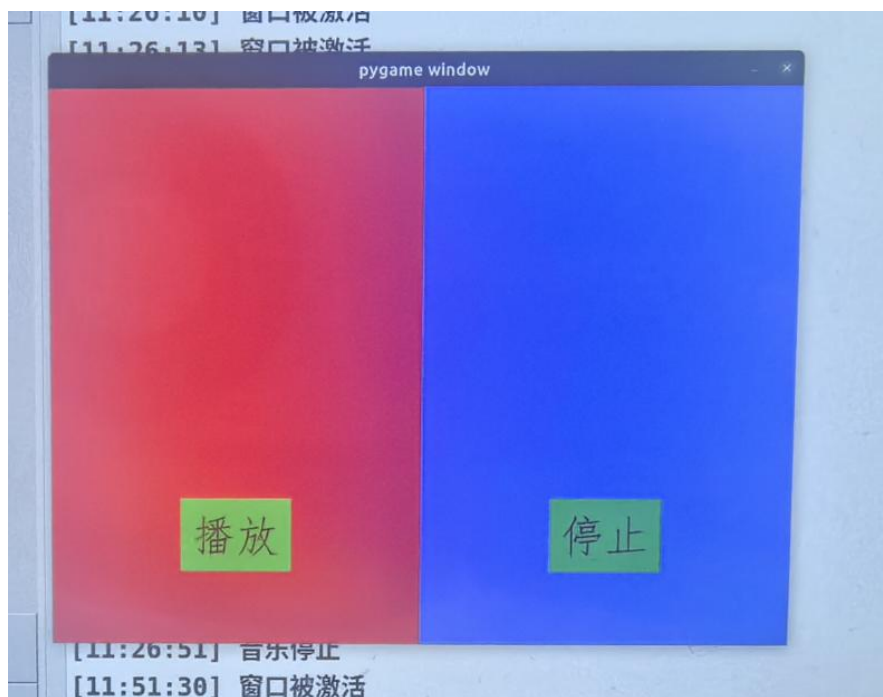
按钮二 rect2 的位置，左上角坐标 540,440，大小 120*80；



当鼠标被点击的事件触发时，记录下鼠标的坐标（x，y），并调用该元祖中的 x 与 y 进行比对，判断是按钮一【播放】被按下，还是按钮二【停止】被按下。



实际效果如下所示：



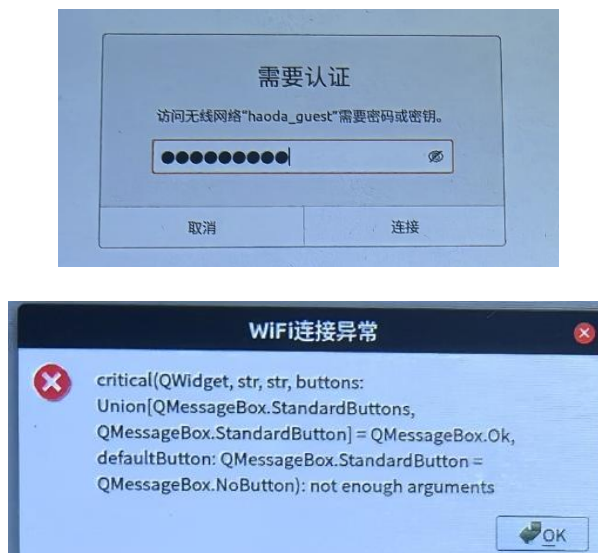
常见问题解决方法

问题一：无法连接到网络

解决方法：

无法连接到网络可能由多种情况导致的。这里列举一些常见情况，用户可以根据自身实际情况进行检查。

①请检查 WiFi 名称和密码是否正确。当提示你需要重新输入 WiFi 密码，或者出现如下报错，均是 WiFi 名称密码输入错误。



②WiFi 支持 2.4G 和 5G，但不支持需要登录认证的网络，如部分校园网、商场网、酒店网，请根据实际情况进行连接。比赛现场推荐使用路由器。

问题二：无法下载程序

解决方法：

①如果使用 IP 网络连接的下载方法，请先检查好搭 AI 派的网络和编程电脑的网络是否处于同一个局域网。

②文件默认下载到根目录中。点击文件管理，勾选文件夹，则会下载到各个文件夹中。

问题三：没电了怎么办

解决方法：

使用套件附带的充电线，连接到好搭 AI 派右下角的充电口，并将开关拨动到右侧，直至主控右上角界面出现【充电】字样。

问题四：突然关机是什么情况？

解决方法：

①好搭 AI 派电量过低，低于 10%时，主控会自动关机。请及时给主控板充电，以备不时之需。

②左上角的长条按钮为音量键，短条按钮为电源键。用户可能不小心误触到电源键导致主控关机。

③长时间没有使用，好搭 AI 派会自动关机。

问题五：从 UI 界面中退出是什么情况？

解决方法：

UI 界面退出后，界面有一个按钮可以一键恢复。

也可以长按左上角的开机按钮，选择【重启】按钮，设备重启后会自动加载默认的 UI 界面。

问题六：程序运行报错

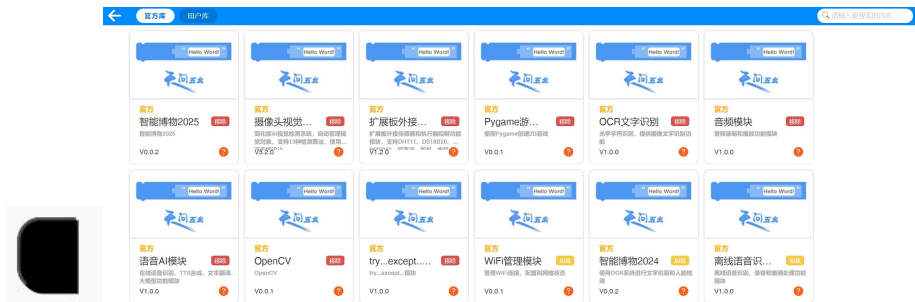
解决方法：

程序报错情况种类较为复杂，推荐联系官方售后进行解答。可以先从硬件连接、摄像头连接、开关按钮、程序本身等方向进行检查。

问题七：程序无法打开

解决方法：

①如果无法打开范例代码，或者打开后如下图所示，是没有添加官方库扩展。请先点击【添加扩展】，添加需要的官方扩展库。



②缺少扩展库，可能会导致打开的程序缺少部分指令。

问题八：保存的音频文件找不到

解决方法：

视频、图片、音频等可以在软件文件管理处进行查看；好搭 AI 派为了界面美观，隐藏了这些文件，只显示 py 文件。

问题九：视频播放没声音

解决方法：

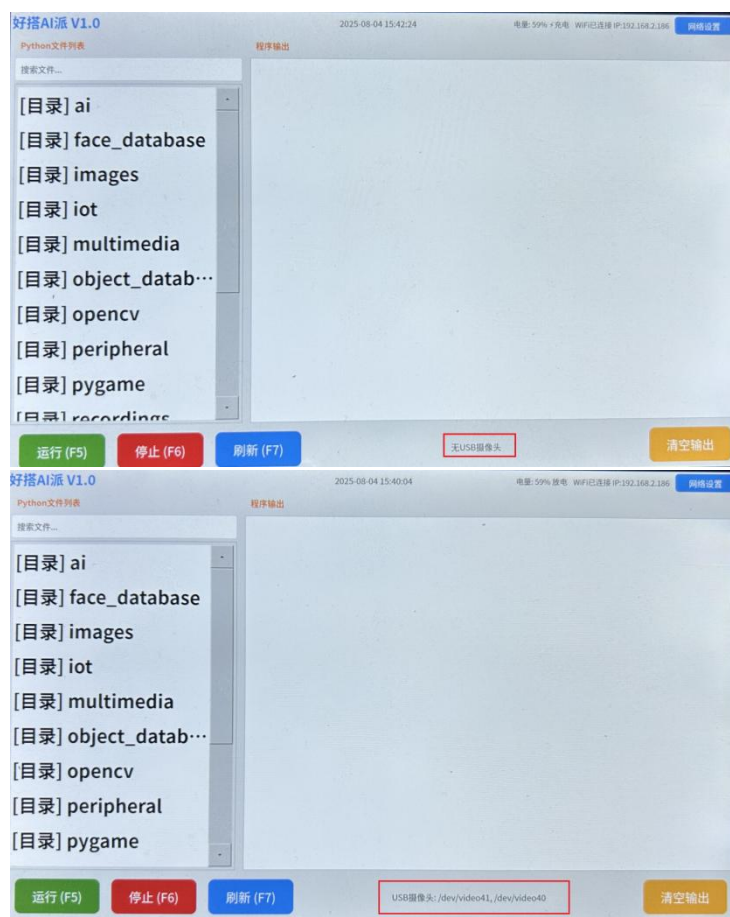
①视频播放需要配合音频一起播报。请先导出需要播放视频的音频文件，并保存到 recordings 文件夹中。具体可参考范例代码 OpenCV-4.播放视频与音频。

②主控的左上角有音量键，可能不小心把声音降至最低。

问题十：无法连接到摄像头

解决方法：

观察界面最下方，是显示无 USB 摄像头，还是显示 USB 摄像头：`/dev/video40`，`/dev/video41`（也有可能显示其他摄像头）。注意主控右下角的开关，必须拨动到左侧。如果还未显示正常连接，再检查连接摄像头的 USB 连接线。



问题十一：无法使用 AI 语音、大模型等功能

解决方法：

AI 语音、大模型相关功能，需要输入经过认证的用户名和密码，用户使用自己的好搭账号，联系售后经过认证后即可使用相关功能。

问题十二：外接设备没反应

解决方法：

注意主控右下角的开关，必须拨动到左侧，才能正常使用外接设备，拨动到右侧，则为充电模式。

问题十三：好搭 block 软件报错

```
adb.e: no devices/emulators found  
cannot connect to 192.168.2.146:5555: 由于连接方在一段时间后没有正确答复或连接的主机没有反应，连接尝试失败。 (10060)
```

解决方法：

未输入正常的 IP 地址，或者当前设备处于关机状态，导致 IP 网络连接失败。请查看正确的 IP 地址并进行 IP 网络连接。可尝试重启好搭 AI 派。

注意电脑和好搭 AI 派需要连接同一个网络。

问题十四：为什么有些指令找不到

解决方法：

部分指令在扩展库中。请先添加需要的官方扩展库。具体可以查看文档的【基础操作】-添加扩展。打开对应范例代码，会加载对应扩展。有其他扩展可能没有加载过，所以没有相关指令。扩展加载一次即可。

问题十五：不小心删除了文件夹怎么办

解决方法：

请及时联系技术人员。

问题十六：为什么点击运行程序没反应？

解决方法：

上一个程序可能还没有关闭；推荐先尝试点击停止按钮，停止上一个程序；再运行下一个程序。

问题十七：未找到可用串口设备

解决方法：

未找到可用串口设备

无法自动检测到 ESP32 设备，请手动指定串口

这两个报错通常一起出现，一般是因为右下角的开关按钮没有被拨动到左侧，但程序中使用了 ESP32 设备导致的。

重新拨动开关到右侧，稍等片刻，一般可以解决该问题。

如还未解决，可以尝试重启，或者直接联系官方售后人员。

问题十八：程序停止后电机/风扇不停止转动

解决方法：

电机或小风扇模块，不会因为程序停止就立刻停止。可以修改程序，确保风扇等模块停止后再关闭，又或者单独编写一个电机停止的程序，用于快速停止电机。

问题十九：文件夹不小心误删除怎么办？

解决方法：

点击软件右上角-更多按钮，选择文件恢复

